



32-Bit Arm® Cortex®-M33/M0+ 双核 Bluetooth® Low Energy 单片机

HT32F67595

开发使用手册

版本：V1.00 日期：2025-05-12

www.holtek.com

目录

1 电源管理控制单元 (PMU).....	8
简介	8
特性	9
功能描述	9
电源管理	9
低功耗管理	9
PMU API	12
PMU 中断 API	12
PMU 电源 API	14
PMU 时钟 API	20
PMU 范例	27
PMU 范例代码	27
2 通用 I/O (GPIO).....	28
简介	28
特性	28
功能描述	28
输入模式	29
输出模式	29
驱动能力	29
功能复用	29
中断功能	29
唤醒功能	30
GPIO API	30
GPIO ROM API	30
SW 端口 API 函数	37
IRAPI 函数	40
DCXO 引脚 API 函数	42
GPIO 范例	42
输入范例代码	42
输出范例代码	43
GPIO 中断范例代码	43
GPIO 唤醒范例代码	44
3 通用定时器 (GPTM).....	45
简介	45
特性	45
功能描述	46
计数器模式 (CNT 模式)	46
捕捉模式	47
PWM 模式	49

GPTM API.....	49
定时器功能 API.....	49
PWM 功能 API.....	60
定时器捕捉功能	64
GPTM 范例	68
计数器模式范例代码	68
PWM 模式范例代码.....	69
4 系统节拍定时器 (STIM).....	70
简介	70
特性.....	70
功能描述	70
时钟精准度和预分频	71
计数器、节拍、溢出和比较	71
STIM API	72
STIM 中断 API.....	72
STIM 唤醒 & 状态 API.....	74
STIM 计数器 & 比较	75
STIM 范例代码	78
STIM 唤醒 & 中断范例代码	78
STIM 溢出中断范例代码.....	79
5 实时时钟 (RTC)	80
简介	80
特性.....	80
功能描述	80
RTC 电源结构.....	81
时钟精准度和预分频	82
计数器、节拍、溢出和比较	82
RTC API	83
RCT 配置 API.....	83
RTC 中断 API	84
RTC 状态 API.....	87
RTC 计数器 & 比较.....	88
RTC 范例代码	91
RTC 开始工作范例代码.....	91
RTC 中断范例代码.....	91
唤醒范例代码	92
6 通用异步收发器 (UART).....	93
简介	93
特性.....	93
功能描述	93

UART API	93
UART 配置 API	93
UART 中断 API	97
UART FIFO API	99
UART RX/TX API	100
UART 范例代码	102
使用 UART1 的应用范例	102
7 IR 模块	104
简介	104
特性	104
功能描述	104
IR 编码	104
IR 解码	105
IR 模块 API	105
IR 编码 HW API	105
IR 编码 HAL API	108
IR 解码 HW API	110
IR 解码 HAL API	112
IR 模块范例	114
IR NEC 编码范例	114
IR 解码范例	115
8 2nd-Boot	117
存储器映射	117
2nd-Boot 代码	118
OTA 流程	118
初始化缓存读取	119
跳转到 SDK	120
9 IPC	121
简介	121
IPC API 介绍	121
基于消息队列实现的 IPC API 介绍	121
基于数组实现的 IPC API 介绍	122
IPC 范例代码	123
IPC 初始化 (MP 与 CP 相同)	123
IPC 消息接收回调函数	123
IPC 消息发送函数	123
10 低功耗蓝牙 GAP	125
简介	125
低功耗蓝牙 GAP API 接口	125
通用接口	125

广播接口	128
扫描接口	132
发起态接口	133
连接态接口	135
配对和加密连接相关接口	139
隐私功能相关接口	146
Stack 消息	149
范例	156
通用接口	156
广播	156
扫描	158
外设 (Peripheral)	160
中央设备 (Central)	164
11 低功耗蓝牙 GATT.....	166
简介	166
低功耗蓝牙 GATT Server API 接口	166
API 接口	166
特征的属性说明	172
写特征回调函数中操作模式说明	172
范例代码：透传 Profile	173
低功耗蓝牙 GATT Client API 接口	181
API 接口	181
回调函数	192
结构体定义	193
错误码	194
范例代码	194
附录	203
常用 UUID 参考	203

表列表

表 1. LUT 表 11

表 2. 查找表事件 ID 11

表 3. GPTM 模式选择 46

表 4. 时间间隔 71

表 5. 存储器映射 117

表
列
表

图列表

图 1. PMU 方框图 8

图 2. GPIO 基本结构图 28

图 3. GPTM 计数器寄存器 46

图 4. GPTM 预分频器 46

图 5. GPTM 工作模式时序图 47

图 6. STIM 参考设计原理图 70

图 7. 计数器触发溢出 71

图 8. RTC 参考设计原理图 81

图 9. RTC 电源参考设计原理图 81

图 10. 计数器工作流程 82

图 11. 计数器触发溢出 82

图 12. IR 编码 104

图 13. IR 解码 105

图 14. 引导码 114

图 15. Bit 说明 115

图 16. 重复码 115

图 17. 启动流程 117

图 18. 2nd-Boot 代码流程 118

图 19. Flash 存储器映射 118

图 20. 2nd-Boot 的 OTA 流程 119

图 21. 初始化缓存 119

图
列
表

1 电源管理控制单元 (PMU)

简介

HT32F67595 单片机具有完全集成的电源管理控制单元 (PMU)，包括一个具有一个输出的单电感单输出 (SISO) DC-DC 转换器，以及用于系统不同电源轨的多个 LDO。

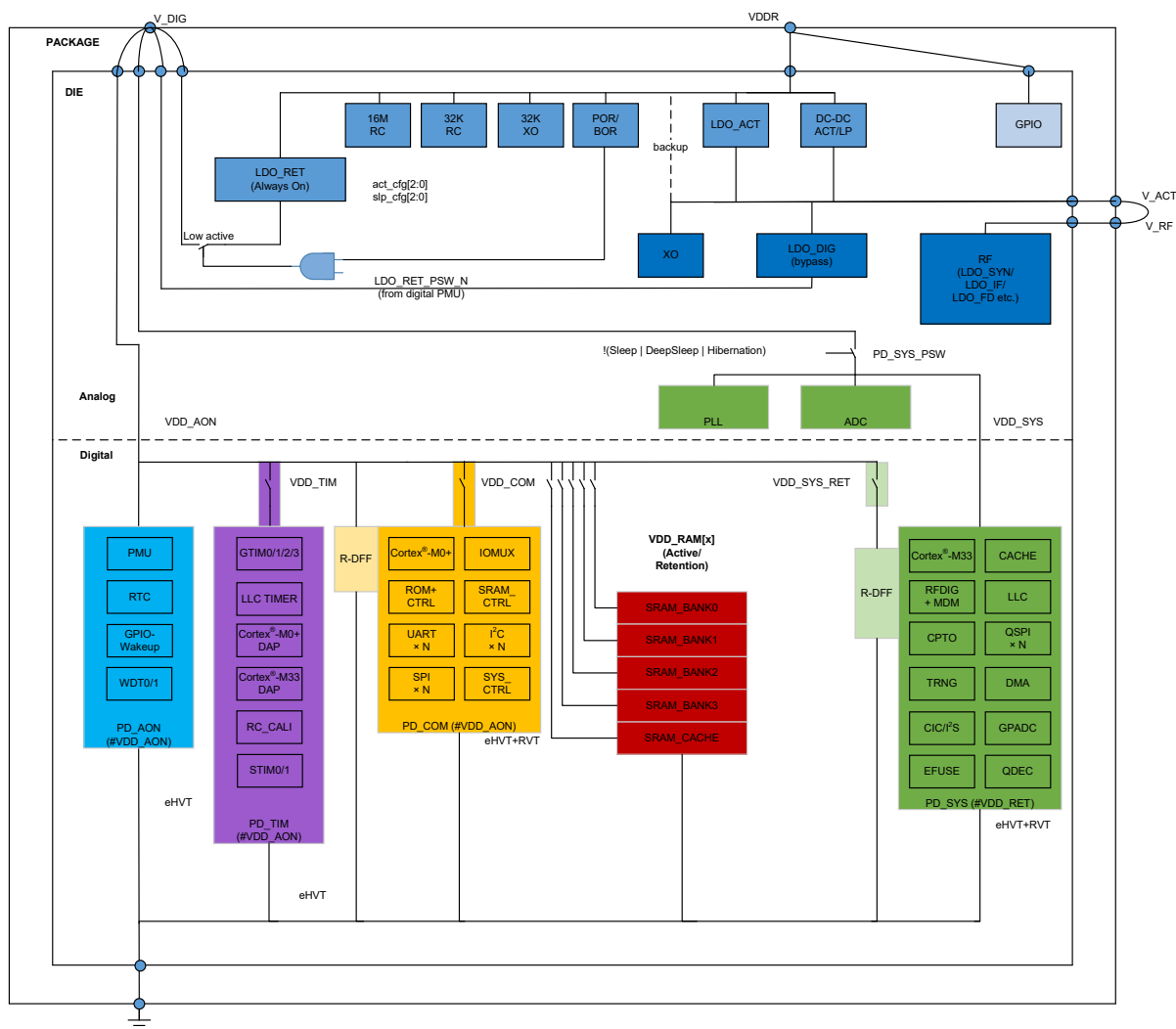


图 1. PMU 方框图

特性

- 支持多个 LDO：
 - LDO_ACT，可配置 0.95 V ~ 1.30 V
 - LDO_DIG，可配置 0.9 V ~ 1.25 V
 - LDO_RET，可配置 0.75 V ~ 1.1 V
- 支持 SISO DC-DC 转换器：
 - DCDC_ACT，可配置 0.95 V ~ 1.30 V
 - DCDC_RET，可配置 0.65 V ~ 1.05 V
 - DCDC_RET 时钟，可配置 1/2/4.../128 分频
- 多个时钟
 - 系统时钟：可配置 RC_HCLK (RC16M/RC48M)、DCXO16M、PLL64M
 - 低功耗时钟：可配置 RC32K、DXO32K
- 四种系统工作模式：
 - 活动模式：系统在正常模式下工作，可以配置不同的电源和系统时钟 (RC_HCLK/DCXO_HCLK/PLL)
 - 睡眠模式：系统时钟关闭，SRAM 保持保存功能
 - 深度睡眠模式：SRAM 关闭，低功耗时钟 (32K) 保持保存功能
 - 休眠模式：低功耗时钟 (32K) 关闭
- 支持在系统进入睡眠状态时可配置每个 SRAM 块保存功能
 - 支持 16 个查找表 (LUT) 用于唤醒
 - 支持电源监控单元、POR 和 BOR

功能描述

电源管理

该单片机支持两种电源模式：LDO 和 DC-DC。系统默认工作在 LDO 模式。电源模式切换只有在芯片唤醒后才会生效。DC-DC 模式需要外接电感，但系统功耗比 LDO 模式低。

低功耗管理

四种工作模式下各模块的工作情况如下：

	活动模式	睡眠模式	深度睡眠模式	休眠模式
CPU	活动	Off	Off	Off
FLASH	On	可用	Off	Off
SRAM	On	On	Off	Off
RADIO	On	Off	Off	Off
SRAM 保存	全部	部分	No	No
16M RC / DCXO	On	Off	Off	Off
32K RC / DCXO	On	On	On	Off
外设	可用	可用	Off	Off

	活动模式	睡眠模式	深度睡眠模式	休眠模式
从 RTC 唤醒	可用	可用	可用	Off
从 GPIO 唤醒	可用	可用	可用	可用

在活动模式下，应用程序 Arm® Cortex®-M33 或 Arm® Cortex®-M0+ CPU 正在主动执行代码。活动模式提供处理器和当前启用的所有外设的正常操作。系统时钟可以是任意可用的时钟源。

在睡眠模式下，可以对所有活动的外设进行时钟设置。但是应用程序的 CPU 核心和存储器没有时钟，也不执行任何代码。任何中断事件都会使处理器回到活动模式。

在深度睡眠模式下，只有始终开启的域处于活动状态。需要外部唤醒事件 RTC 事件才能将单片机恢复到活动模式。

在休眠模式下，单片机完全关闭，包括始终开启的域。I/O 与进入休眠模式之前的值一起锁存。任何 I/O 引脚上的状态变化(定义为从休眠引脚唤醒)都会唤醒单片机并用作复位触发器。

进入低功耗模式

由于 HT32F67595 是双核单片机，Cortex®-M33 和 Cortex®-M0+ 必须在芯片开始进入睡眠过程之前进入睡眠状态。Cortex®-M0+ 默认配置为低功耗模式。用户主要在 Cortex®-M33 中配置低功耗。

在 main.c 中，调用 lpwr_ctrl_init 函数进行低功耗初始化，如下所示：

lpwr_ctrl_init(enMode, lpwr_before_sleep, lpwr_after_wakeup);

enMode 指的是 EN_LPWR_MODE_SEL_T，如下所示：

```
typedef enum
{
    LPWR_MODE_ACTIVE = 0x00,
    LPWR_MODE_IDLE = 0x01,
    LPWR_MODE_SLEEP = 0x02, /* SRAM keep retention, system clock will power down. */
    LPWR_MODE_DEEPSLEEP = 0x03, /* SRAM will power down, LCLK clock will keep working (shutdown with 32K). */
    LPWR_MODE_HIBERNATION = 0x04, /* SRAM will power down, LCLK clock will turn off(shutdown without 32K). */
}EN_LPWR_MODE_SEL_T;
```

- 注：1. 如果 enMode 设置为 LPWR_MODE_ACTIVE，CPU 将保持活动，系统时钟不会关闭。
2. 如果 enMode 设置为 LPWR_MODE_IDLE，CPU 将保持空闲模式，系统时钟不会关闭。
- lpwr_before_sleep 是一个回调函数，用于检查系统是否可以进入睡眠模式。用户可以在这里添加应用程序代码来确定系统是否可以进入睡眠模式。当返回 0 时，系统不会进入睡眠模式，否则系统将进入睡眠模式。
 - lpwr_after_wakeup 是一个回调函数，是系统从睡眠模式唤醒后第一个执行的函数。用户可以在这里添加应用程序代码来重新初始化外设。
 - 当 lpwr_ctrl_goto_sleep() 被执行后，系统将进入 enMode 定义的低功耗模式。

系统唤醒

该单片机的 PMU 模块具有 13-bit (1 word) 查找表 (LUT)。它最多可以定义 16 个事件，当事件发生时将系统从低功耗模式 (睡眠 / 深度睡眠或休眠模式) 唤醒。

查找表 (LUT) 由 Cortex®-M33 在冷启动时初始化。指示 PDC 根据触发源激活哪些数字电源域。

表 1. LUT 表

	位	功能			
触发器	0	如果是 0x00 则为	如果是 0x01 则为	如果是 0x02 则为	如果是 0x03 则为
	1	P0 GPIO	P1 GPIO	P2 GPIO	外设
	2	Pin_ID	Pin_ID	Pin_ID	Periph_ID0
	3	Pin_ID	Pin_ID	Pin_ID	Periph_ID1
	4	Pin_ID	Pin_ID	Pin_ID	Periph_ID2
	5	Pin_ID	Pin_ID	Pin_ID	Periph_ID3
	6	Pin_ID	Pin_ID	Pin_ID	Periph_ID4
做什么	7	使能 PD_SYS			
	8	使能 DCXO16M			
	9	使能向 Cortex®-M0+ 发送中断			
	10	使能向 Cortex®-M33 发送中断			

LUT 的深度为 16 层，因此系统支持多达 16 种不同的唤醒配置，但可以通过应用软件动态更改。

当触发器根据每个 LUT 入口的 bit 0 和 bit 1 的值来触发时，会评估四种不同的触发类型：

- 如果 Bit[1:0] = 00，则是来自 P0 的 GPIO 翻转。Bit[6:2] 包含被触发的引脚编号；
- 如果 Bit[1:0] = 01，则是来自 P1 的 GPIO 翻转。Bit[6:2] 包含被触发的引脚编号；
- 如果 Bit[1:0] = 10，则是来自 P2 的 GPIO 翻转。Bit[6:2] 包含被触发的引脚编号；
- 如果 Bit[1:0] = 11，则是来某个外设的触发。Bit[6:2] 按以下方式定义外设：

表 2. 查找表事件 ID

ID (十进制)	外设
0	LUT_TRIG_ID_RTC_CH0
1	LUT_TRIG_ID_RTC_CH1
2	LUT_TRIG_ID_RTC_CH2
3	LUT_TRIG_ID_RTC_CH3
4	LUT_TRIG_ID_LLC
5	LUT_TRIG_ID_GTIM0
6	LUT_TRIG_ID_GTIM1
7	LUT_TRIG_ID_GTIM2
8	LUT_TRIG_ID_GTIM3
9	LUT_TRIG_ID_GPIO
10	LUT_TRIG_ID_CP_SWD
11	LUT_TRIG_ID_MP_SWD
12	保留
13	LUT_TRIG_ID_STIM0_CH0
14	LUT_TRIG_ID_STIM0_CH1
15	LUT_TRIG_ID_STIM0_CH2
16	LUT_TRIG_ID_STIM0_CH3
17	LUT_TRIG_ID_STIM1_CH0
18	LUT_TRIG_ID_STIM1_CH1
19	LUT_TRIG_ID_STIM1_CH2
20	LUT_TRIG_ID_STIM1_CH3
21	LUT_TRIG_ID_WDT1

注：ID 9 表示所有 GPIO 唤醒都有去抖，只有在睡眠和深度睡眠模式下才可设置。
Bit[1:0] = 00/01/10 表示所有 GPIO 唤醒都不去抖。
Bit[10:7] 解释了如前所述的触发源在触发后需要执行的操作。更具体地说，触发源可能会有以下请求：

- 使能 DCXO16M。如果应用程序（例如，低功耗蓝牙 (Bluetooth® Low Energy) 操作，需要 PLL64 的高性能模式）需要时钟精准度，则设置此位。
- 使能向 Cortex®-M0+ / Cortex®-M33 发送中断。外设 PMU 中断将发送到 Cortex®-M0+ 或 Cortex®-M33。

PMU API

PMU 中断 API

rom_hw_pmu_get_interrupt_flag

函数原型
EN_ERR_STA_T rom_hw_pmu_get_interrupt_flag (uint32_t* pu32IntMsk);
描述
通过读 PMU_LUT_INT_FLAG 寄存器来获取 PMU 中断标志 (状态)。
参数

参数	描述
pu32IntMsk	指示将读取哪个中断标志。
返回	
EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。

rom_hw_pmu_clear_interrupt_flag

函数原型
EN_ERR_STA_T rom_hw_pmu_clear_interrupt_flag (uint32_t u32IntMsk);
描述
通过写 PMU_LUT_INT_CLR 寄存器来清除 PMU 中断标志 (状态)。
参数

参数	描述
u32IntMsk	指示将清除哪个标志。
返回	
EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。

rom_hw_pmu_set_wakeup_source

函数原型

EN_ERR_STA_T rom_hw_pmu_set_wakeup_source (uint8_t u8LutIdx, PUM_LUT_TRIG_SEL_T enTrigSel, EN_LUT_TRIG_ID_T enLutTrigId, EN_LUT_ACT_T enLutAction);

描述

选择 PMU 唤醒源。

参数

参数	描述
u8LutIdx	LUT 索引，应小于 16。
enTrigSel	LUT 触发器选择，@ ref PUM_LUT_TRIG_SEL_T。
enLutTrigId	LUT 触发器 ID 选择，@ ref EN_LUT_TRIG_ID_T。
enLutAction	LUT 动作选择，@ ref EN_LUT_ACT_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_gpio_wakeup_source

函数原型

EN_ERR_STA_T rom_hw_pmu_set_gpio_wakeup_source (uint8_t u8LutIdx, PUM_LUT_TRIG_SEL_T enTrigSel, uint32_t u32Pin, EN_LUT_ACT_T enLutAction)

描述

设置 GPIO 从低功耗模式唤醒系统而不去抖。

参数

参数	描述
u8LutIdx	LUT 索引，需小于 16。
enTrigSel	LUT 触发器配置，必须选择 PMU_LUT_TRIG_GPIOA、PMU_LUT_TRIG_GPIOB、PMU_LUT_TRIG_GPIOC。
u32Pin	需要配置哪个引脚，一次只能配置一个引脚。
u8LutAction	LUT 动作配置，@ ref EN_LUT_ACT_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_sram_block_ret

函数原型

EN_ERR_STA_T rom_hw_pmu_set_sram_block_ret (uint32_t u32SramBlock);

描述

当系统进入低功耗模式时，配置 SRAM 存储器保持保存功能。

参数

参数	描述
u32SramBlock	将保持保存功能的 RAM 块。每个位对应一个 SRAM 块。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_clr_sram_block_ret

函数原型

EN_ERR_STA_T rom_hw_pmu_clr_sram_block_ret (uint32_t u32SramBlock);

描述

当系统进入睡眠模式时，清除指示的 SRAM 块保存功能。

参数

参数	描述
u32SramBlock	将关闭的 RAM 块。每个位对应一个 SRAM 块。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

PMU 电源 API

rom_hw_pmu_enable_ldo_act_output

函数原型

EN_ERR_STA_T rom_hw_pmu_enable_ldo_act_output (void);

描述

使能 LDO_ATC 输出。

当系统工作在活动模式时，LDO_ACT 输出为 CPU 供电。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_disable_ldo_act_output

EN_ERR_STA_T rom_hw_pmu_disable_ldo_act_output(void);

描述

除能 LDO_ATC 输出。

参见 rom_hw_pmu_enable_ldo_act_output。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_ldo_act_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_set_ldo_act_voltage(EN_LDO_ACT_VOLT_T enVolt);

描述

配置 LDO_ACT 输出电压，当系统工作在活动模式时，连接到 VDD_RF 为 RF 模块供电。

参数

参数	描述
enVolt	参考 EN_LDO_ACT_VOLT_T。 范围为 950 mV ~ 1300 mV，每阶 50 mV。 注：LDO_ACT 输出电压必须比 LDO_DIG 输出电压至少高 50 mV。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_enable_ldo_dig_output

函数原型

EN_ERR_STA_T rom_hw_pmu_enable_ldo_dig_output(void);

描述

使能 LDO_DIG 输出。

当系统工作在活动模式时，LDO_DIG 输出为数字模块供电。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_disable_ldo_dig_output

函数原型

EN_ERR_STA_T rom_hw_pmu_disable_ldo_dig_output(void);

描述

除能 LDO_DIG 输出。

参见 rom_hw_pmu_enable_ldo_dig_output。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_ldo_dig_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_set_ldo_dig_voltage(EN_LDO_DIG_VOLT_T enVolt);

描述

配置 LDO_DIG 输出电压。

参见 rom_hw_pmu_enable_ldo_dig_output。

参数

参数	描述
enVolt	LDO_DIG 定义的电压。 范围为 900 mV ~ 1100 mV，每阶 50 mV。 注：1. 当 SYS_CLK 工作在 16 MHz 以下时，LDO_DIG 输出电压可设置为 900 mV。 2. SYS_CLK 工作在 128 MHz 时，LDO_DIG 输出电压必须大于 1100 mV。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_get_ldo_dig_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_get_ldo_dig_voltage(uint8_t* pu8Volt);

描述

获取当前 LDO_DIG 输出电压配置。

参数

参数	描述
enVolt	获取 LDO 输出电压，@ ref EN_LDO_DIG_VOLT_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_ldo_dig_and_ret_act_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_set_ldo_dig_and_ret_act_voltage (EN_LDO_DIG_VOLT_T enVolt);

描述

同时配置 LDO_DIG、LDO_RET 和 LDC_ACT 输出电压。

参数

参数	描述
enVolt	LDO_DIG、LDO_RET 和 LDO_ACT 输出电压，范围为 900 mV ~ 1100 mV，每阶 50 mV。 更多信息参见 LDO_DIG、LDO_RET 和 LDO_ACT。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_ldo_ret_act_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_set_ldo_ret_act_voltage (EN_LDO_RET_VOLT_T enVolt);

描述

配置系统工作在活动模式时的 LDO_RET 输出电压。

参数

参数	描述
enVolt	系统工作在活动模式时的 LDO_RET 输出电压。范围为 750 mV ~ 1100 mV，每阶 50 mV。 注：当系统工作在活动模式时，请保持 LDO_RET 输出与 LDO_DIG 输出电压相同。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_ldo_ret_sleep_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_set_ldo_ret_sleep_voltage (EN_LDO_RET_VOLT_T enVolt);

描述

配置系统工作在睡眠模式时的 LDO_RET 输出电压。

参数

参数	描述
enSleepVolt	配置系统工作在睡眠模式时的 LDO_RET 输出电压。 范围为 750 mV ~ 1100 mV，每阶 50 mV。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_dcdc_init

函数原型

EN_ERR_STA_T rom_hw_pmu_dcdc_init(void);

描述

初始化 DC-DC 模块。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_enable_dcdc_act_output

函数原型

EN_ERR_STA_T rom_hw_pmu_enable_dcdc_act_output(void);

描述

当系统工作在活动模式时，使能 DC-DC 输出。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_disable_dcdc_act_output

函数原型

EN_ERR_STA_T rom_hw_pmu_disable_dcdc_act_output(void);

描述

使能指示的 DMA 控制器。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_dcdc_act_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_set_dcdc_act_voltage(EN_DCDC_ACT_VOLT_T enVolt);

描述

当系统工作在活动模式时，配置 DC-DC 模块的输出电压。

DC-DC 模块在 DC-DC 模式下为 RF 模块和数字模块供电。

参数

参数	描述
enVolt	DC-DC 输出电压。范围为 950 mV ~ 1300 mV，每阶 50 mV。 注：DC-DC 输出电压至少要比 LDO_DIG 输出电压高 50 mV。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_enable_dcdc_ret_output

函数原型

EN_ERR_STA_T rom_hw_pmu_enable_dcdc_ret_output(void);

描述

当系统工作在低功耗模式 (包括睡眠、深度睡眠和休眠模式) 时，使能 DC-DC 模块输出。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_disable_dcdc_ret_output

函数原型

EN_ERR_STA_T rom_hw_pmu_disable_dcdc_ret_output(void);

描述

当系统工作在低功耗模式 (包括睡眠、深度睡眠和休眠模式) 时，除能 DC-DC 模块输出。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_dcdc_ret_voltage

函数原型

EN_ERR_STA_T rom_hw_pmu_set_dcdc_ret_voltage(EN_DCDC_LPWR_VOLT_T enLpwrVolt);

描述

当系统工作在低功耗模式 (包括睡眠、深度睡眠和休眠模式) 时，配置 DC-DC 模块输出电压。

参数

参数	描述
enLpwrVolt	系统工作在低功耗模式时的 DC-DC 模块输出电压。范围为 650 mV ~ 1050 mV，每阶 50 mV。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

PMU 时钟 API

rom_hw_pmu_set_dcdc_ret_clk_divisor

函数原型

EN_ERR_STA_T rom_hw_pmu_set_dcdc_ret_clk_divisor (EN_DCDC_LPWR_CLK_T enDiv);

描述

配置 DC-DC 模块来自内部 RC_CLK 的时钟分频值。当系统工作在低功耗模式 (包括睡眠、深度睡眠和休眠模式) 时，DC-DC 模块时钟来自内部 RC_CLK。

参数

参数	描述
enDiv	来自内部 RC_CLK 的分频值。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_low_power_mode

函数原型

EN_ERR_STA_T rom_hw_pmu_set_low_power_mode (EN_PWR_MODE_T enMode);

描述

设置系统进入指示的低功耗模式，包括睡眠、深度睡眠和休眠模式。

参数

参数	描述
enMode	低功耗工作模式，@ ref EN_PWR_MODE_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_rc_lclk_is_enable

函数原型

bool rom_hw_pmu_rc_lclk_is_enable (void);

描述

检查内部 RC_CLK 是否使能。

参数

无

返回

status	FALSE(0) - RC_LCLK 除能； TURE(1) - RC_LCLK 使能。
--------	---

rom_hw_pmu_dcxo_lclk_is_power_on

函数原型

bool rom_hw_pmu_dcxo_lclk_is_power_on (void);

描述

检查 DCXO 低速时钟 (DCXO_LCLK) 是开启还是关闭。

参数

无

返回

status	FALSE(0) - DCXO_LCLK 开启; TURE(1) - DCXO_LCLK 关闭。
--------	---

rom_hw_pmu_dcxo_lclk_is_clk_out

函数原型

bool rom_hw_pmu_dcxo_lclk_is_clk_out (void);

描述

检查 DCXO 低速时钟 (DCXO_LCLK) 输出是否使能。

参数

无

返回

status	FALSE(0) - DCXO_LCLK 输出除能; TRUE(1) - DCXO_LCLK 输出使能。
--------	---

rom_hw_pmu_rc_hclk_is_power_on

函数原型

bool rom_hw_pmu_rc_hclk_is_power_on (void);

描述

检查 RC 高速时钟 (RC_HCLK) 是开启还是关闭。

参数

无

返回

status	FALSE(0) - RC_HCLK 关闭; TRUE(1) - RC_HCLK 开启。
--------	---

rom_hw_pmu_dcxo_hclk_is_clk_out

函数原型

bool rom_hw_pmu_dcxo_hclk_is_clk_out (void);

描述

检查 DCXO 高速时钟 (DCXO_HCLK) 输出是使能还是除能。

参数

无

返回

status	FALSE(0) - DCXO_HCLK 输出除能; TURE(1) - DCXO_HCLK 输出使能。
--------	---

rom_hw_pmu_dcxo_hclk_is_power_on

函数原型

bool rom_hw_pmu_dcxo_hclk_is_power_on(void);

描述

检查 DCXO 高速时钟 (DCXO_HCLK) 是开启还是关闭。

参数

无

返回

status	FALSE(0) - DCXO_HCLK 关闭; TRUE (1) - DCXO_HCLK 开启。
--------	--

rom_hw_pmu_rc_hclk_is_clk_out

函数原型

bool rom_hw_pmu_rc_hclk_is_clk_out(void);

描述

检查 RC 高速时钟 (RC_HCLK) 输出是使能还是除能。

参数

无

返回

status	FALSE(0) - RC_HCLK 输出除能; TURE(1) - RC_HCLK 输出使能。
--------	---

rom_hw_pmu_pll_clk_is_power_on

函数原型

bool rom_hw_pmu_pll_clk_is_power_on(void);

描述

检查 PLL 时钟 (PLL_CLK) 是开启还是关闭。

参数

无

返回

status	FALSE(0) - PLL_CLK 关闭; TURE(1) - PLL_CLK 开启。
--------	---

rom_hw_pmu_pll_clk_is_clk_out

函数原型

bool rom_hw_pmu_pll_clk_is_clk_out(void);

描述

检查 PLL 时钟 (PLL_CLK) 输出是使能还是除能。

参数

无

返回

status	FALSE(0) - PLL_CLK 输出除能; TRUE (1) - PLL_CLK 输出使能。
--------	--

rom_hw_pmu_turn_clk_power_on

函数原型

EN_ERR_STA_T rom_hw_pmu_turn_clk_power_on (EN_CLK_POWER_CTRL_T enCLK,
uint32_t u32TimeUs);

描述

开启指示的时钟电源。

参数

参数	描述
enCLK	指示将开启哪个时钟电源，@ ref EN_CLK_POWER_CTRL_T。
u32TimeUs	时钟的设置时间，单位：μs。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_turn_clk_power_off

函数原型

EN_ERR_STA_T rom_hw_pmu_turn_clk_power_off (EN_CLK_POWER_CTRL_T enCLK);

描述

关闭指示的时钟电源。

参数

参数	描述
enCLK	指示将关闭哪个时钟电源，@ ref EN_CLK_POWER_CTRL_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_enable_clk_output

函数原型

EN_ERR_STA_T rom_hw_pmu_enable_clk_output (EN_CLK_OUT_CTRL_T enCLK);

描述

使能指示的时钟输出。

参数

参数	描述
enCLK	指示将使能哪个时钟输出，@ ref EN_CLK_OUT_CTRL_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_disable_clk_output

函数原型

EN_ERR_STA_T rom_hw_pmu_disable_clk_output (EN_CLK_OUT_CTRL_T enCLK);

描述

除能指示的时钟输出。

参数

参数	描述
enCLK	指示将除能哪个时钟输出，@ ref EN_CLK_OUT_CTRL_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_dcxo_lclk_cfg_is_valid

函数原型

bool rom_hw_pmu_dcxo_lclk_cfg_is_valid (stDCXOBuf_t* pstBuf, stDCXOParam_t* pstParam);

描述

检查 DCXO 低速时钟 (DCXO_LCLK) 配置是否有效。

参数

参数	描述
pstBuf	指向 stDCXOBuf_t 结构的指针。
pstParam	指向 stDCXOParam_t 结构的指针。

返回

status	FALSE(0) - DCXO_LCLK 配置无效; TURE(1) - DCXO_LCLK 配置有效。
--------	---

rom_hw_pmu_set_dcxo_lclk_param

函数原型

EN_ERR_STA_T rom_hw_pmu_set_dcxo_lclk_param (stDCXOBuf_t* pstBuf, stDCXOParam_t* pstParam);

描述

配置 DCXO 低速时钟 (DCXO_LCLK, 32 kHz) 参数，包括逆变器、电流和负载电容。

参数

参数	描述
pstBuf	指向 stDCXOBuf_t 结构的指针，此结构包含以下成员： u8PosBuf, u8NegBuf: 逆变器数量。取值范围为 [0:7]。
pstParam	指向 stDCXOParam_t 结构的指针，此结构包含以下成员： u8Ib: DCXO 电流。取值范围为 [0:7]。 u8Ngm: 与振荡器有源部分并联的逆变器数量。取值范围为 [0:7]。建议 cfg 为 2 ~ 4。 u8Cap: DCXO 负载电容。单位: 0.1 pF，范围为 [0:255]，即 3.0 pF ~ 28.5 pF。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_rc_lclk_tune

函数原型

EN_ERR_STA_T rom_hw_pmu_set_rc_lclk_tune (uint8_t u8Val);

描述

配置 RC_LCLK 校准值，对 RC_LCLK 进行校准。

参数

参数	描述
u8Val	RC_LCLK 调整值，范围为 [0:255]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_rc_hclk_tune

函数原型

EN_ERR_STA_T rom_hw_pmu_set_rc_hclk_tune (uint8_t u8Val);

描述

配置校准值，对 RC_HCLK 进行校准。

参数

参数	描述
u8Val	RC_HCLK 调整值，范围为 [0:127]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_rc_hclk_sync_out

函数原型

EN_ERR_STA_T rom_hw_pmu_set_rc_hclk_sync_out (EN_RC_HCLK_SYNC_OUT_MODE_T enMode);

描述

开启 RC_HCLK 时，使能或除能 RC_HCLK 时钟输出。

参数

参数	描述
enMode	RC_HCLK 时钟输出模式。 b0 – 开启 RC_HCLK 后延迟 0.5 ~ 1.5 个 32K 时钟周期再输出 RC_HCLK。 b1 – 开启 RC_HCLK 时立即输出 RC_HCLK。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_sel_dcxo_hclk_pwr

函数原型

EN_ERR_STA_T rom_hw_pmu_sel_dcxo_hclk_pwr (EN_DCXO_HCLK_PWR_T enPwr);

描述

配置 DCXO 高速时钟 (DCXO_HCLK) 电源为 DC-DC 或 VDDR。

参数

参数	描述
enPwr	DCXO_HCLK 电源选择，@ ref EN_DCXO_HCLK_PWR_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_dcxo_hclk_stable_time

函数原型

EN_ERR_STA_T rom_hw_pmu_set_dcxo_hclk_stable_time (uint8_t u8Time);

描述

配置 DCXO 高速时钟 (DCXO_HCLK) 唤醒后的稳定时间，单位为 31.25 μ s。

参数

参数	描述
u8Time	稳定时间，范围为 [00:255]，单位为 31.25 μ s。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_pmu_set_dcxo_hclk_param

函数原型

EN_ERR_STA_T rom_hw_pmu_set_dcxo_hclk_param(stDCXOParam_t* pstParam);

描述

配置 DCXO 高速时钟 (DCXO_HCLK) 电流和负载电容。

参数

参数	描述
pstParam	指向 stDCXOParam_t 结构的指针，此结构包含以下成员： u8Ib: DCXO 电流。取值范围为 [0:7]。 u8Ngm: 与振荡器有源部分并联的逆变器数量。取值范围为 [0:7]。建议 cfg 为 2~4。 u8Cap: DCXO 负载电容。单位：0.1 pF，范围为 [0:255]，即 3.0 pF ~ 28.5 pF。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

PMU 范例

PMU 范例代码

```
/**
 * @ brief System power manage.
 * @ param enSel: Select DCDC or LDO, @ ref EN_PMU_POWER_SEL_T.
 */
static void system_power_init(EN_PMU_PWR_SEL_T enSel)
{
    // Set ldo_act voltage.
    rom_hw_pmu_set_ldo_act_voltage(EN_LDO_ACT_1200mV);
    // Init dcdc configuration and set dcdc_act voltage.
    patch_hw_pmu_dcdc_init();
    rom_hw_pmu_set_dcdc_act_voltage(EN_DCDC_ACT_VOLT_1200mV);
    // Set ldo_dig and ldo_ret voltage.
    rom_hw_pmu_set_ldo_dig_and_ret_act_voltage(EN_LDO_DIG_1100mV);
    rom_hw_pmu_set_ldo_ret_sleep_voltage(EN_LDO_RET_1100mV);
    // Power selection. It will be valid after the system
    // gets into sleep, default is ldo mode.
    rom_hal_pmu_sel_power_act_out_mode(enSel);
}
```

2 通用 I/O (GPIO)

简介

GPIO 是通用输入 / 输出端口 (General Purpose Input and Output port) 的简称, 可以通过软件控制其输入和输出。

特性

- 支持多达 16 个 I/O 引脚
- 输出状态: 浮空 + 上拉 / 下拉
- 输入状态: 施密特 / CMOS + 上拉 / 下拉
- 每个 I/O 支持单独的高 / 低电平设置
- 每个 I/O 支持读取当前电平状态 / 输出状态

功能描述

该单片机中的任何 GPIO 都支持输入和输出功能的软件配置、其他外设接口的多路复用配置、外部中断功能的配置, 并可配置 CPU 睡眠唤醒。

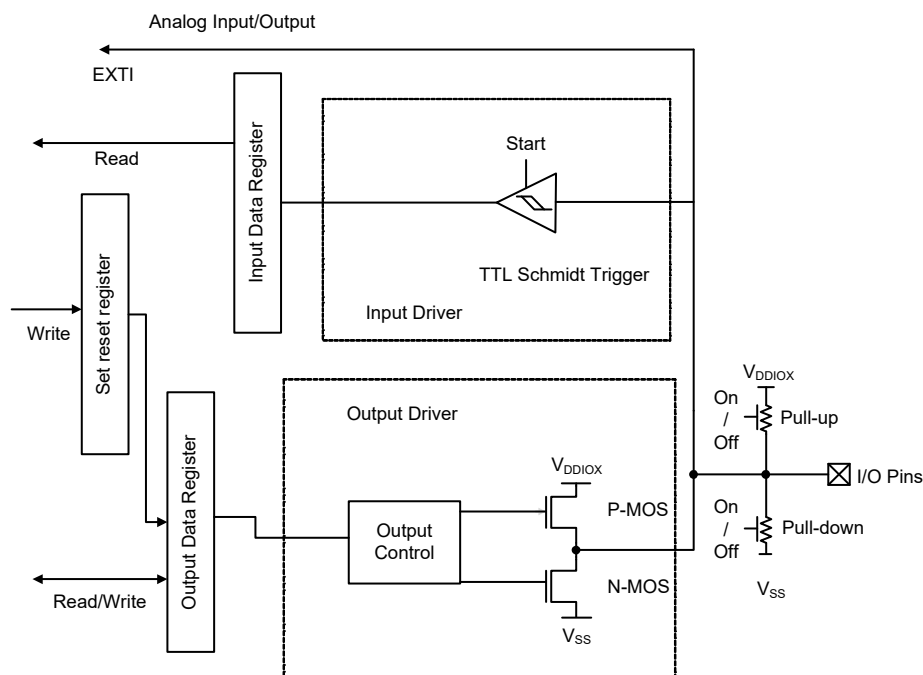


图 2. GPIO 基本结构图

如上图所示, 该模块的功能主要是通过不同的输入和输出配置来实现的。该单片机的每个 I/O 口都可以通过软件配置相应的 I/O 模式, 主要有以下几种配置模式:

输入	输入浮空	输入上拉	输入下拉	模拟输入	施密特 TTL 输入	施密特 CMOS 输入
输出	输出上拉	输出下拉				

输入模式

该单片机的每个 I/O 都可以用作输入，当 I/O 端口配置为输入时：

- 输出缓冲器关闭
- 施密特触发器开启
- 上拉电阻和下拉电阻根据寄存器状态开启或关闭
- 输入数据寄存器每 2 个 APB 时钟周期对 I/O 引脚上的数据进行采样
- 通过输入数据寄存器获取 I/O 状态

输出模式

该单片机的每个 I/O 输出不同的电平并读取输出状态，当 I/O 端口配置为输出时：

- 输出缓冲器开启
 - 当需要高输出电平时，PMOS 导通
 - 当需要低输出电平时，NMOS 导通
- 上拉电阻和下拉电阻根据寄存器状态开启或关闭
- 对输出状态寄存器进行读访问，以获取最后一次写入的值
- I/O 默认状态为高电阻下拉 (特殊要求的 I/O 除外，如 SWD、CLOCK 等)

驱动能力

单片机 GPIO 驱动能力的两个重要指标是拉电流 (源电流) 和填充电流 (灌电流)。此外，单个 I/O 的驱动能力和总 I/O 驱动能力也是衡量芯片驱动能力的重要指标。

- 拉电流 (源电流)：相对于单片机从外部输出电流的能力就是拉电流能力。例如，在单片机 I/O 和 GND 之间连接一个 1 K 负载电阻，单片机输出高电平并形成环路。电阻将电流从单片机内部拉出的过程就是拉电流过程。
- 灌电流：相对于单片机而言，主动吸收电流的能力就是灌电流能力。例如，在单片机 I/O 和 VCC 之间连接 1 K 负载电阻时，单片机输出低电平并形成环路。外部电源将电流注入单片机内部的过程称为填充电流过程。

该单片机的每个 I/O 有四个等级的可配置驱动能力，Level 0 ~ 3，其中 Level 0 的驱动能力最弱，Level 3 驱动能力最强。

功能复用

GPIO 功能多路复用是将通用 I/O 用于其他功能，如需要 SCL 和 SDA 功能的 I²C I/O。对于该单片机，任何 GPIO 都可以被单片机支持的外设使用，如 SPI、UART、I²C、SPI、PS 等。单片机为所有外设定义了一个唯一的 ID，并且每个 I/O 可以在初始化时设置为外设的相应 ID。

中断功能

任何 GPIO 都可以产生外部中断，生成的外部中断由 CPU 处理并配置到中断组中。该单片机支持 3 组外部中断，分别为 GPIO_INT0 / GPIO_INT1 / GPIO_INT2 (IRQ_20、IRQ21、IRQ22)。单片机支持任意 GPIO 产生外部中断，触发方式可通过软件配置。触发方式包括：

- 高电平触发
- 低电平触发
- 上升沿触发
- 下降沿触发
- 双沿触发

唤醒功能

任何 GPIO 都可以将芯片从休眠或睡眠模式中唤醒。通过配置相应的寄存器，使能引脚触发唤醒请求。当引脚的唤醒功能使能时，可以通过配置相应的寄存器使能输入滤波器 (去抖功能) 进行去抖。去抖电路可避免毛刺引起的误唤醒。此外，还可以配置 GPIO 唤醒请求的极性。

GPIO API

GPIO ROM API

rom_hw_gpio_set_pin_cfg

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_cfg(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, uint32_t u32Cfg);

描述

配置 GPIO 引脚。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚。
u32Cfg	引脚配置。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_pin_init

函数原型

EN_ERR_STA_T rom_hw_gpio_pin_init(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, uint32_t u32Cfg);

描述

初始化 GPIO。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚。
u32Cfg	引脚配置。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_pin_input_output

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_input_output(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, EN_GPIO_PIN_MODE_T enMode);

描述

配置 GPIO 工作在输入和输出模式。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
enMode	引脚输入或输出模式，@ ref EN_GPIO_PIN_MODE_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_pin_output_level

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_output_level(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, EN_GPIO_LEVEL_T enLevel);

描述

设置或清除指示引脚的输出电平。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
enLevel	输出电平状态，@ ref EN_GPIO_LEVEL_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_toggle_pin_output_level

函数原型

EN_ERR_STA_T rom_hw_gpio_toggle_pin_output_level(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin);

描述

翻转指示引脚的输出电平。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_get_pin_output_level

函数原型

EN_ERR_STA_T rom_hw_gpio_get_pin_output_level(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, uint32_t* pu32Level);

描述

获取指示引脚的输出状态。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
pu32Level	引脚状态，@ ref EN_GPIO_LEVEL_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_get_pin_input_level

函数原型

EN_ERR_STA_T rom_hw_gpio_get_pin_input_level(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, uint32_t* pu32Level);

描述

获取指示引脚的输入状态。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
pu32Level	引脚状态，@ ref EN_GPIO_LEVEL_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_pin_pid

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_pid(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, EN_GPIO_PID_T enPID);

描述

设置指示引脚的外设功能。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
enPID	引脚外设功能选择，@ ref EN_GPIO_PID_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_enable_qspi_pid

函数原型

EN_ERR_STA_T rom_hw_gpio_enable_qspi_pid(EN_QSPI_FIXED_GPIO_EN_T enCh);

描述

使能用于 QSPI 功能的 GPIO。

参数

参数	描述
enCh	QSPI 通道，@ ref EN_QSPI_FIXED_GPIO_EN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_disable_qspi_pid

函数原型

EN_ERR_STA_T rom_hw_gpio_disable_qspi_pid(EN_QSPI_FIXED_GPIO_EN_T enCh);

描述

除能在 QSPI 功能中工作的指示引脚。

参数

参数	描述
enCh	QSPI 通道，@ ref EN_QSPI_FIXED_GPIO_EN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_pin_pull_mode

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_pull_mode(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, EN_GPIO_PULL_MODE_T enMode);

描述

配置指示引脚的上拉或下拉模式。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
enMode	引脚上拉 / 下拉模式，@ ref EN_GPIO_PULL_MODE_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_pin_drive_strength

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_drive_strength(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, EN_GPIO_DRV_STRENGTH_T enStrength);

描述

配置指示引脚的驱动强度。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
enStrength	引脚驱动强度，@ ref EN_GPIO_DRIVE_STRENGTH_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_pin_interrupt_type

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_interrupt_type(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, EN_GPIO_INT_CH_T enCh, EN_GPIO_INT_TYPE_T enType);

描述

设置指示引脚的中断类型。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
enCh	中断句柄选择，@ ref EN_GPIO_INT_CH_T。
enType	中断类型，@ ref EN_GPIO_INT_TYPE_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_get_pin_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_gpio_get_pin_interrupt_flag(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, uint32_t* pu32Msk);

描述

获取指示引脚的中断标志 (状态)。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_clear_pin_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_gpio_clear_pin_interrupt_flag(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin);

描述

清除指示引脚的中断标志 (状态)。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_enable_pin_interrupt

函数原型

EN_ERR_STA_T rom_hw_gpio_enable_pin_interrupt(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin);

描述

使能指示引脚的中断。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_disable_pin_interrupt

函数原型

EN_ERR_STA_T rom_hw_gpio_disable_pin_interrupt(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin);

描述

除能指示引脚的中断。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_pin_wakeup_debounce

函数原型

EN_ERR_STA_T rom_hw_gpio_set_pin_wakeup_debounce(uint8_t u8Time, EN_GPIO_WAKEUP_DEBOUNCE_UNIT_T enUnit);

描述

设置所有引脚唤醒去抖，所有引脚共用一个唤醒去抖时间。

参数

参数	描述
u8Time	去抖时间，0 ~ 63。
enUnit	去抖单位，@ ref EN_GPIO_WAKEUP_DEBOUNCE_UNIT_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_enable_pin_wakeup

函数原型

EN_ERR_STA_T rom_hw_gpio_enable_pin_wakeup(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin, EN_GPIO_WAKEUP_POL_T enPol);

描述

使能指示引脚的唤醒功能。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。
enPol	唤醒极性，@ ref EN_GPIO_WAKEUP_POL_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_disable_pin_wakeup

函数原型

EN_ERR_STA_T rom_hw_gpio_disable_pin_wakeup(stGPIO_Handle_t* pstGPIO, uint32_t u32Pin);

描述

除能指示引脚的唤醒功能。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 GPIOA / GPIOB / GPIOC。
u32Pin	需要配置哪些引脚，@ ref EN_GPIO_PIN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_clear_wakeup_flag

函数原型

EN_ERR_STA_T rom_hw_gpio_clear_wakeup_flag(void);

描述

清除指示引脚的唤醒标志。

参数

无

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

SW 端口 API 函数

rom_hw_gpio_enable_swd

函数原型

EN_ERR_STA_T rom_hw_gpio_enable_swd(stGPIO_SWD_Handle_t* pstGPIO);

描述

使能 Cortex®-M0+ 或 Cortex®-M33 SWD 端口。如果 SWD 端口配置为通用 I/O，需要先除能 SWD 端口功能。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 SWD_CP / SWD_MP。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_disable_swd

函数原型

EN_ERR_STA_T rom_hw_gpio_disable_swd(stGPIO_SWD_Handle_t* pstGPIO);

描述

除能 Cortex®-M0+ 或 Cortex®-M33 SWD 端口。如果 SWD 端口配置为通用 I/O，需要先除能 SWD 端口功能。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 SWD_CP / SWD_MP。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_sw_input_output

函数原型

EN_ERR_STA_T rom_hw_gpio_set_sw_input_output(stGPIO_SWD_Handle_t* pstGPIO, uint8_t u8Pin, EN_GPIO_PIN_MODE_T enMode);

描述

配置 SWD 接口引脚 (SWDIO 或 SWCLK) 的输入或输出模式。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 SWD_CP / SWD_MP。
u8Pin	要配置的引脚，@ ref EN_GPIO_SW_NUM_T。
enMode	引脚输入或输出模式，@ ref EN_GPIO_PIN_MODE_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_sw_output_level

函数原型

EN_ERR_STA_T rom_hw_gpio_set_sw_output_level(stGPIO_SWD_Handle_t* pstGPIO, uint8_t u8Pin, EN_GPIO_LEVEL_T enLevel);

描述

设置指示引脚 (SWDIO 或 SWCLK) 的输出状态。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 SWD_CP / SWD_MP。
u8Pin	要配置的引脚，@ ref EN_GPIO_SW_NUM_T。
enLevel	输出电平状态，@ ref EN_GPIO_LEVEL_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_get_sw_input_level

函数原型

EN_ERR_STA_T rom_hw_gpio_get_sw_input_level(stGPIO_SWD_Handle_t* pstGPIO, uint8_t u8Pin, uint8_t* pu8Level);

描述

获取指示引脚 (SWDIO 或 SWCLK) 的输入状态。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 SWD_CP / SWD_MP。
u8Pin	要配置的引脚，@ ref EN_GPIO_SW_NUM_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_sw_pull_mode

函数原型

EN_ERR_STA_T rom_hw_gpio_set_sw_pull_mode(stGPIO_SWD_Handle_t* pstGPIO, uint8_t u8Pin, EN_GPIO_PULL_MODE_T enMode);

描述

配置 SWD 接口引脚 (SWDIO 或 SWCLK) 的上拉模式或下拉模式。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 SWD_CP / SWD_MP。
u8Pin	要配置的引脚，@ ref EN_GPIO_SW_NUM_T。
enMode	引脚上拉 / 下拉模式，@ ref EN_GPIO_PULL_MODE_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_sw_drive_strength

函数原型

EN_ERR_STA_T rom_hw_gpio_set_sw_drive_strength(stGPIO_SWD_Handle_t* pstGPIO, uint8_t u8Pin, EN_GPIO_DRV_STRENGTH_T enStrength);

描述

配置 SWD 接口引脚 (SWDIO 或 SWCLK) 的驱动强度。

参数

参数	描述
pstGPIO	GPIO 外设句柄，应为 SWD_CP / SWD_MP。
u8Pin	要配置的引脚，@ ref EN_GPIO_SW_NUM_T。
enStrength	引脚驱动强度，@ ref EN_GPIO_DRV_STRENGTH_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

IR API 函数

rom_hw_gpio_enable_ir_tx_out

函数原型

EN_ERR_STA_T rom_hw_gpio_enable_ir_tx_out(void);

描述

使能 IR TX 输出。

参数

无

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_disable_ir_tx_out

函数原型

EN_ERR_STA_T rom_hw_gpio_disable_ir_tx_out(void);

描述

除能 IR TX 输出。

参数

无

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_enable_ir_rx_amp

函数原型

EN_ERR_STA_T rom_hw_gpio_enable_ir_rx_amp(void);

描述

使能 IR RX AMP。

参数

无

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_disable_ir_rx_amp

函数原型

```
EN_ERR_STA_T rom_hw_gpio_disable_ir_rx_amp(void);
```

描述

除能 IR RX AMP。

参数

无

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_set_ir_rx_rtune

函数原型

```
EN_ERR_STA_T rom_hw_gpio_set_ir_rx_rtune(uint8_t u8Rtune);
```

描述

设置 RC 滤波器中 R 的阻值。

参数

参数	描述
u8Rtune	$R = 50\text{ K} \times 2^{(u8Rtune)}$ 。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_gpio_get_ir_ctrl_value

函数原型

```
EN_ERR_STA_T rom_hw_gpio_get_ir_ctrl_value(uint32_t* pu32IrCtrlValue);
```

描述

获取 IR 控制寄存器值。

参数

参数	描述
pu32IrCtrlValue	指向 32-bit 缓冲器的指针，用于返回 IR CTRL 寄存器值。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

DCXO 引脚 API 函数

rom_hw_gpio_enable_dcxo32k_output

函数原型

EN_ERR_STA_T rom_hw_gpio_enable_dcxo32k_output(uint8_t u8Io);

描述

使能 PB07 (P39) 或 (和) PB10 (P42) 的 DCXO32K 时钟输出。

参数

参数	描述
u8Io	输出 I/O, @ ref EN_DCXO32K_OUT_CFG_T。

返回

HW status	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
-----------	-------------------------------

rom_hw_gpio_disable_dcxo32k_output

函数原型

EN_ERR_STA_T rom_hw_gpio_disable_dcxo32k_output(uint8_t u8Io);

描述

除能 PB07 (P39) 或 (和) PB10 (P42) 的 DCXO32K 时钟输出。

参数

参数	描述
u8Io	输出 I/O, @ ref EN_DCXO32K_OUT_CFG_T。

返回

HW status	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
-----------	-------------------------------

GPIO 范例

上一节阐述了 GPIO 的主要函数, 本节将基于 GPIO 的主要函数开发 DEMO 工程。

输入范例代码

输入工程主要用于测试 GPIO 能否通过 I/O 口读取外部传输的信号。范例代码如下所示。

```
uint32_t i;
uint32_t u32Level = 0;
rom_hw_gpio_set_pin_pull_mode(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO, enPullMode);
rom_hw_gpio_set_pin_input_output(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO,
                                GPIO_MODE_INPUT);

for (i = 0; i < 16; i++)
{
    rom_hw_gpio_get_pin_input_level(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO,
                                   &u32Level);
    PRINTF ("GPIO input level: %08X\n", u32Level);
    rom_delay_ms (1000);
}
```

输出范例代码

输出范例工程主要用于测试 GPIO 是否可以通过 I/O 口向外传输信号。范例代码如下所示。

```
uint32_t u32Level;
rom_hw_gpio_set_pin_input_output(GPIO_PORT_OUTPUT_IO, GPIO_PIN_OUTPUT_IO,
                                  GPIO_MODE_OUTPUT);
rom_hw_gpio_set_pin_drive_strength(GPIO_PORT_OUTPUT_IO, GPIO_PIN_OUTPUT_IO,
                                    enStrength);
rom_hw_gpio_set_pin_output_level(GPIO_PORT_OUTPUT_IO, GPIO_PIN_OUTPUT_IO, 1);
rom_hw_gpio_get_pin_output_level(GPIO_PORT_OUTPUT_IO, GPIO_PIN_OUTPUT_IO,
                                  &u32Level);
PRINTF("(1) GPIO output level: %08X\n", u32Level);
rom_delay_ms(1000);
rom_hw_gpio_set_pin_output_level(GPIO_PORT_OUTPUT_IO, GPIO_PIN_OUTPUT_IO, 0);
rom_hw_gpio_get_pin_output_level(GPIO_PORT_OUTPUT_IO, GPIO_PIN_OUTPUT_IO,
                                  &u32Level);
PRINTF("(2) GPIO output level: %08X\n", u32Level);
```

GPIO 中断范例代码

该范例工程主要用于测试是否能够响应通过 GPIO 产生的外部中断。范例工程如下所示。

中断处理程序

```
uint32_t u32Flag, u32Level;
/* Get interrupt status */
rom_hw_gpio_get_pin_interrupt_flag(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO,
                                    &u32Flag);
rom_hw_gpio_clear_pin_interrupt_flag(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO);
PRINTF("io int flag: %08X\n", u32Flag);
rom_hw_gpio_get_pin_input_level(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO, &u32Level);
PRINTF("io int level: %08X\n", u32Level);
```

中断 Demo

```
rom_hw_gpio_set_pin_input_output(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO, GPIO_
                                MODE_INPUT);
if (GPIO_INT_HIGH_LEVEL == enType)
{
    rom_hw_gpio_set_pin_pull_mode(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO, G PIO_
    PULL_DOWN);
}
else
{
    rom_hw_gpio_set_pin_pull_mode(GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO, G PIO_
    PULL_UP);
}
/* Configure for gpio interrupt */
/* Registration interruption */
rom_hw_sys_ctrl_enable_peri_int (SYS_CTRL_MP, GPIO_IRQ0);
/* Enable gpio1 interrupt */
NVIC_ClearPendingIRQ (GPIO_IRQ0);
NVIC_SetPriority (GPIO_IRQ0, 0x3);
NVIC_EnableIRQ (GPIO_IRQ0);
/* Disable gpio interrupt */
rom_hw_gpio_disable_pin_interrupt (GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO);
/* Initialize io to interrupt mode. */
rom_hw_gpio_set_pin_interrupt_type (GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO,
                                GPIO_INT_CH0, enType);

/* Enable gpio interrupt */
rom_hw_gpio_enable_pin_interrupt (GPIO_PORT_INPUT_IO, GPIO_PIN_INPUT_IO);
```

GPIO 唤醒范例代码

该范例工程项目用于测试是否可以通过 GPIO 唤醒处于睡眠模式的 CPU。

```
#define WAKEUP_PIN GPIO_21
/* Configure wake-up source */
rom_hw_pmu_set_gpio_wakeup_source(n, PMU_LUT_TRIG_GPIOB, WAKEUP_PIN, LUT_ACT_PD_
                                SYS_ON);

/* Configure debounce time */
rom_hw_gpio_set_pin_wakeup_debounce(60, GPIO_WAKEUP_DEB_UNIT_30US);
/* enable wakeup */
rom_hw_gpio_enable_pin_wakeup(GPIOB, WAKEUP_PIN, GPIO_WAKEUP_HIGH_LEVEL);
```

3 通用定时器 (GPTM)

简介

通用定时器 GPTM，也称为 GTIM，是一个 32-bit 定时器，配置了一个 4 通道比较器以使能定时器功能。GPTM 工作在 32K 时钟域，当单片机进入睡眠模式后，GPTM 仍然可以运行并唤醒系统。GPTM 引入了一种灵活的时钟方案，可提供所需的功能和性能，同时较大限度地降低功耗。

特性

- 可使用 16 MHz / 32 kHz 时钟或 GPIO 触发源作为时钟源
- 中断类型：cpu_cap / 溢出 / 匹配 / capture_coverd / 解码
- 2 路 PWM 和组合 32 位 PWM
- 1 路红外发射
- 2 路 GPIO / IR 触发捕捉功能
 - 支持 CNT_A / CNT_B 捕捉 (不支持同时)
 - 支持组合 32 位捕捉 (仅 CNT_A 使能)
- 2 路 GPIO / IR 触发解码功能
 - 支持 CNT_A / CNT_B 解码 (支持同时)
 - 支持组合 32 位解码 (仅 CNT_A 使能)
- 支持解码捕捉环回 (CNT_A 解码, CNT_B 捕捉)
- 可作为一个 32-bit 计数器或两个 16-bit 分频计数器使用
- 支持中断捕捉模式 (与 DMA 捕捉模式同时只能选择一种)
- 支持捕捉电平指示 (在 16-bit 模式下 capcnt[15] 表示 CNT_B 高电平 (为 0 时) 和低电平 (为 1 时) 的计数值, capcnt[31] 表示 CNT_A 电平的计数值, 在 32-bit 模式下 capcnt[31] 表示组合 32 位 cnt 电平的计数值)
- 支持多个 GPTM 同时启动和停止
- 比较值在下个周期生效

功能描述

该单片机主要具有计数、产生 PWM 波形和采样功能。

计数器模式 (CNT 模式)

当 GPTM 工作在计数器模式时，32-bit 计数器寄存器包含两个 16-bit 计数器，CNT_A 和 CNT_B，如下图所示：

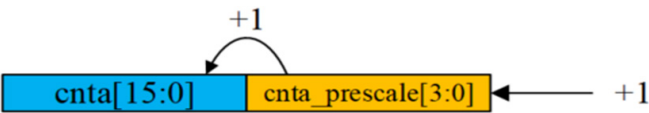


图 3. GPTM 计数器寄存器

如上图所示，当 CNT_A 计数输入有效时，cnta_prescale 先计数，当 prescale 计数达到设定值时，cnta 加 1。

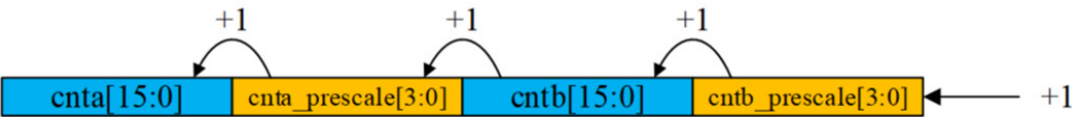


图 4. GPTM 预分频器

当两个 cnt 组合使用时，cnta_prescale 的值应设为 0。cntb_prescale 先计数，当计数达到设定值时，CNT_B 加 1。当 CNT_B 满时，将它送入 cnta_prescale，当 cnta_prescale 计数到设定值后，再将它送入 CNT_A。

计数以递增方式开始，寄存器控制变化点是比较点 (大于零的比较值被认为是有效值) 还是溢出点，以及到达变化点后的变化是复位 (递增方式复位为 0，递减方式复位为全 f) 还是渐变 (递增改为递减，递减改为递增)。比较值设置原理如下表所示。

表 3. GPTM 模式选择

rst_mode	rstval_mode	递增	递减
0	0	计数到 compare 后继续计数，当达到 16'hffff_ffff 时复位为 0。	计数到 compare 后继续计数，当达到 0 时复位为 16'hffff_ffff。
0	1	当达到 compare 时，计数复位为 0，并继续递增。	当达到 compare 时，计数复位为 16'hffff_ffff，并继续递减。
1	0	计数到 compare 后继续计数，当达到 16'hffff_ffff 时变为递减。	计数到 compare 后继续计数，当达到 0 时变为递增。
1	1	计数到 compare 后变为递减。	计数到 compare 后变为递增。

上表显示了不同模式下的工作原理。为了更形象地说明原理，下面将给出不同模式下的时序图。

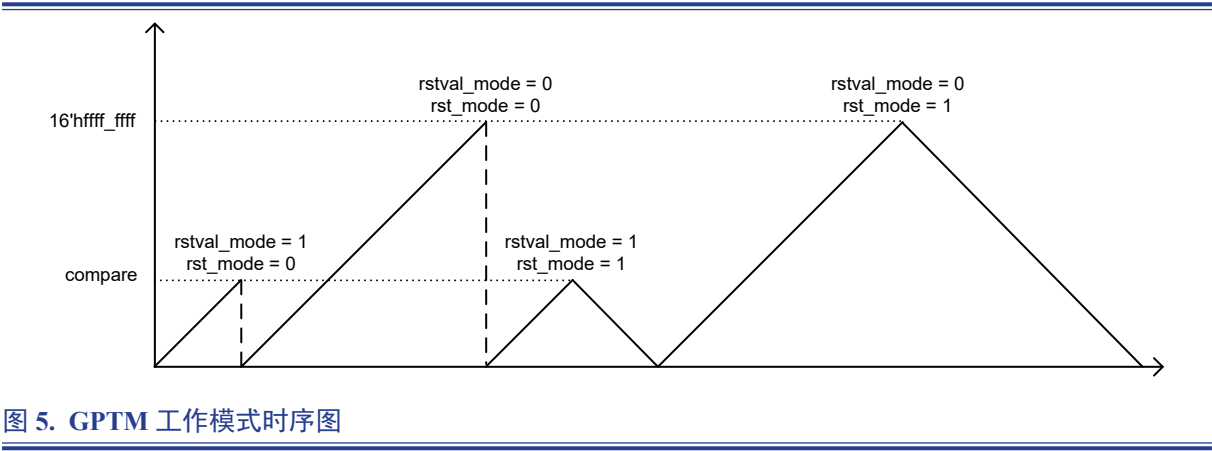
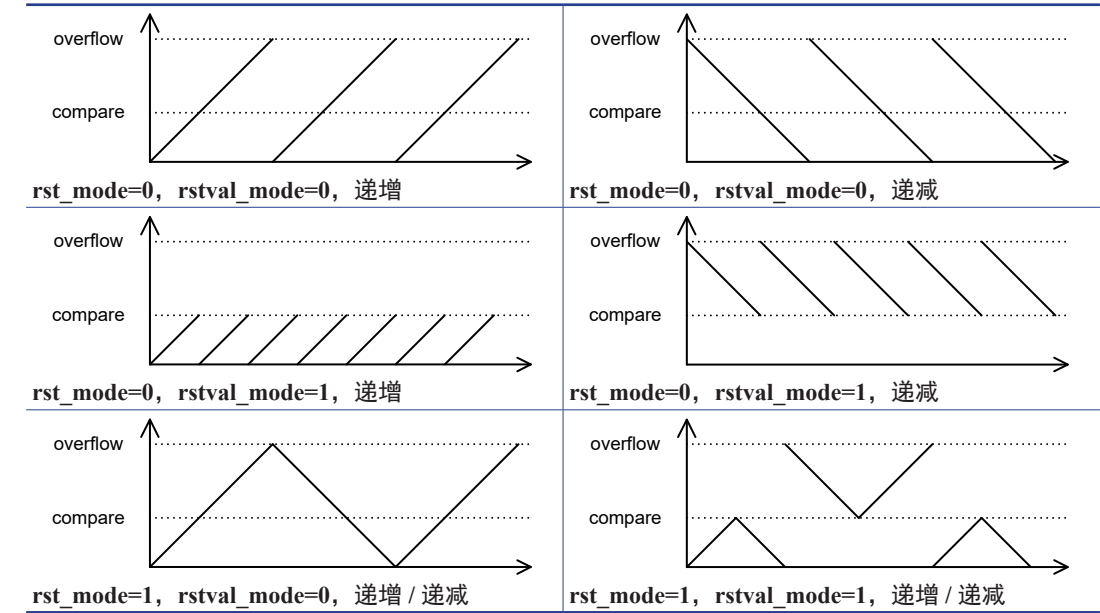


图 5. GPTM 工作模式时序图

通用功能时序说明

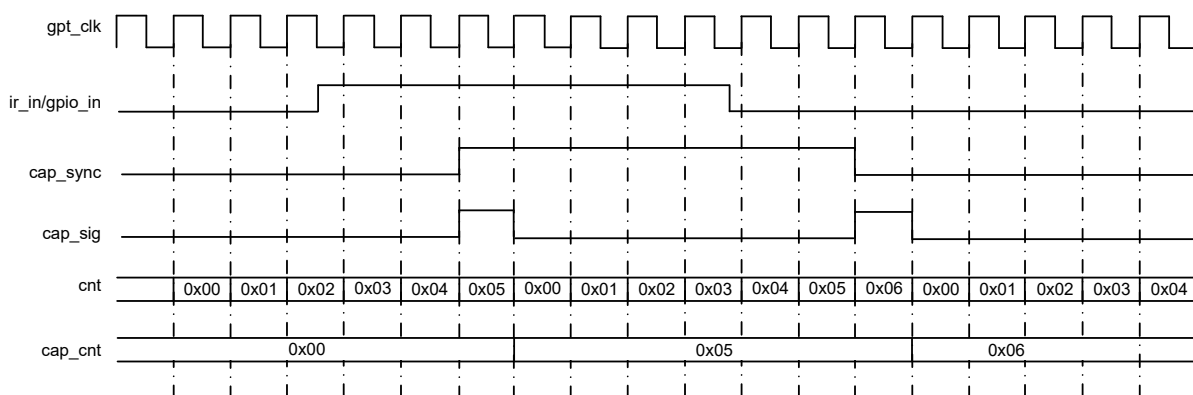


捕捉模式

捕捉功能用于检测 IR 输入信号或 GPIO 输入信号的边沿间隔。

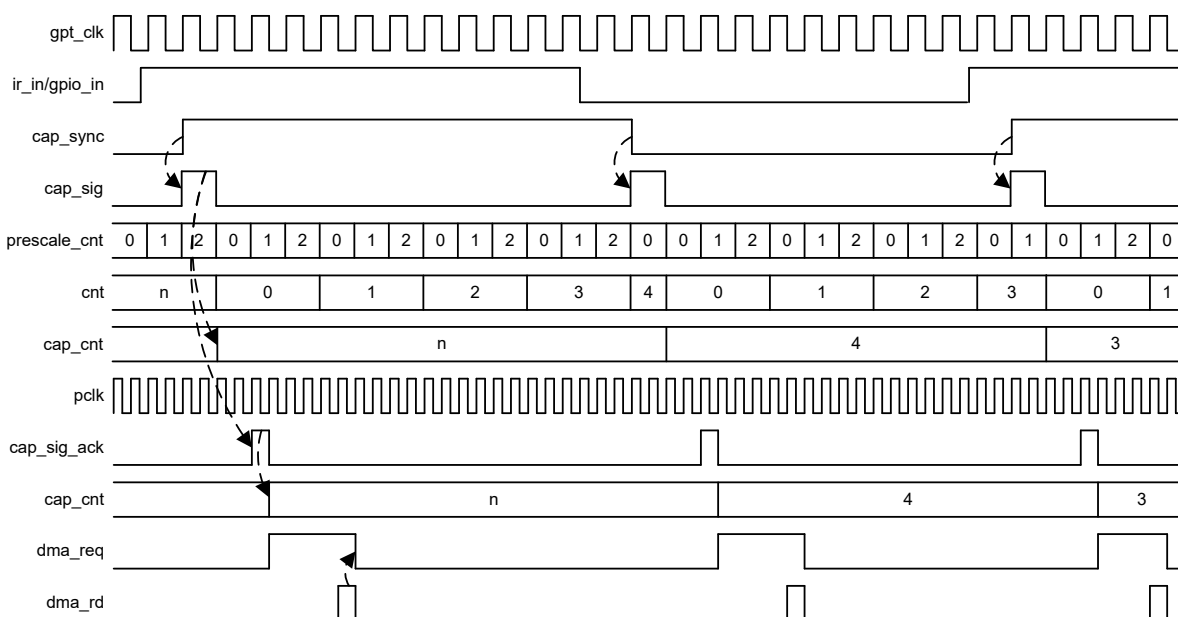
输入信号同步后，进行边沿检测，可选择上升沿、下降沿或双边沿采样。对于双边沿采样，可以使用电平指示功能将最高位作为电平指示位。每个采样点首先将 cnt 值保存到 cnt_tmp，并将 cnt 值清零，然后产生一个 dma 请求信号请求 dma 携带数据，或者使用中断捕捉方式产生中断，CPU 读取 cnt 数据。

如果在请求生成和数据处理之间的时间段内出现一个新的采样点，则忽略该采样点并报告 capcov 中断。需注意的是，采用点会将 cnt 清零，但不会覆盖 cnt_tmp 值。捕捉模式的基本功能如下图所示。



从上图可以看出，在 cnt 不除频的情况下，由于清零动作需要占用一个周期或者信号边沿本身占用一个周期，所以实际计数两个信号边沿从 0x0 到 0x6 共 7 个周期，此时可以直接使用 $\text{cap_cnt} + 1$ 作为实际长度。

当有预分频频率时，如 3 分频，如下图所示，由于除法本身造成的误差，cap_sig 可能到达除法计数器 prescale_cnt 的 2、0 或 1，那么实际的计数周期 (gpt_clk) 应分别是 $\text{cap_cnt} \times 3 + 3$ 、 $\text{cap_cnt} \times 3 + 1$ 、 $\text{cap_cnt} \times 3 + 2$ ，此时不能直接使用 $\text{cap_cnt} + 1$ 作为实际的 cnt 计数值。



捕捉新的附加电平指示功能：由于双边沿采样难以区分上下边沿，因此占用 1-bit 最高位作为电平指示（非双边沿采样请关闭此功能），16-bit cnt 计数时，bit[15] 作为电平指示位，32-bit cnt 计数时，位 [31] 作为电平指示位，1 表示高电平。当使用这个功能时，compare 应配置为 15' h7fff (16-bit cnt) 或 31' h7fff_fff 作为溢出中断，因为最高位被占用了。

PWM 模式

GPTM 模块也用于生成指定的波形。其工作原理描述如下。

比较值是计数周期，当计数值在 `pwm_low` 和 `pwm_hi` 之间时输出指定的电平（高低电平由 `pwm_pol` 决定）。

`pwm_low` 和 `pwm_hi` 的值在配置完成触发器开始后第一次生效，其次在比较周期结束后生效。比较值可以在当前周期或下一个周期生效。

注： $pwm_low \leq cnt < pwm_hi$ 输出指定的电平，所以配置占空比为 0%：`pwm_low` 和 `pwm_hi` 都大于 `compare` 或 $pwm_low \geq pwm_hi$ ；配置占空比为 100%：`pwm_low` = 0，`pwm_hi` 大于 `compare`（`compare` 配置为全 f，不能产生 100% 占空比）。

GPTM API

定时器功能 API

`rom_hw_timer_trig_cfg_valid`

函数原型

`EN_ERR_STA_T rom_hw_timer_trig_cfg_valid(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);`

描述

使定时器的配置有效。

参数

参数	描述
<code>pstTIMER</code>	TIMER 句柄，应为 <code>TIMER0</code> / <code>TIMER1</code> / <code>TIMER2</code> / <code>TIMER3</code> 。
<code>enCh</code>	指示将配置哪个通道，参考 <code>EN_TIMER_CH_T</code> 枚举定义。

返回

HW status	函数返回状态，参考 <code>EN_ERR_STA_T</code> 枚举定义。
-----------	---

`rom_hw_timer_set_work_mode`

函数原型

`EN_ERR_STA_T rom_hw_timer_set_work_mode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);`

描述

通过写入具有指示位的 `TIMER_MODE` 寄存器来选择定时器工作模式。

参数

参数	描述
<code>pstTIMER</code>	TIMER 句柄，应为 <code>TIMER0</code> / <code>TIMER1</code> / <code>TIMER2</code> / <code>TIMER3</code> 。
<code>enCh</code>	指示将配置哪个通道，参考 <code>EN_TIMER_CH_T</code> 枚举定义。

返回

HW status	函数返回状态，参考 <code>EN_ERR_STA_T</code> 枚举定义。
-----------	---

rom_hw_timer_get_mode

函数原型

EN_ERR_STA_T rom_hw_timer_get_mode(stTIMER_Handle_t* pstTIMER, uint32_t* pu32Mode);

描述

获取定时器的模式。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
pu32Mode	指向定时器模式配置。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_clock_prescale

函数原型

EN_ERR_STA_T rom_hw_timer_set_clock_prescale(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint8_t u8Prescale);

描述

配置指示定时器的时钟预分频。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
u8Prescale	输入时钟的分频，u8precale 的范围为 0 ~ 0x0F。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_counter_mode

函数原型

EN_ERR_STA_T rom_hw_timer_set_counter_mode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_TIMER_CNT_MODE_T enMode);

描述

配置指示定时器工作在计数器模式。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
enMode	定时器计数器模式控制，参考 EN_TIMER_CNT_MODE_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_cfg

函数原型

EN_ERR_STA_T rom_hw_timer_get_cfg(stTIMER_Handle_t* pstTIMER, uint32_t* pu32Cfg);

描述

获取定时器配置。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
pu32Cfg	指向定时器配置。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_start

函数原型

EN_ERR_STA_T rom_hw_timer_start(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

启动指示的定时器。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_stop

函数原型

EN_ERR_STA_T rom_hw_timer_stop(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

停止指示的定时器。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_counter

函数原型

```
EN_ERR_STA_T rom_hw_timer_get_counter(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t* pu32Cnt);
```

描述

获取指示的定时器当前计数器。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
pu32Cnt	当前定时器计数器。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_clear_counter

函数原型

```
EN_ERR_STA_T rom_hw_timer_clear_counter(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);
```

描述

清除指定的定时器计数器。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_compare

函数原型

```
EN_ERR_STA_T rom_hw_timer_set_compare(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32Value);
```

描述

配置指定的定时器比较值。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
u32Value	比较值。 16-bit 定时器：比较值范围为 0 ~ 0xFFFF。 32-bit 定时器：比较值范围为 0 ~ 0xFFFFFFFF。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_compare

函数原型

EN_ERR_STA_T rom_hw_timer_get_compare(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t* pu32Value);

描述

获取指示的定时器比较值。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
pu32Value	指向比较值。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_timer_get_interrupt_flag(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t* pu32Msk);

描述

通过读取 TIMER_INT_FLAG 寄存器来获取定时器中断标志 (状态)。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
pu32Msk	指示将读取哪个中断标志。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_clear_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_timer_clear_interrupt_flag(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32Msk);

描述

通过写入 TIMER_INT_CLR 寄存器来清除指示的定时器中断标志 (状态)。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
pu32Msk	指示将清除哪个标志。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_enable_interrupt

函数原型

EN_ERR_STA_T rom_hw_timer_enable_interrupt(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32IntEn);

描述

通过写入具有指示位的 TIMER_INT_EN 寄存器来使能指定的定时器中断。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
u32IntEn	指示将使能哪个中断。 Bit 为 1 表示使能 Bit 为 0 表示不影响 参考 EN_TIMER_INT_FLAG_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_disable_interrupt

函数原型

EN_ERR_STA_T rom_hw_timer_disable_interrupt(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32IntDis);

描述

通过写入具有指示位的 TIMER_INT_EN 寄存器来除能指定的定时器中断。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
u32IntDis	指示将除能哪个中断。 Bit 为 1 表示除能 Bit 为 0 表示不影响 参考 EN_TIMER_INT_FLAG_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_enable_wakeup

函数原型

EN_ERR_STA_T rom_hw_timer_enable_wakeup(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32En);

描述

使能定时器唤醒功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
u32En	指示将使能哪个唤醒源，参考 EN_TIMER_WAKEUP_SRC_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_disable_wakeup

函数原型

EN_ERR_STA_T rom_hw_timer_disable_wakeup(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32Dis);

描述

除能定时器唤醒功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
u32Dis	指示将除能哪个唤醒源，参考 EN_TIMER_WAKEUP_SRC_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_enable_sync_start

函数原型

EN_ERR_STA_T rom_hw_timer_enable_sync_start(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

使能指示定时器的开始同步工作功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_disable_sync_start

函数原型

EN_ERR_STA_T rom_hw_timer_disable_sync_start(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

除能指示定时器的开始同步工作功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_enable_sync_stop

函数原型

EN_ERR_STA_T rom_hw_timer_enable_sync_stop(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

使能指示定时器的停止同步工作功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_disable_sync_stop

函数原型

EN_ERR_STA_T rom_hw_timer_disable_sync_stop(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

除能指示定时器的停止同步工作功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_enable_compare_volid_delay

函数原型

EN_ERR_STA_T rom_hw_timer_enable_compare_volid_delay(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

配置指示的定时器比较值在下一个比较周期有效。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个定时器，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_disable_compare_volid_delay

函数原型

EN_ERR_STA_T rom_hw_timer_disable_compare_volid_delay(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

配置指示的定时器比较值立即有效。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个定时器，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_work_status

函数原型

EN_ERR_STA_T rom_hw_timer_get_work_status(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint8_t* pu8Status);

描述

获取指示的定时器工作状态。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示哪个定时器，参考 EN_TIMER_CH_T 枚举定义。

参数	描述
pu8Status	定时器的工作状态，参考 EN_TIMER_STATUS_T 枚举定义。 TIMER_STATUS_RUNNING：定时器正在运行； TIMER_STATUS_STOP：定时器除能或停止。
返回	
HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。

rom_hw_timer_get_counter_mode

函数原型

EN_ERR_STA_T rom_hw_timer_get_counter_mode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint8_t* pu8Mode);

描述

获取指示的定时器计数器模式。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	enCh：指示哪个定时器，参考 EN_TIMER_CH_T 枚举定义。
pu8Mode	定时器计数器模式，参考 EN_TIMER_COUNTER_MODE_T 枚举定义。 TIMER_COUNTER_MODE_DECREASE：定时器工作在递减模式； TIMER_COUNTER_MODE_INCREASE：定时器工作在递增模式。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_wakeup_status

函数原型

EN_ERR_STA_T rom_hw_timer_get_wakeup_status(stTIMER_Handle_t* pstTIMER, uint8_t* pu8State);

描述

获取定时器唤醒源的工作状态。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
pu8Status	指向保存定时器唤醒源工作状态。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_enable_clk

函数原型

EN_ERR_STA_T rom_hal_timer_enable_clk(stTIMER_Handle_t* pstTIMER);

描述

使能定时器 clk。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_disable_clk

函数原型

EN_ERR_STA_T rom_hal_timer_disable_clk(stTIMER_Handle_t* pstTIMER);

描述

除能定时器 clk。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_init

函数原型

EN_ERR_STA_T rom_hal_timer_init(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, stTimerInit_t* pstTimerInit);

描述

初始化指示的定时器。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个通道，请参考 EN_TIMER_CH_T 枚举定义。
pstTimerInit	初始化定时器参数，参考 stTimerInit_t 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_set_compare

函数原型

```
EN_ERR_STA_T rom_hal_timer_set_compare(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32Value);
```

描述

配置指示的定时器比较值。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置给哪个通道比较值。
u32Value	比较值。 16-bit 定时器：比较值范围为 0 ~ 0xFFFF。 32-bit 定时器：比较值范围为 0 ~ 0xFFFFFFFF。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

PWM 功能 API

rom_hw_timer_enable_pwm

函数原型

```
EN_ERR_STA_T rom_hw_timer_enable_pwm(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);
```

描述

使能 PWM 模块。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个定时器的 PWM，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_disable_pwm

函数原型

```
EN_ERR_STA_T rom_hw_timer_disable_pwm(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);
```

描述

除能 PWM 模块。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将除能哪个定时器的 PWM，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_pwm_hi_low_cnt

函数原型

EN_ERR_STA_T rom_hw_timer_set_pwm_hi_low_cnt(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32HiCount, uint32_t u32LoCount);

描述

配置指示定时器的比较值高低计数器。

注：16-bit 定时器：计数器取值范围为 0 ~ 0xFFFF。

32-bit 定时器：计数器取值范围为 0 ~ 0xFFFFFFFF。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道。
u32HiCount	比较值高计数器。
u32LoCount	比较值低计数器。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_pwm_hi_low_cnt

函数原型

EN_ERR_STA_T rom_hw_timer_get_pwm_hi_low_cnt(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t* pu32HiCount, uint32_t* pu32LoCount);

描述

获取高低计数器的指定 PWM 通道。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道。
pu32HiCount	比较值 PWM 高计数器。
pu32LoCount	比较值 PWM 低计数器。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_pwm_polarity

函数原型

EN_ERR_STA_T rom_hw_timer_set_pwm_polarity(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_PWM_POL_T enPol);

描述

设置 PWM 输出极性。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道。
enPol	PWM 极性，参考 EN_PWM_POLARITY_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_pwm_init

函数原型

EN_ERR_STA_T rom_hal_timer_pwm_init(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32Frequency, uint16_t u16Duty);

描述

配置指定的 PWM 频率和占空比。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置给哪个 PWM 频率和占空比，参考 EN_PWM_CH_T 枚举定义。
u32Frequency	PWM 输出频率，单位：Hz
u16Duty	PWM 占空比：0 ~ 10000 → (0 % ~ 100.00 %)， 精准度：定时器 CLK / PWM 频率

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_pwm_hi_lo_cnt

函数原型

EN_ERR_STA_T rom_hal_timer_set_pwm_hi_lo_cnt(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t u32HiCount, uint32_t u32LoCount);

描述

配置指示的 PWM 比较值和高、低计数器。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个 PWM，参考 EN_PWM_CH_T 枚举定义。
u32HiCount	PWM 高计数器。
u32LoCount	PWM 低计数器。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_enable_pwm

函数原型

EN_ERR_STA_T rom_hal_timer_enable_pwm(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

使能 PWM 模块。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个 PWM，参考 EN_PWM_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_disable_pwm

函数原型

EN_ERR_STA_T rom_hal_timer_disable_pwm(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

除能 PWM 模块。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将除能哪个 PWM，参考 EN_PWM_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

定时器捕捉功能

rom_hw_timer_enable_capture

函数原型

EN_ERR_STA_T rom_hw_timer_enable_capture(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

使能定时器捕捉功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个通道，参考 EN_PWM_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_disable_capture

函数原型

EN_ERR_STA_T rom_hw_timer_disable_capture(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

除能定时器捕捉功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将除能哪个通道，参考 EN_PWM_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_capture_and_decode_src

函数原型

EN_ERR_STA_T rom_hw_timer_set_capture_and_decode_src(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_CAP_DECODE_SIGNAL_T enSignal);

描述

配置捕捉和解码信号源。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示哪个通道，参考 EN_PWM_CH_T 枚举定义。
enSignal	捕捉信号，参考 EN_CAP_DECODE_SIGNAL_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_capture_chb_src

函数原型

EN_ERR_STA_T rom_hw_timer_set_capture_chb_src(stTIMER_Handle_t* pstTIMER, EN_CAP_CHB_SRC_T enSrc);

描述

设置指定的定时器捕捉通道 B 信号源。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enSrc	捕捉通道 B 的信号源，参考 EN_CAP_CHB_SRC_T 枚举定义。 TIMER_CAP_CHB_SRC_GPIO_IR：捕捉通道 B 的信号源是 GPIO 或 IR TIMER_CAP_CHB_SRC_DECODE：捕捉通道 B 的信号源是通道 A 解码信号

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_capture_mode

函数原型

EN_ERR_STA_T rom_hw_timer_set_capture_mode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_CAP_MODE_T enMode);

描述

配置捕捉模式。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个定时器，参考 EN_TIMER_CH_T 枚举定义。
enMode	捕捉模式，参考 EN_CAP_MODE_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_get_capture_counter

函数原型

EN_ERR_STA_T rom_hw_timer_get_capture_counter(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint32_t* pu32CapCnt);

描述

获取指示的定时器捕捉计数器。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示哪通道，参考 EN_TIMER_CH_T 枚举定义。
pu32CapCnt	指向捕捉计数器。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_capture_trig_mode

函数原型

EN_ERR_STA_T rom_hw_timer_set_capture_trig_mode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_CAP_TRIG_MODE_T enMode);

描述

配置捕捉触发方式。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示哪个通道，参考 EN_TIMER_CH_T 枚举定义。
enMode	捕捉触发方式，参考 EN_CAP_TRIG_MODE_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_timer_set_capture_counter_format

函数原型

EN_ERR_STA_T rom_hw_timer_set_capture_counter_format(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_CAP_CNT_FORMAT_T enFormat);

描述

配置捕捉计数器格式。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示哪个通道，参考 EN_TIMER_CH_T 枚举定义。
enFormat	捕捉计数器格式，参考 EN_CAP_CNT_FORMAT_T 枚举定义。 CAP_CNT_FORMAT_UNSIGNED: 捕捉计数器为无符号格式。 CAP_CNT_FORMAT_SIGNED: 捕捉计数器为有符号格式。 当捕捉工作在 16 位时，counter[15] = 0，表示输入信号为低电平； counter[15] = 1，表示输入信号为高电平。 当捕捉工作在 32 位时，counter[31] = 0，表示输入信号为低电平； counter[31] = 1，表示输入信号为高电平。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_capture_init

函数原型

EN_ERR_STA_T rom_hal_timer_capture_init(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, stCapInit_t* pstCapInit);

描述

初始化指示的定时器捕捉功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个通道，参考 EN_TIMER_CH_T 枚举定义。
pstCapInit	捕捉初始结构，参考 stCapInit_t 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_enable_capture

函数原型

EN_ERR_STA_T rom_hal_timer_enable_capture(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

使能定时器捕捉功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hal_timer_disable_capture

函数原型

EN_ERR_STA_T rom_hal_timer_disable_capture(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

除能定时器捕捉功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将除能哪个通道，参考 EN_TIMER_CH_T 枚举定义。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

GPTM 范例

第一节详细介绍了 GPTM 的主要功能，本节在前两节基础上开发了相应的 DEMO。

计数器模式范例代码

本范例以 GPTM2 为例，在溢出和比较两种模式下实现 cnt 模块的功能，并在发生中断时进行电平翻转。

中断处理程序

```
static void gtim2_irq_handler(void)
{
    uint32_t u32IntFlag = 0;
    rom_hw_timer_get_interrupt_flag(TIMER2, TIMER_CHA, &u32IntFlag);
    rom_hw_timer_clear_interrupt_flag(TIMER2, TIMER_CHA, u32IntFlag);
    if(u32IntFlag & TIMER_INT_MATCH)
    {
        rom_hw_gpio_toggle_pin_output_level(GPIO_PORT_TIM2_CHA, GPIO_PIN_TIM2_CHA);
    }
    u32IntFlag = 0;
    rom_hw_timer_get_interrupt_flag(TIMER2, TIMER_CHB, &u32IntFlag);
    rom_hw_timer_clear_interrupt_flag(TIMER2, TIMER_CHB, u32IntFlag);
    if(u32IntFlag & TIMER_INT_OVERFLOW)
    {
        rom_hw_gpio_toggle_pin_output_level(GPIO_PORT_TIM2_CHB, GPIO_PIN_TIM2_CHB);
    }
}
```

GPTM 计数器模式范例代码

```
static void gtim2_mode_set(void)
{
    uint8_t u8Status = 0;
    stTimerInit_t pstTimerInit;
    pstTimerInit.u32Compare = 0;
    pstTimerInit.u8CounterMode = 0;
    pstTimerInit.u8Prescale = 0;
    stTimerInit_t pstTimerInit_A;
    pstTimerInit_A.u32Compare = 32767;
    pstTimerInit_A.u8CounterMode = 1;
    pstTimerInit_A.u8Prescale = 0;
    // Enable GPTM2 Clock
    rom_hal_timer_enable_clk(TIMER2);
    // Init GPTM2 CHB and CHA(Compare, Counter Mode and Prescale)
    rom_hal_timer_init(TIMER2, TIMER_CHB, &pstTimerInit);
    rom_hal_timer_init(TIMER2, TIMER_CHA, &pstTimerInit_A); 20.
    // Set GPTM2 CHB INT of Overflow and CHA INT of compare
    rom_hw_timer_enable_interrupt(TIMER2, TIMER_CHB, TIMER_INT_OVERFLOW);
    rom_hw_timer_enable_interrupt(TIMER2, TIMER_CHA, TIMER_INT_MATCH); 24.
    // Enable GPTM2 Peri INT
    rom_hw_sys_ctrl_enable_peri_int(SYS_CTRL_MP, TIMER2_IRQ); 27.
    // Register and Enable INT
    g_periIrqFuncTable[TIMER2_IRQ] = gtim2_irq_handler;
    NVIC_ClearPendingIRQ (TIMER2_IRQ);
    NVIC_SetPriority (TIMER2_IRQ, 0x3);
    NVIC_EnableIRQ (TIMER2_IRQ);
    __enable_irq();
    // START GPTM2 CHB and CHA
    rom_hw_timer_start(TIMER2, TIMER_CHB);
```

```
    rom_hw_timer_start(TIMER2, TIMER_CHA);  
}
```

PWM 模式范例代码

此 DEMO 使用 GPTM2 在 CHA 通道和 CHB 通道上产生具有不同占空比的 PWM 波形。

中断处理程序

```
static void gtim3_irq_handler(void)  
{  
    uint32_t u32IntFlag = 0;  
    rom_hw_timer_get_interrupt_flag(TIMER2, TIMER_CHA, &u32IntFlag);  
    rom_hw_timer_clear_interrupt_flag(TIMER2, TIMER_CHA, u32IntFlag);  
}
```

设置 PWM 模式

```
// GPIO PWM Init  
rom_hw_gpio_set_pin_pid(GPIO_PORT_TIM3_CHB, GPIO_PIN_TIM3_CHB, PID_GTIM_PWM3_CHB);  
rom_hw_gpio_set_pin_pid(GPIO_PORT_TIM3_CHA, GPIO_PIN_TIM3_CHA, PID_GTIM_PWM3_CHA);  
  
// GPTM3 PWM Init  
rom_hal_timer_pwm_init(TIMER3, TIMER_CHB, 16, 3000);  
rom_hal_timer_pwm_init(TIMER3, TIMER_CHA, 16, 7000);  
rom_hal_timer_set_pwm_polarity(TIMER3, TIMER_CHB, PWM_POL_RISING);  
rom_hal_timer_set_pwm_polarity(TIMER3, TIMER_CHA, PWM_POL_RISING);  
// Enable PWM Mode  
rom_hal_timer_enable_pwm(TIMER3, TIMER_CHB);  
rom_hal_timer_enable_pwm(TIMER3, TIMER_CHA);  
rom_hw_sys_ctrl_enable_peri_int(SYS_CTRL_MP, TIMER3_IRQ);  
g_periIrqFuncTable[TIMER3_IRQ] = gtim3_irq_handler;  
NVIC_ClearPendingIRQ (TIMER3_IRQ);  
NVIC_SetPriority (TIMER3_IRQ, 0x3);  
NVIC_EnableIRQ (TIMER3_IRQ);  
__enable_irq();  
// START GPTM3  
rom_hw_timer_start(TIMER3, TIMER_CHB);  
rom_hw_timer_start(TIMER3, TIMER_CHA);
```

4 系统节拍定时器 (STIM)

简介

该单片机有两个 32-bit 系统节拍定时器 (STIM)。每个 32-bit 定时器都有一个 4 通道比较寄存器。

STIM 工作在低速时钟域，可以在睡眠模式下继续工作。STIM 引入了一种灵活的时钟方案，可提供所需的功能和性能，同时较大幅度地降低功耗。

特性

- 2 个独立的 STIM：STIM0 和 STIM1
- 支持低速 32 kHz 时钟作为时钟源
- 每个 STIM 支持 32-bit 计数器，12-bit 预分频
- 每个 STIM 支持 3 种类型的中断和唤醒：溢出 / 比较 / 节拍
- 每个 STIM 支持 4 通道比较中断和睡眠唤醒
- 独立支持 CPU 中断和 PMU 唤醒中断
- 支持配置成功标志检查

注：STIM 外设依靠低频时钟源提供频率。使用频率稳定的时钟源可以获得更好的时钟精度。

功能描述

STIM 外设由 STIM0 和 STIM1 组成，每个 STIM 的参考设计原理图如下所示。所有 STIM 有两个计数器、八个比较、两个节拍和溢出。

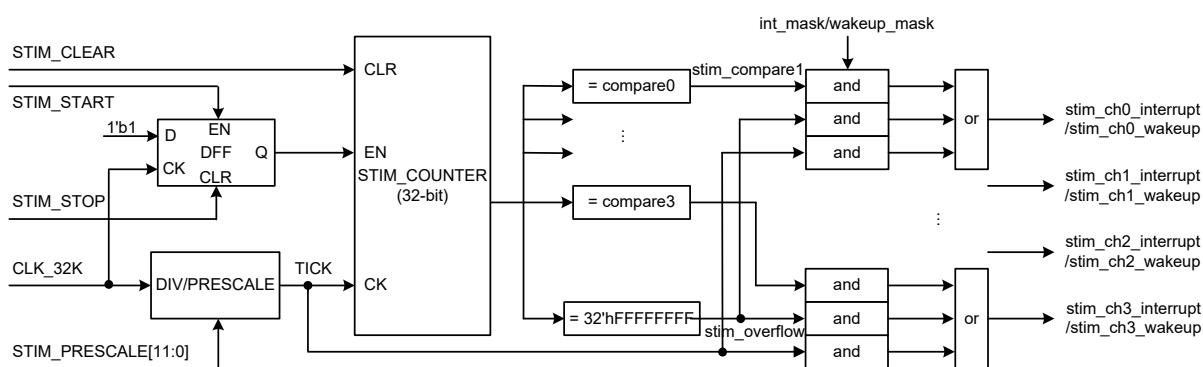


图 6. STIM 参考设计原理图

每个 STIM 是一个只能向上计数的 32-bit 计数器。它采用四个时钟源，分别是：

- RC 32K 低频时钟：RC_LCLK；
- DCXO 32K 低频时钟：DCXO_LCLK；
- RC 高频时钟分频：RC_HCLK_DIV_LCLK；
- DCXO 高频时钟分频：DCXO_HCLK_DIV_LCLK。

时钟精准度和预分频

该单片机的 STIM 使用 32768 Hz 时钟源，最小时间分辨率为 30.517 μs (一个节拍间隔)。下表是不同分频系数下每个节拍的时间间隔。溢出时间是计数器从 0 计数到 0xFFFF FFFF 所花费的时间。

表 4. 时间间隔

PRESCALE	节拍间隔	溢出时间
0	30.517 μs	2183.275 分钟
2 ⁸ - 1	7812.5 μs	9320.676 小时
2 ¹² - 1	125 ms	6213.784 天

设 PRESCALE 为分频系数，每个节拍的时间间隔可计算如下：

$$T_{RTC} = (\text{PRESCALE} + 1) / 32768, 0 \leq \text{PRESCALE} \leq 2^{12} - 1$$

注：该单片机的 PRESCALE 寄存器配置需要一个触发器 (Trig) 才能生效。

计数器、节拍、溢出和比较

该单片机的两个 STIM 外设的计数器长度均为 32-bit，可以在 STIM0 和 STIM1 中独立计数。每个 STIM 支持唤醒和中断配置：4 通道比较，溢出和节拍。当 STIM 工作时，计数器根据时钟信号上升沿的数量而增加并触发节拍中断。当计数器寄存器值达到比较 (n) 寄存器值时触发比较 (n) 中断；当计数器寄存器满 (0xFFFF FFFF) 时，溢出中断将被触发，计数器值将被清零，如下图所示。

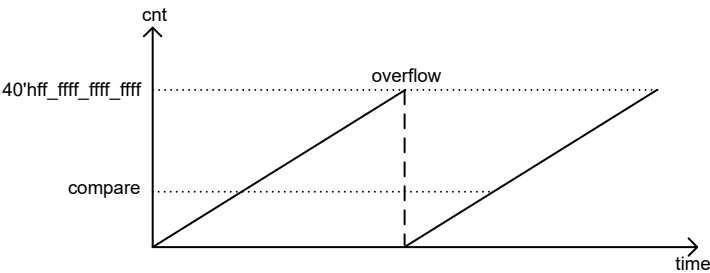


图 7. 计数器触发溢出

注：单片机默认除能节拍、溢出和比较 (n) 中断，使用时必须将其使能。

STIM API

STIM 中断 API

rom_hw_stim_get_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_stim_get_interrupt_flag (stSTIM_Handle_t* pstSTIM, uint16_t* pul6Flag);

描述

通过读取 STIM_INT_FLAG 寄存器来获取指示的定时器中断标志 (状态)。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
pul6Flag	指示将读取哪个中断标志。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_clear_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_stim_clear_interrupt_flag (stSTIM_Handle_t* pstSTIM, uint16_t u16Flag);

描述

通过写入 STIM_INT_CLR 寄存器来清除指示的 STIM 中断标志 (状态)。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
u16Flag	指示将清除哪个标志。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_enable_interrupt

函数原型

EN_ERR_STA_T rom_hw_stim_enable_interrupt (stSTIM_Handle_t* pstSTIM, uint16_t u16IntEn);

描述

通过写入具有指定位的 STIM_INT_EN 寄存器来使能指示的 STIM 中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
u16IntEn	指示将使能哪个中断，@ ref EN_STIM_INT_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_disable_interrupt

函数原型

EN_ERR_STA_T rom_hw_stim_disable_interrupt (stSTIM_Handle_t* pstSTIM, uint16_t u16IntDis);

描述

通过写入具有指定位的 STIM_INT_EN 寄存器来除能指示的 STIM 中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
u16IntDis	指示将除能哪个中断，@ ref EN_STIM_INT_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_enable_tick_overflow_interrupt

函数原型

EN_ERR_STA_T rom_hw_stim_enable_tick_overflow_interrupt (stSTIM_Handle_t* pstSTIM, uint16_t u16IntEn);

描述

使能指示的 STIM 节拍和溢出中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
u16IntEn	指示将使能哪个中断，@ ref EN_STIM_INT_TICK_OVFLW_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_disable_tick_overflow_interrupt

函数原型

EN_ERR_STA_T rom_hw_stim_disable_tick_overflow_interrupt (stSTIM_Handle_t* pstSTIM, uint16_t u16IntDis);

描述

除能指示的 STIM 节拍和溢出中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
u16IntDis	指示将除能哪个中断，@ ref EN_STIM_INT_TICK_OVFLW_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

STIM 唤醒 & 状态 API

rom_hw_stim_enable_wakeup

函数原型

EN_ERR_STA_T rom_hw_stim_enable_wakeup (stSTIM_Handle_t* pstSTIM, uint16_t u16WakeupEn);

描述

使能指示的 STIM 唤醒中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
u16WakeupEn	指示将使能哪个唤醒中断，@ ref EN_STIM_INT_WAKEUP_EN_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_disable_wakeup

函数原型

EN_ERR_STA_T rom_hw_stim_disable_wakeup (stSTIM_Handle_t* pstSTIM, uint16_t u16WakeupDis);

描述

除能指示的 STIM 唤醒中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
u16WakeupDis	指示将除能哪个唤醒中断，@ ref EN_STIM_INT_WAKEUP_EN_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_start

函数原型

EN_ERR_STA_T rom_hw_stim_start (stSTIM_Handle_t* pstSTIM);

描述

启动 STIM 计数器。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_stop

函数原型

EN_ERR_STA_T rom_hw_stim_stop (stSTIM_Handle_t* pstSTIM);

描述

停止 STIM 计数器。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_get_work_status

函数原型

EN_ERR_STA_T rom_hw_stim_get_work_status (stSTIM_Handle_t* pstSTIM, uint8_t* pu8Status);

描述

获取 STIM 工作状态。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
pu8Status	指向保存工作状态的指针。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

STIM 计数器 & 比较

rom_hw_stim_clear_count

函数原型

EN_ERR_STA_T rom_hw_stim_clear_count (stSTIM_Handle_t* pstSTIM);

描述

清除指示的 STIM 计数器寄存器。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_set_count_overflow

函数原型

EN_ERR_STA_T rom_hw_stim_set_count_overflow (stSTIM_Handle_t* pstSTIM);

描述

软件通过将 STIM_COUNTER_REG 寄存器设置为 0xFFFF FFF0 来生成溢出事件。当相应的中断使能时，将发生 STIM 溢出中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_get_count

函数原型

EN_ERR_STA_T rom_hw_stim_get_count (stSTIM_Handle_t* pstSTIM, uint32_t* pu32Count);

描述

获取指示的 STIM 计数器寄存器值。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
pu32Count	指向保存 STIM 当前计数器值 [31:0] 的指针。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_set_prescale

函数原型

EN_ERR_STA_T rom_hw_stim_set_prescale (stSTIM_Handle_t* pstSTIM, uint16_t u16Prescale);

描述

为指示的 STIM 设置预分频值。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
enCh	STIM 比较通道，@ ref EN_STIM_CH_T。
u32Comp	STIM 比较计数器值 [31:0]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_set_compare

函数原型

EN_ERR_STA_T rom_hw_stim_set_compare (stSTIM_Handle_t* pstSTIM, EN_STIM_CH_T enCh, uint32_t u32Comp);

描述

为指示的 STIM 设置一个 32-bit 比较值。当 STIM 计数器递增到与比较值相同的值时，在中断使能时将发生中断。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
enCh	STIM 比较通道，@ ref EN_STIM_CH_T。
u32Comp	STIM 比较计数器值 [31:0]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_stim_get_compare

函数原型

EN_ERR_STA_T rom_hw_stim_get_compare (stSTIM_Handle_t* pstSTIM, EN_STIM_CH_T enCh, uint32_t* pu32Comp);

描述

获取指示的 STIM 比较寄存器 STIM_COMP_REG 值。

参数

参数	描述
pstSTIM	STIM 句柄，应为 STIM0 / STIM1。
enCh	STIM 比较通道，@ ref EN_STIM_CH_T。
pu32Comp	指向保存 STIM 比较计数器值的指针。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

STIM 范例代码

STIM 外设的主要函数已经在前面一节描述过了，接下来的小节将演示 STIM 的主要函数。

STIM 唤醒 & 中断范例代码

范例代码工程

此工程演示了使用 STIM0 通道 0 唤醒 CPU 并进入中断处理程序。

```
uint8_t u8Status = 0;
/* 1. Enable stim0 clock gate first. */
hw_crg_enable_clk_gate (STIM0_CLK_GATE);
/* 2. Clear stim counter if needed. */
hw_stim_work_status (STIM0, &u8Status);
if (STIM_IS_WORKING == u8Status)
{
    hw_stim_stop (STIM0);
    hw_stim_clear_count (STIM0);
}
/* 3. Set stim prescale to 0(default). */
hw_stim_set_prescale (STIM0, 0);
/* 4. Configure STIM0 Channel0 Interrupt Handler*/
g_periIrqFuncTable[STIM0_IRQ0] = stim_stim0_ch0_handler;
NVIC_ClearPendingIRQ (STIM0_IRQ0);
NVIC_SetPriority (STIM0_IRQ0, 3);
NVIC_EnableIRQ (STIM0_IRQ0);
/* 5. Configure STIM0 interrupt and compare value. */
/* 5.1 Disable and clear interrupt flag*/
hw_stim_disable_wakeup (STIM0, STIM_INT_WAKEUP_EN_MASK);
hw_stim_disable_interrupt (STIM0, STIM_INT_MASK);
hw_stim_clear_interrupt_flag (STIM0, STIM_INT_TICK);
hw_stim_disable_tick_overflow_interrupt (STIM0, STIM_INT_TICK_OVERFLOW_MASK);
/* 5.2 Enable interrupt and wake up*/
hw_sys_ctrl_enable_peri_int (SYS_CTRL_MP, STIM0_IRQ0);
hw_stim_enable_wakeup (STIM0, STIM_CH0_INT_MATCH_WAKEUP);
hw_stim_enable_interrupt (STIM0, STIM_CH0_INT_MATCH);
/* 5.3 Set the compare0 channel0 to 200ms, the macro definition is used here to
    convert ms to a counter value */
hw_stim_set_compare(STIM0, STIM_CH0, STIM_MS_TO_CNT(LPWR_CLOCK_32K_HZ, 200));
/* 6. Configure stim0 channel0 wakeup source. */
hw_pmu_set_wakeup_source (0, LUT_TRIG_ID_OTHER, LUT_TRIG_ID_STIM0_CH0, LUT_
    STIM0_CH0_ACT);
/* 7. Start stim0 counter. */
hw_stim_start(STIM0);
```

STIM 中断处理程序范例代码

```
void stim_stim0_ch0_handler (void)
{
    uint32_t count = 0;
    uint16_t u16Flag;
    /* Get stim0 channel0 interrupt flag and clear it. */
    hw_stim_get_interrupt_flag (STIM0, &u16Flag);
    u16Flag &= (STIM_CH0_INT_MATCH | STIM_INT_TICK | STIM_INT_OVERFLOW);
    hw_stim_clear_interrupt_flag(STIM0, u16Flag);
    /* Get STIM0 Counter and print it. */
    hw_stim_get_count(STIM0, &count);
    PRINTF("STIM0 Counter is : %x", count);
}
```

STIM 溢出中断范例代码

此工程演示了如何使用 STIM0 溢出来唤醒 CPU 并进入中断处理程序。这里仍然使用前一节的中断处理程序。

范例代码工程

```
/* 1. Enable stim0 clock gate first. */
hw_crg_enable_clk_gate (STIM0_CLK_GATE);
/* 2. Configure Handler and Enable NVIC */
g_periIrqFuncTable[STIM0_IRQ0] = stim_stim0_ch0_handler;
NVIC_ClearPendingIRQ (STIM0_IRQ0);
NVIC_SetPriority (STIM0_IRQ0, 3);
NVIC_EnableIRQ (STIM0_IRQ0);
/* 3. Configure wake up source */
hw_pmu_set_wakeup_source (n, LUT_TRIG_ID_OTHER, LUT_TRIG_ID_STIM0_CH0, LUT_
                        STIM0_CH0_ACT);
/* 4. Enable stim0 channel 0 overflow interrupt and wakeup system. */
hw_stim_enable_wakeup (STIM0, STIM_INT_OVERFLOW_WAKEUP);
hw_stim_enable_interrupt (STIM0, STIM_INT_OVERFLOW);
hw_stim_enable_tick_overflow_interrupt (STIM0, STIM_CH0_INT_OVFLW);
/* 5. Trigger stim counter to 0xFFFFFFFF0 */
hw_stim_set_count_overflow(STIM0);
/* 6. Start STIM. */
hw_stim_start (STIM0);
```

5 实时时钟 (RTC)

简介

该单片机的实时时钟 (RTC) 外设依赖低频时钟源 (LCLK) 上提供了一个通用的低功耗定时器。当主电源正常运行时，主电源为 RTC 外设供电。当主电源断开时，RTC 外设可通过外部锂电池供电。在电源切换过程中，RTC 的数据始终保存在 RTC 的备份域中。

特性

- 支持低速 32 kHz 时钟作为时钟源
- 支持 40-bit 计数器，12-bit 预分频
- 支持 3 种类型的中断：溢出 / 比较 / 节拍
- 支持 4 通道中断和睡眠唤醒：
 - 通道 0：比较 0 / 节拍 / 溢出
 - 通道 1：比较 1 / 节拍 / 溢出
 - 通道 2：比较 2 / 节拍 / 溢出
 - 通道 3：比较 3 / 节拍 / 溢出
- 支持启动、停止、清除触发器 (Trig) 信号控制
- 独立支持 CPU 中断和 PMU 唤醒中断
- 支持配置成功标志检查

注：RTC 外设依赖低频时钟源提供频率。使用频率稳定的时钟源可以获得更好的时钟精度。

功能描述

该单片机 RTC 的定时器是一个只能向上计数的 40-bit 计数器。它采用三个时钟源，分别是：

- 128 个高速外部时钟：HSE / 128
- 内部时钟：LSI
- 低速外部时钟：LSE

主电源关闭会影响 HSE 分频和 LSI 时钟源，无法保证电源关闭时 RTC 的正常工作。因此，RTC 通常采用低速外部时钟源 LSE。下图为该单片机 RTC 参考设计原理图。

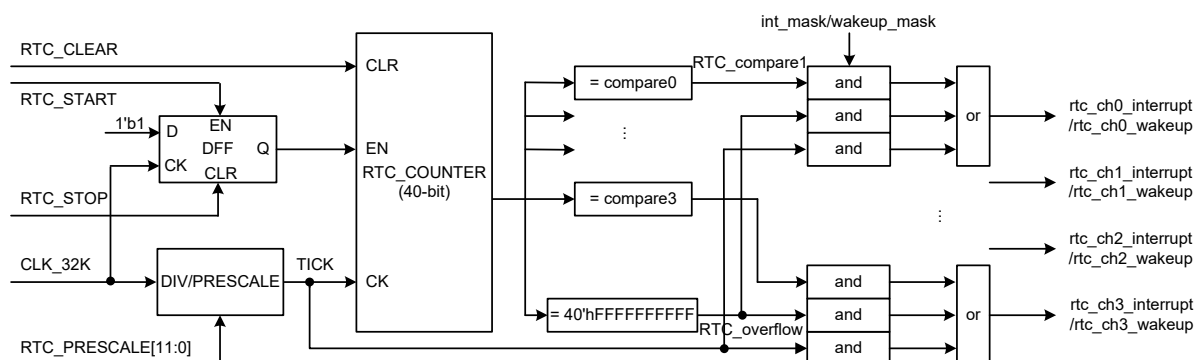


图 8. RTC 参考设计原理图

RTC 电源结构

为了保证掉电情况下的正常工作，该单片机 RTC 由主电源 VDDRTC 和备用电源 VDDRTC_BACK 供电。下图为该单片机 RTC 电源参考设计原理图。

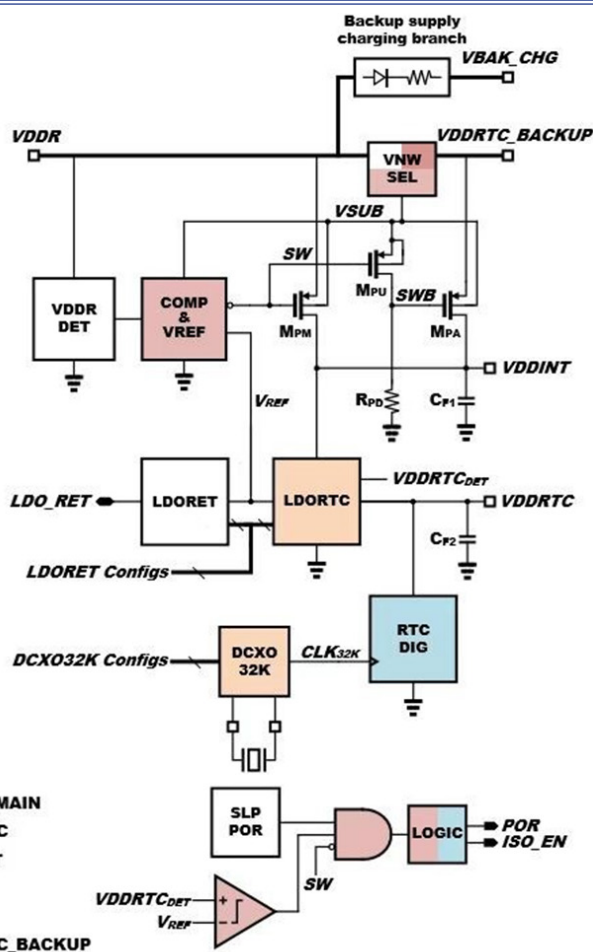


图 9. RTC 电源参考设计原理图

时钟精度和预分频

该单片机的 RTC 外设使用 32768 Hz 时钟源，最小时间分辨率为 30.517 μs (一个节拍间隔)。下表是不同分频系数下每个节拍的时间间隔。溢出时间是计数器从 0 计数到 0xFFFF FFFF 所花费的时间。

PRESCALE	节拍间隔	溢出时间
0	30.517 μs	9315 小时
2 ⁸ - 1	7812.5 μs	99420 天
2 ¹² - 1	125 ms	1590728 天

设 PRESCALE 为分频系数，每个节拍的时间间隔可计算如下：

$$T_{RTC} = (PRESCALER + 1) / 32768, 0 \leq PRESCALER \leq 2^{12} - 1$$

注：该单片机的 PRESCALE 寄存器配置需要一个触发器 (Trig) 才能生效。

计数器、节拍、溢出和比较

该单片机 RTC 外设的计数器寄存器长度为 40-bit，可同时配置比较、溢出和节拍中断或唤醒 4 个通道。下图显示了计数器工作流程。当 RTC 工作时，计数器根据时钟信号上升沿的数量而增加并触发节拍中断。当计数器寄存器值达到比较 (n) 寄存器值时触发比较 (n) 中断；当计数器寄存器满 (0xFF FFFF FFFF) 时，溢出中断将被触发，计数器值将被清零，如下图所示。

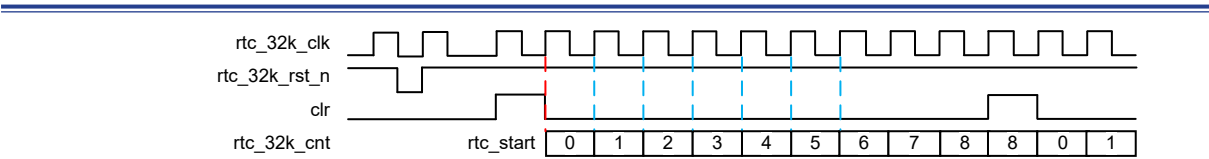


图 10. 计数器工作流程

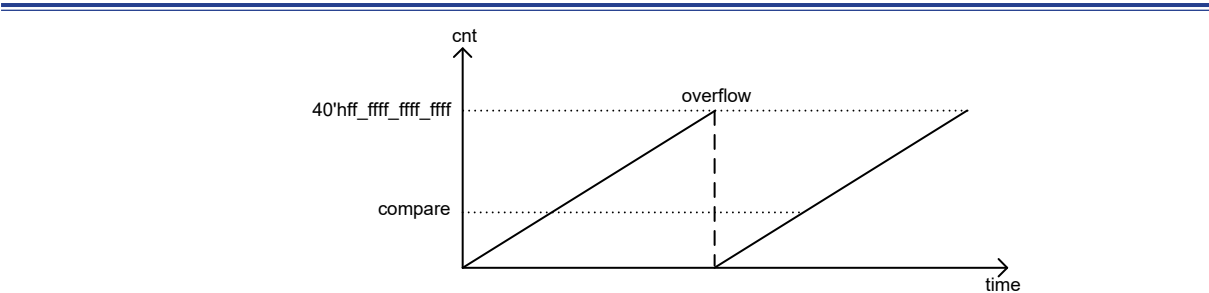


图 11. 计数器触发溢出

注：单片机默认除能节拍、溢出和比较 (n) 中断，使用时必须将其使能。

RTC API

RCT 配置 API

rom_hw_rtc_set_rtc_clk_src

函数原型

EN_ERR_STA_T rom_hw_rtc_set_rtc_clk_src (EN_RTC_CLK_SRC_T enSrc);

描述

选择 RTC_CLK 时钟源。

参数

参数	描述
enSrc	RTC 时钟源选择, @ ref EN_RTC_CLK_SRC_T。

返回

EN_ERR_STA_T	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
--------------	-------------------------------

rom_hw_rtc_set_prescale

函数原型

EN_ERR_STA_T rom_hw_rtc_set_prescale (uint16_t u16Prescale);

描述

设置 RTC 时钟的预分频值。

参数

参数	描述
u16Prescale	预分频值。

返回

EN_ERR_STA_T	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
--------------	-------------------------------

rom_hw_rtc_set_ldo_ret_output_voltage

函数原型

EN_ERR_STA_T rom_hw_rtc_set_ldo_ret_output_voltage (EN_RTC_LDO_RET_VOLT_T enVolt);

描述

配置 VDD_RET 输出电压。当 CPU 进入睡眠模式时, 保存 LDO 将工作并输出 VDD_RET 电压以保持系统处于睡眠模式。

参数

参数	描述
enVolt	配置 LDO_RET (VDDRTC) 输出电压, @ ref EN_RTC_LDO_RET_VOLT_T。

返回

EN_ERR_STA_T	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
--------------	-------------------------------

rom_hw_rtc_pdw_reset

函数原型

EN_ERR_STA_T rom_hw_rtc_pdw_reset (void);

描述

复位 RTC 模块电源。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

RTC 中断 API

rom_hw_rtc_get_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_rtc_get_interrupt_flag (uint16_t *pu16Msk);

描述

通过读 RTC_INT_FLAG 寄存器来获取指示的 RTC 中断标志 (状态)。

参数

参数	描述
pu16Flag	指示将读取哪个中断标志。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_clear_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_rtc_clear_interrupt_flag (uint16_t u16Msk)

描述

通过写 RTC_INT_CLR 寄存器来清除指示的 RTC 中断标志 (状态)。

参数

参数	描述
u16Msk	指示将清除哪个标志。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_enable_interrupt

函数原型

EN_ERR_STA_T rom_hw_rtc_enable_interrupt (uint16_t u16IntEn);

描述

通过写具有指定位的 RTC_INT_EN 寄存器来使能指示的 RTC 中断。

参数

参数	描述
u16IntEn	指示将使能哪个中断。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_disable_interrupt

函数原型

EN_ERR_STA_T rom_hw_rtc_disable_interrupt (uint16_t u16IntDis);

描述

通过写具有指定位的 RTC_INT_EN 寄存器来除能指示的 RTC 中断。

参数

参数	描述
u16IntDis	指示将除能哪个中断。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_enable_tick_overflow_interrupt

函数原型

EN_ERR_STA_T rom_hw_rtc_enable_tick_overflow_interrupt (uint16_t u16IntEn);

描述

使能指示的 RTC 节拍和溢出中断。

参数

参数	描述
u16IntEn	指示将使能哪个中断，@ ref EN_RTC_INT_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_disable_tick_overflow_interrupt

函数原型

EN_ERR_STA_T rom_hw_rtc_disable_tick_overflow_interrupt (uint16_t u16IntDis);

描述

除能指示的 RTC 节拍和溢出中断。

参数

参数	描述
u16IntDis	指示将除能哪个中断, @ ref EN_RTC_INT_T。

返回

EN_ERR_STA_T	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
--------------	-------------------------------

rom_hw_rtc_enable_wakeup

函数原型

EN_ERR_STA_T rom_hw_rtc_enable_wakeup (uint16_t u16WakeupEn);

描述

使能指示的 RTC 唤醒中断。在相应的中断使能后, CPU 将从睡眠模式中唤醒。

参数

参数	描述
u16WakeupEn	指示将使能哪个唤醒中断, @ ref EN_RTC_INT_WAKEUP_EN_T。

返回

EN_ERR_STA_T	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
--------------	-------------------------------

rom_hw_rtc_disable_wakeup

函数原型

EN_ERR_STA_T rom_hw_rtc_disable_wakeup (uint16_t u16WakeupDis);

描述

除能指示的 RTC 唤醒中断。

参数

参数	描述
u16WakeupDis	指示将除能哪个唤醒中断, @ ref EN_RTC_INT_WAKEUP_EN_T。

返回

EN_ERR_STA_T	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
--------------	-------------------------------

RTC 状态 API

rom_hw_rtc_start

函数原型

EN_ERR_STA_T rom_hw_rtc_start(void);

描述

启动 RTC 计数器。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_stop

函数原型

EN_ERR_STA_T rom_hw_rtc_stop(void);

描述

停止 RTC 计数器。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_get_work_status

函数原型

EN_ERR_STA_T rom_hw_rtc_get_work_status(uint8_t* pu8Status);

描述

获取 RTC 工作状态。

参数

参数	描述
pu8Status	指向保存 RTC 工作状态的指针。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_get_wakeup_status

函数原型

EN_ERR_STA_T rom_hw_rtc_get_wakeup_status (uint8_t* pu8Status);

描述

获取 RTC 唤醒源工作状态，PMU 处于停止状态时才会进入低功耗模式。

参数

参数	描述
pu8Status	指向保存 RTC 唤醒源工作状态的指针，@ ref EN_RTC_WAKEUP_STATUS_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_set_sw_flag

函数原型

EN_ERR_STA_T rom_hw_rtc_set_sw_flag (uint32_t u32SwFlag);

描述

配置软件标志。检查 RTC 模块是否初始化。当 VDDR 掉电时，此标志将被保留。

参数

参数	描述
pu8Status	软件标志。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

RTC 计数器 & 比较

rom_hw_rtc_get_count

函数原型

EN_ERR_STA_T rom_hw_rtc_get_count (uint8_t* pu8Hi, uint32_t* pu32Lo);

描述

获取 RTC 当前计数器值。

参数

参数	描述
pu8Hi	指向保存 RTC 当前计数器值 [39:32]。
pu32Lo	指向保存 RTC 当前计数器值 [31:0]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_get_count64

函数原型

EN_ERR_STA_T rom_hw_rtc_get_count64 (uint64_t* pu64Count);

描述

获取 RTC 当前计数器值。

参数

参数	描述
pu64Count	指向保存 RTC 当前计数器值 [39:0]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_clear_count

函数原型

EN_ERR_STA_T rom_hw_rtc_clear_count (void);

描述

清除 RTC 计数器。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_set_count_overflow

函数原型

EN_ERR_STA_T rom_hw_rtc_set_count_overflow (void);

描述

将 RTC 计数器设置为 0xFFFFFFFFF0，可以快速触发溢出中断。

参数

无

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_set_compare

函数原型

EN_ERR_STA_T rom_hw_rtc_set_compare(EN_RTC_CH_T enCh, uint8_t u8Hi, uint32_t u32Lo);

描述

设置 RTC 比较计数器值。当 RTC 递增到与比较计数器值相同的值时，RTC 将在中断使能时产生中断。

参数

参数	描述
enCh	RTC 比较通道，@ ref EN_RTC_CH_T。
u8Hi	RTC 比较计数器值 [39:32]。
u32Lo	RTC 比较计数器值 [31:0]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_set_compare64

函数原型

EN_ERR_STA_T rom_hw_rtc_set_compare64 (EN_RTC_CH_T enCh, uint64_t u64Compare);

描述

设置 RTC 比较计数器值。当 RTC 递增到与比较计数器值相同的值时，RTC 将在中断使能时产生中断。

参数

参数	描述
enCh	RTC 比较通道，@ ref EN_RTC_CH_T。
u64Compare	RTC 比较计数器值 [39:0]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_get_compare

函数原型

EN_ERR_STA_T rom_hw_rtc_get_compare (EN_RTC_CH_T enCh, uint8_t* pu8Hi, uint32_t* pu32Lo);

描述

获取 RTC 比较计数器值。

参数

参数	描述
enCh	RTC 比较通道，@ ref EN_RTC_CH_T。
pu8Hi	指向保存 RTC 比较计数器值 [39:32]。
pu32Lo	指向保存 RTC 比较计数器值 [31:0]。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_rtc_get_compare64

函数原型

EN_ERR_STA_T rom_hw_rtc_get_compare64 (EN_RTC_CH_T enCh, uint64_t* pu64Compare);

描述

获取 RTC 比较计数器值。

参数

参数	描述
enCh	RTC 比较通道, @ ref EN_RTC_CH_。
pu64Compare	指向保存 RTC 比较计数器值 [39:0]。

返回

EN_ERR_STA_T	函数返回状态, 参考 EN_ERR_STA_T 枚举定义。
--------------	-------------------------------

RTC 范例代码

RTC 外设的主要函数已经在上面的小节中描述过了, 接下来一节将演示 RTC 的主要函数。

RTC 开始工作范例代码

当使用 RTC 外设时, 需要设置 VDDRTC 电压并启动时钟。此工程演示了 RTC 正常工作时的时钟设置。

```
/* Open the system APB clock & RC clock */
rom_hw_crg_enable_clk_gate(CRG_RTC_APB_CLK_GATE);
rom_hw_crg_enable_clk_gate(CRG_RTC_RC_LCLK_GATE);
/* Set the voltage of the VDDRTC */
patch_hw_rtc_set_ldo_ret_output_voltage(EN_RTC_LDO_RET_950mV);
/* Set RC 32K as the RTC clock source */
patch_hw_rtc_set_rtc_clk_src(EN_RTC_CLK_SRC_RC_LCLK);
/* RTC start to work */
patch_hw_rtc_start();
```

RTC 中断范例代码

此范例工程旨在测试 RTC 是否可以通过获取当前计数器值并将比较 1 设置为当前计数器值 +1000 来响应比较中断。范例如下:

RTC 中断范例代码

```
void RTC_Interrupt (void)
{
    uint64_t now_counter = 0;
    uint64_t compare_val = 1000;
    /*Stop RTC counter*/
    patch_hw_rtc_stop();
    /*Set prescale value*/
    patch_hw_rtc_set_prescale(0);
    /*Enable RTC channel 1 interrupt*/
    patch_hw_rtc_enable_interrupt(RTC_CH1_INT_MATCH);
    /*Get counter value*/
    patch_hw_rtc_get_count64(&now_counter);
    /*Set compare1 value, Interrupt will be triggered after 1000 counters*/
    patch_hw_rtc_set_compare64(RTC_CH1, now_counter + compare_val);
    /*Configure for RTC interrupt handler*/
```

```
g_periIrqFuncTable[RTC_CH1_IRQ] = RTC_Interrupt_Handler;
/*Enable system interrupt*/
NVIC_ClearPendingIRQ (RTC_CH1_IRQ);
NVIC_SetPriority (RTC_CH1_IRQ, 3);
NVIC_EnableIRQ (RTC_CH1_IRQ);
__enable_irq();
/*RTC start to work*/
patch_hw_rtc_start();
}
```

RTC 中断处理程序范例代码

```
void RTC_Interrupt_Handler(void)
{
    uint16_t int_flag = 0;
    uint64_t compare_val = 0;
    uint64_t counter_val = 0;
    /*Clear RTC Interrupt flag*/
    patch_hw_rtc_get_interrupt_flag(&int_flag);
    patch_hw_rtc_clear_interrupt_flag(int_flag);
    /*Gets the value of the current counter*/
    patch_hw_rtc_get_count64(&counter_val);
    PRINTF("The counter value is: %llx \n", counter_val);
}
```

唤醒范例代码

下面的范例代码显示了 RTC 如何在睡眠模式下唤醒 CPU。

```
void RTC_Wake(void)
{
    uint64_t now_counter = 0;
    uint64_t compare_val = 1000;
    patch_hw_rtc_stop();
    patch_hw_rtc_set_prescale(0);
    patch_hw_rtc_enable_interrupt(RTC_CH1_INT_MATCH);
    patch_hw_rtc_get_count64(&now_counter);
    patch_hw_rtc_set_compare64(RTC_CH1, now_counter + compare_val);
    /*Set RTC CH1 as the wake source*/
    rom_hw_pmu_set_wakeup_source (9, LUT_TRIG_ID_OTHER, LUT_TRIG_ID_RTC_CH1,
    LUT_ACT_PD_SYS_ON | LUT_ACT_MP_IRQ_EN);
    /*Enable wakeup*/
    patch_hw_rtc_enable_wakeup(RTC_CH1_INT_MATCH_WAKEUP);
    patch_hw_rtc_start();
}
```

6 通用异步收发器 (UART)

简介

UART 符合 16550 工业标准，用于与外设数据集进行串行通信。数据通过 APB 总线从主机 (CPU / DMA) 写入到 UART，然后转换为串行形式并传输到目标设备。串行数据也由 UART 接收并存储，供主机 (CPU / DMA) 回读。这些 UART 都支持硬件流控制信号 (RTS 和 CTS)。

特性

- 8-byte 发送和接收 FIFO
- 硬件流控制支持 (CTS / RTS)
- 基于 16550 工业标准的功能
- 可编程字符属性，如每个字符的数据位数 (5 ~ 8) 可选
- 奇偶校验位 (奇 / 偶 / 粘性 / 不选择) 和停止位 (1、1.5 或 2)
- 线中止错误生成和检测
- 支持 RX 超时中断
- 可编程串行数据波特率

功能描述

该单片机 UART 功能主要包括：UART 初始化、UART 配置、串行数据接收 / 发送、中断配置等。

UART API

UART 配置 API

rom_hw_uart_deinit

函数原型

EN_ERR_STA_T rom_hw_uart_deinit(stUART_Handle_t* pstUART);

描述

初始化 UART 外设。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
返回	
HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。

rom_hw_uart_init

函数原型

EN_ERR_STA_T rom_hw_uart_init(stUART_Handle_t* pstUART, stUartInit_t* pstInitType);

描述

根据 stUartInit_t 中指定的参数来初始化 UART 模式，并创建相关句柄。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
pstInitType	指向 stUartInit_t 结构的指针。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_baudrate

函数原型

EN_ERR_STA_T rom_hw_uart_set_baudrate(stUART_Handle_t* pstUART, uint32_t u32BaudRate);

描述

设置 UART 波特率。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u32BaudRate	波特率。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_data_sizes

函数原型

EN_ERR_STA_T rom_hw_uart_set_data_sizes(stUART_Handle_t* pstUART, EN_UART_DATA_SIZE_T enSize);

描述

设置 UART 数据位。数据位向上兼容，高数据位可以正确接收低数据位的数据。databit5 最大值为 31，databit6 最大值为 63，databit7 最大值为 127，databit8 最大值为 255。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
enSize	UART 数据大小，@ ref EN_UART_DATA_SIZE_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_stop_sizes

函数原型

EN_ERR_STA_T rom_hw_uart_set_stop_sizes(stUART_Handle_t* pstUART, EN_UART_STOP_SIZE_T enStopbits);

描述

设置 UART 停止位。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
enStopbits	UART 停止位，@ ref EN_UART_STOP_SIZE_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_parity

函数原型

EN_ERR_STA_T rom_hw_uart_set_parity(stUART_Handle_t* pstUART, EN_UART_PARITY_BITS_T enParity);

描述

设置 UART 奇偶校验。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
enParity	UART 奇偶校验位，@ ref EN_UART_PARITY_BITS_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_mode

函数原型

EN_ERR_STA_T rom_hw_uart_set_mode(stUART_Handle_t* pstUART, EN_UART_ENDIAN_T enMode);

描述

设置 UART 大小端 (Endian) 模式。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
enMode	UART 大小端模式，@ ref EN_UART_ENDIAN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_line_break_enable

函数原型

EN_ERR_STA_T rom_hw_uart_line_break_enable(stUART_Handle_t* pstUART, EN_UART_LB_EN_T enEn);

描述

配置 UART 发送线中止错误功能。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
enEn	使能或除能 TX 线中止错误功能。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_rx_timeout

函数原型

EN_ERR_STA_T rom_hw_uart_set_rx_timeout(stUART_Handle_t* pstUART, uint8_t u8SymbolNum);

描述

设置 UART RX 超时时间。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u8SymbolNum	超时值，单位：ms

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_flow_enable

函数原型

EN_ERR_STA_T rom_hw_uart_flow_enable(stUART_Handle_t* pstUART, EN_UART_FLOW_EN_T enEn);

描述

UART 配置流功能。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
enEn	使能或除能流功能，@ ref EN_UART_FLOW_EN_T。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_rts_thld

函数原型

EN_ERR_STA_T rom_hw_uart_set_rts_thld(stUART_Handle_t* pstUART, uint8_t u8Thld);

描述

设置 RTS 阈值。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u8Thld	UART RTS 阈值。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

UART 中断 API

rom_hw_uart_get_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_uart_get_interrupt_flag(stUART_Handle_t* pstUART, uint16_t* pul6Falg);

描述

获取 UART 中断标志 (状态)。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
pul6Falg	指示将读取哪个中断标志。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_enable_interrupt

函数原型

EN_ERR_STA_T rom_hw_uart_enable_interrupt(stUART_Handle_t* pstUART, uint16_t ul6Msk);

描述

设置 UART 中断使能。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
ul6Msk	指示将使能哪个中断。 Bit 为 1 表示使能 Bit 为 0 表示无影响

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_disable_interrupt

函数原型

EN_ERR_STA_T rom_hw_uart_disable_interrupt(stUART_Handle_t* pstUART, uint16_t u16Msk);

描述

设置 UART 中断除能。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u16Msk	指示将除能哪个中断。 Bit 为 1 表示除能 Bit 为 0 表示无影响

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_clear_interrupt_flag

函数原型

EN_ERR_STA_T rom_hw_uart_clear_interrupt_flag(stUART_Handle_t* pstUART, uint16_t u16Msk);

描述

清除指示的 UART 中断标志。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u16Msk	指示将清除哪个标志。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

UART FIFO API

rom_hw_uart_set_txfifo_thld

函数原型

EN_ERR_STA_T rom_hw_uart_set_txfifo_thld(stUART_Handle_t* pstUART, uint8_t u8Thld);

描述

设置 TX FIFO 低于阈值级别。当 TX FIFO 低于该阈值时将触发中断。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u8Thld	UART TX FIFO 阈值。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_set_rxfifo_thld

函数原型

EN_ERR_STA_T rom_hw_uart_set_rxfifo_thld(stUART_Handle_t* pstUART, uint8_t u8Thld);

描述

设置 RX FIFO 超过阈值级别。当 RX FIFO 超过该阈值时触发中断。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u8Thld	UART RX FIFO 阈值。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_clear_rxfifo

函数原型

EN_ERR_STA_T rom_hw_uart_clear_rxfifo(stUART_Handle_t* pstUART);

描述

配置 UART 清除 FIFO 功能。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_get_txfifo_cnt

函数原型
EN_ERR_STA_T rom_hw_uart_get_txfifo_cnt(stUART_Handle_t* pstUART, uint8_t* pu8Cnt);
描述
获取 UART TX FIFO 计数。
参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
pu8Cnt	TX FIFO 计数值。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_get_rxfifo_cnt

函数原型
EN_ERR_STA_T rom_hw_uart_get_rxfifo_cnt(stUART_Handle_t* pstUART, uint8_t* pu8Cnt);
描述
获取 UART RX FIFO 计数。
参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
pu8Cnt	RX FIFO 计数值。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

UART RX/TX API

rom_hw_uart_get_byte

函数原型
EN_ERR_STA_T rom_hw_uart_get_byte(stUART_Handle_t* pstUART, uint8_t* pu8Data);
描述
在非阻塞模式下获取一个字节的数据。
参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
pu8Data	指向数据缓冲器的指针。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_receive

函数原型

EN_ERR_STA_T rom_hw_uart_receive(stUART_Handle_t* pstUART, uint8_t* pu8Buf, uint8_t u8BufSize);

描述

在非阻塞模式下接收一定数量的数据。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
pu8Data	指向数据缓冲器的指针。
u16Len	要接收的数据量。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_send_byte

函数原型

EN_ERR_STA_T rom_hw_uart_send_byte(stUART_Handle_t* pstUART, uint8_t u8Data);

描述

当 TX FIFO 未滿时，通过 UART 传输单个数据，发送时阻塞。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
u8Data	要传输的数据。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

rom_hw_uart_transmit

函数原型

EN_ERR_STA_T rom_hw_uart_transmit(stUART_Handle_t* pstUART, uint8_t* pu8Buf, uint16_t u16Len);

描述

在阻塞模式下传输一定数量的数据。

参数

参数	描述
pstUART	UART 句柄，应为 UART0 / UART1 / UART2。
pu8Data	指向数据缓冲器的指针。
u16Len	要发送的数据量。

返回

HW status	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
-----------	------------------------------

UART 范例代码

使用 UART1 的应用范例

初始化串口引脚

要初始化串口引脚，步骤 1：定义串口的 GPIO，步骤 2：初始化串口时钟，步骤 3：设置串口 GPIO。代码如下：

```
#define GPIO_PIN_UART1_TX(GPIO_PIN_10)
#define GPIO_PIN_UART1_RX(GPIO_PIN_11)
static void uart_config(void)
{
    // Initialize the uart clock
    rom_hw_crg_enable_clk_gate(CRG_UART1_CLK_GATE);
    // Initialize the serial port pin
    patch_hw_gpio_set_pin_pid(GPIOA, GPIO_PIN_UART1_TX, PID_UART1_TXD);
    patch_hw_gpio_set_pin_pid(GPIOB, GPIO_PIN_UART1_RX, PID_UART1_RXD);
    // Initializing the serial port
    uart_init(UART1);
}
```

初始化 UART

要初始化 UART，步骤 1：初始化 UART 句柄，用于注册中断函数。步骤 2：初始化串口结构，设置波特率、FIFO、数据位和奇偶校验。步骤 3：设置中断接收模式。步骤 4：注册中断服务函数。步骤 5：使能超时中断。代码如下：

```
static void uart_init(pstUART_Handle_t UART_Handle)
{
    // Initialize the uart handle
    pstUART_Handle_t g_pu32UartHandle = UART_Handle;
    // Initialize the initial uart
    struct stUartInit_t stUartInit;
    // Set the communication baud rate
    stUartInit.u32UartBaudRate = UART_BAUDRATE_921600;
    // Set the default configuration of the serial port
    // TxFifoThld and RxFifoThld is 8, StopBit is 1, 8bit, no parity, little-
    // ending
    stUartInit.unUartCfg.u32UartCfg = UART_INIT_DEFAULT(UART_PARITY_NONE);
    // Write the default configuration of the serial port
    rom_hw_uart_init(g_pu32UartHandle, &stUartInit);
    // Receive data with a timeout interrupt
    rom_hw_uart_set_rx_timeout(g_pu32UartHandle, 0xFF);
    // Register the interrupt service function
    uart_nvic_init(UART_Handle);
    // Enable timeout interrupts
    rom_hw_uart_enable_interrupt(g_pu32UartHandle, UART_INT_RX_TIMEOUT);
}
```

注册中断服务函数

当接收到 UART 数据时，将进入 UART 中断回调函数。中断服务函数如下：

```
static void uart_nvic_init(pstUART_Handle_t UART_Handle)
{
    // Clear the interrupt status flag
    rom_hw_uart_clear_interrupt_flag(UART_Handle, 0x1fff);
    // Register the interrupt callback function.
    g_periIrqFuncTable[UART1_IRQ] = (PERI_IRQ_FUNC)UART1_IRQ_Handler;
```

```
// Clear the interrupt
NVIC_ClearPendingIRQ (UART1_IRQ);
// Set interrupt priority
NVIC_SetPriority (UART1_IRQ, 0x3);
// enable interrupt
NVIC_EnableIRQ (UART1_IRQ);
// Here is divided into cortex_M33 and cortex_M0+
#ifdef MAIN_PROCESSOR
rom_hw_sys_ctrl_enable_peri_int(SYS_CTRL_MP, UART1_IRQ);
#else
rom_hw_sys_ctrl_enable_peri_int(SYS_CTRL_CP, UART1_IRQ);
#endif
__enable_irq();
}
```

接收数据

数据通常在 UART 中断回调函数中接收。当判断数据可用时，就可以接收数据。接收数据代码如下：

```
static void UART1_IRQ_Handler(void)
{
    static uint8_t u8Cnt = 0;
    // Determines if data is available.
    while(UART1->UART_RXFIFO_CNT & 0x1F)
    {
        // Save data
        g_PCMsg.Data[u8Cnt++] = UART1 -> UART_RX_FIFO & 0xFF;
    }
    // Clear interruption flags.
    rom_hw_uart_clear_interrupt_flag(UART1, UART_INT_RX_TIMEOUT);
}
```

发送数据

此函数可以用于单字节发送。详细参数请参考前一节。

```
rom_hw_uart_send_byte (stUART_Handle_t* pstUART, uint8_t u8Data);
```

7 IR 模块

简介

该单片机可支持 IR 编码和 IR 解码，两者均由芯片的 GTIM 部分完成。

IR 编码：芯片通过 GTIM 实现 IR 编码，生成的 IR 信号通过连接的 IR 二极管发送。

IR 解码：芯片包含 IR 接收电路，可以接收 IR 信号，然后通过软件实现 IR 解码。

特性

- 1 通道内部 IR 传输电路
- 4 通道 IR 编码
 - IR 编码信号可通过 GPIO 输出
 - 通过在 GPIO 上连接 IR 传输电路，可实现 4 通道 IR 发射
- 4 通道 IR 触发解码功能
 - 支持 cnta/cntb 解码 (支持 cnta 和 cntb 同时解码)
 - 支持组合 32-bit 解码

功能描述

IR 编码

它用于产生 IR 波形。

IR 波形由 GTIM 的 cnta 和 cntb 产生的两个 PWM 波形组合而成。

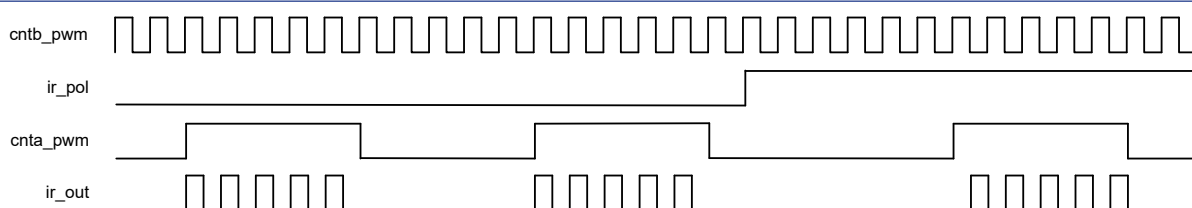


图 12. IR 编码

用 `cntb` 输出指定频率、占空比为 50 % 的 PWM 作为载波，用 `cnta` 输出指定 PWM 作为数据波形信号，然后用逻辑“与”将两个 PWM 信号合成所需的 IR 信号。

`ir_pol` 可用于对载波信号 `cntb` 波形进行反相，以满足特定需求。

注：该单片机有四个 IR 编码通道，但只有一个内部 IR 发送引脚。

单片机的 IR 编码信号可以通过 GPIO 输出，因此该单片机可以借助外部硬件设备实现四通道 IR 发射的效果。

IR 解码

它用于 IR 解码，将带载波的 IR 信号还原为电平信号。

每个 GTIM 包含两个通道 (通道 A 和通道 B)，每个通道中都有一个解码计数器。当波形变化时间在配置计数时间 (解码间隔) 内时，输出高电平。当波形变化时间超过配置时间时，输出低电平。(decode_gpout_pol 可以配置为反相输出)

两个通道也可以组合成一个 32-bit 通道使用。

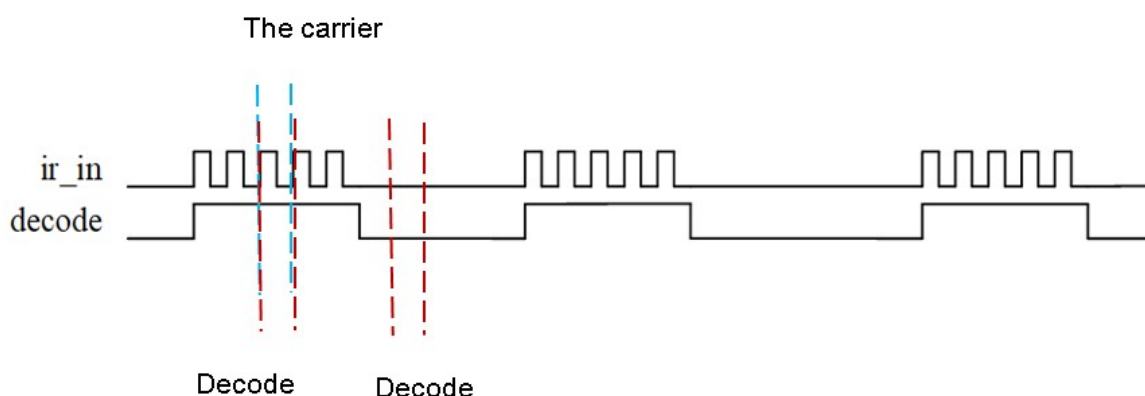


图 13. IR 解码

注：配置计数时间在代码配置中称为解码间隔。

当检测到输入信号为高电平时，解码信号将输出高电平，然后解码开始计数。如果在一段时间间隔内检测到输入 IR 信号的波形变化，则解码将继续输出高电平。如果在一段时间间隔内没有检测到波形变化，解码将立即拉低。

我们需要保证输入波形的一个变化周期时间在解码间隔内，并且解码间隔也尽可能接近输入波形的一个变化周期时间，以减少解码误差。

IR 模块 API

IR 编码 HW API

rom_hw_ir_set_send_path

函数原型

```
EN_ERR_STA_T rom_hw_ir_set_send_path (stTIMER_Handle_t* pstIR, EN_IR_SEND_PATH_T enPath);
```

描述

设置 IR 发送信号路径。

EN_IR_SEND_PATH_T 的定义如下：

```
typedef enum
{
    IR_SEND_DEFAULT_PATH = 0,
    IR_SEND_BY_IR_TX_PIN = 1,
    IR_SEND_BY_GPIO = 2,
    IR_SEND_BY_BOTH = 3,
} EN_IR_SEND_PATH_T;
```

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
enPath	IR 发送信号路径，参考 EN_IR_SEND_PATH_T 定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_ir_set_pwm_current_compare_and_polarity

函数原型

EN_ERR_STA_T rom_hw_ir_set_pwm_current_compare_and_polarity (stTIMER_Handle_t* pstIR, uint16_t u16Compare, EN_PWM_POL_T enPol)

描述

配置 IR TX 参数，包括电流、PWM 极性等。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
u16Compare	比较值。 范围为 0 ~ 65535。
enPol	PWM 极性，参考 EN_PWM_POL_T 定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_ir_pwm_set_next_compare

函数原型

EN_ERR_STA_T rom_hw_ir_pwm_set_next_compare (stTIMER_Handle_t* pstIR, uint16_t u16Compare, uint16_t u16LoCnt, uint16_t u16HiCnt);

描述

配置 IR TX 载波 PWM 周期和宽度。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
u16Compare	比较值。 范围为 0 ~ 65535。
u16HiCnt	比较高计数器。当计数器计数到高计数器值时，电平将翻转一次。 范围为 0 ~ 65535。
u16LoCnt	比较低计数器。当计数器计数到低计数器值时，电平将翻转一次。 范围为 0 ~ 65535。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_ir_enable

函数原型

EN_ERR_STA_T rom_hw_ir_enable (stTIMER_Handle_t* pstIR);

描述

使能 GPTM 工作在 IR 功能。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_ir_disable

函数原型

EN_ERR_STA_T rom_hw_ir_disable (stTIMER_Handle_t* pstIR);

描述

除能 GPTM IR 功能。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_ir_set_polarity

函数原型

EN_ERR_STA_T rom_hw_ir_set_polarity (stTIMER_Handle_t* pstIR, EN_IR_POL_T enPol);

描述

配置 IR TX 输出载波极性。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
enPol	IR 极性，参考 EN_PWM_POL_T 定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_ir_enable_channel

函数原型

EN_ERR_STA_T rom_hw_ir_enable_channel (stTIMER_Handle_t* pstIR, EN_IR_CH_T enCh);

描述

使能 IR 通道。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
enCh	IR 通道，参考 EN_IR_CH_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_ir_disable_channel

函数原型

EN_ERR_STA_T rom_hw_ir_disable_channel (stTIMER_Handle_t* pstIR, EN_IR_CH_T enCh);

描述

除能 IR 通道。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
enCh	IR 通道，参考 EN_IR_CH_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

IR 编码 HALAPI

rom_hal_ir_send_init

函数原型

EN_ERR_STA_T rom_hal_ir_send_init (stTIMER_Handle_t* pstIR, uint32_t u32Freq, uint16_t u16Duty, EN_IR_SEND_PATH_T enPath);

描述

初始化指示的 IR 定时器时钟。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
u32Frequency	载波的 PWM 输出频率，单位：Hz
u8Duty	载波的 PWM 占空比

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_ir_start_send

函数原型

EN_ERR_STA_T rom_hal_ir_start_send (stTIMER_Handle_t* pstIR);

描述

启动 IR 发送。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_ir_stop_send

函数原型

EN_ERR_STA_T rom_hal_ir_stop_send (stTIMER_Handle_t* pstIR);

描述

停止 IR 发送。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_ir_config_clock

函数原型

EN_ERR_STA_T rom_hal_ir_config_clock (stTIMER_Handle_t* pstIR, EN_IR_CH_T enCh, uint8_t u8Prescale);

描述

配置指示的通道预分频。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
enCh	IR 通道。
u8Prescale	输入时钟的分频，范围为 0 ~ 15。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_ir_send_next_signal_level

函数原型

EN_ERR_STA_T rom_hal_ir_send_next_signal_level (stTIMER_Handle_t* pstIR, unIR_SendSignalData_t* punIrData);

描述

配置 IR 下一周期比较值。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
punIrData	配置数据，参考 unIR_SendSignalData_t 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

IR 解码 HW API

rom_hw_timer_enable_decode

函数原型

EN_ERR_STA_T rom_hw_timer_enable_decode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

使能定时器解码功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个定时器，参考 EN_TIMER_CH_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_timer_disable_decode

函数原型

EN_ERR_STA_T rom_hw_timer_disable_decode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

除能定时器解码功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个定时器，参考 EN_TIMER_CH_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_timer_set_decode_pol

函数原型

EN_ERR_STA_T rom_hw_timer_set_decode_pol(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_DECODE_POL_T enPol);

描述

配置解码极性。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个定时器，参考 EN_TIMER_CH_T 枚举定义。
enMode	解码极性，参考 EN_DECODE_POL_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_timer_set_decode_mode

函数原型

EN_ERR_STA_T rom_hw_timer_set_decode_mode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, EN_DECODE_MODE_T enMode);

描述

配置解码模式。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个定时器，参考 EN_TIMER_CH_T 枚举定义。
enMode	解码模式，参考 EN_DECODE_MODE_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hw_timer_set_decode_interval

函数原型

EN_ERR_STA_T rom_hw_timer_set_decode_interval(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, uint8_t u8Val);

描述

设置指示的定时器解码间隔。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将使能哪个定时器，参考 EN_TIMER_CH_T 枚举定义。
u8Val	解码间隔值。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

IR 解码 HALAPI

rom_hal_ir_study_init

函数原型

EN_ERR_STA_T rom_hal_ir_study_init(stTIMER_Handle_t* pstIR, stDecodeInit_t* pstDecodeInit, stCapInit_t* pstCapInit);

描述

初始化 IR 学习模式。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
pstDecodeInit	解码初始化结构类型。 .u8Prescale: 输入时钟的分频，u8Prescale 的范围为 0 ~ 15。 .u8Mode: 解码模式，参考 EN_DECODE_MODE_T 枚举定义。 .u8Pol: 解码极性，参考 EN_DECODE_POL_T 枚举定义。 .u8Value: 解码间隔值。
pstCapInit	捕捉初始化结构类型。 .u8Prescale: 输入时钟的分频，u8Prescale 的范围为 0 ~ 15。 .u8Signal: 捕捉信号，@ ref EN_CAP_SIGNAL_T。 .u8Mode: 捕捉模式，@ ref EN_CAP_MODE_T。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_ir_start_study

函数原型

EN_ERR_STA_T rom_hal_ir_start_study(stTIMER_Handle_t* pstIR, uint8_t u8Rtune);

描述

启动 IR 学习模式。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。
u8Rtune	设置 RC 滤波器中 R 的阻值， $R = 50K \times 2^{(u8Rtune)}$ 。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_ir_stop_study

函数原型

EN_ERR_STA_T rom_hal_ir_stop_study(stTIMER_Handle_t* pstIR);

描述

停止 IR 学习模式。

参数

参数	描述
pstIR	IR 句柄，应为 IR0 / IR1 / IR2 / IR3。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_timer_decode_init

函数原型

EN_ERR_STA_T rom_hal_timer_decode_init(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh, stDecodeInit_t* pstDecodeInit);

描述

初始化指示的定时器，配置解码。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个定时器，参考 EN_TIMER_DECODE_CH_T 枚举定义。
pstDecodeInit	解码初始化结构类型。 .u8Prescale: 输入时钟的分频。 16-bit 定时器: u8Prescale 范围为 [0:16)。 32-bit 定时器: u8Prescale 范围为 [0:256)。 .u8Mode: 解码模式，参考 EN_DECODE_MODE_T。 .u8Pol: 解码极性，参考 EN_DECODE_POL_T。 .u8Value: 解码间隔值。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_timer_enable_decode

函数原型

EN_ERR_STA_T rom_hal_timer_enable_decode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

使能定时器解码功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个定时器，参考 EN_TIMER_DECODE_CH_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

rom_hal_timer_disable_decode

函数原型

EN_ERR_STA_T rom_hal_timer_disable_decode(stTIMER_Handle_t* pstTIMER, EN_TIMER_CH_T enCh);

描述

除能定时器解码功能。

参数

参数	描述
pstTIMER	TIMER 句柄，应为 TIMER0 / TIMER1 / TIMER2 / TIMER3。
enCh	指示将配置哪个定时器，参考 EN_TIMER_DECODE_CH_T 枚举定义。

返回

EN_ERR_STA_T	函数返回状态，参考 EN_ERR_STA_T 枚举定义。
--------------	------------------------------

IR 模块范例

IR NEC 编码范例

IR NEC 编码说明

该单片机的 IR 模块支持多种协议。本范例以 NEC IR 协议为例，介绍 IR 模块的使用方法。
在 NEC 协议中，载波频率为 38 kHz，引导码、Code 0、Code 1 和重复码的取值如下：
引导码

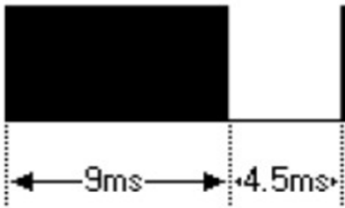


图 14. 引导码

Bit 说明

- Bit 0: 脉冲时间 – 560 μs，间隔时间 – 565 μs，周期时间 – 1125 μs
Bit 1: 脉冲时间 – 560 μs，间隔时间 – 1690 μs，周期时间 – 2250 μs

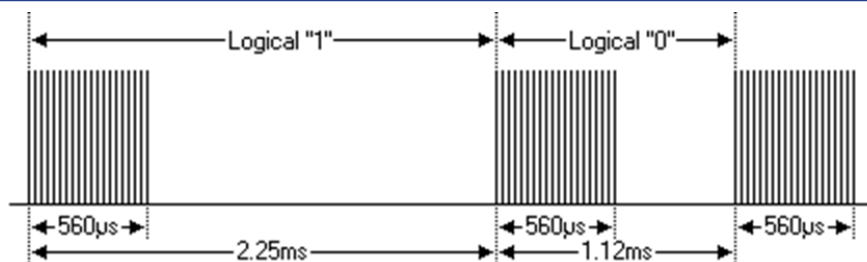


图 15. Bit 说明

重复码

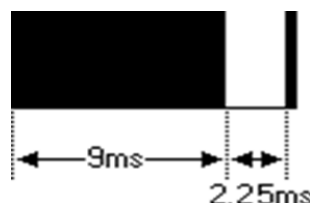


图 16. 重复码

IR NEC 编码范例代码

```
static uint8_t ir_tx_data_signal_generate (void)
{
    // GPIO init
    patch_hw_gpio_set_pin_input_output(GPIO_PORT_IR_OUT, GPIO_PIN_0, GPIO_MODE_
    IMPEDANCE);
    patch_hw_gpio_set_pin_pull_mode(GPIO_PORT_IR_OUT, GPIO_PIN_0, GPIO_PULL_NONE);
    patch_hw_gpio_set_pin_pid(GPIO_PORT_IR_OUT, GPIO_PIN_7, PID_IR_OUT);
    patch_hw_gpio_set_pin_pid(GPIO_PORT_PWM_OUT, GPIO_PIN_9, PID_GTIM_PWM0_CHA);
    patch_hw_gpio_set_pin_pid(GPIO_PORT_IR_SIGNLE_OUT, GPIO_PIN_11,
    PID_GTIM_PWM0_CHB);
    // IR send init, config the leader code
    rom_hal_ir_send_init(IR0, 38000, 50);
    // Config data
    rom_hw_ir_set_pwm_current_compare_and_polarity(IR0, CLEAR_SIGN(u16Data[0]),
    PWM_POL_FALLING);
    // Start send
    rom_hal_start_send(IR0);
}
```

IR 解码范例

IR 解码范例代码

```
static uint8_t decode_func(void)
{
    // Decode Init struct type
    stDecodeInit_t g_stDecodeInit;
    g_stDecodeInit.u16CompVal = u32Compare;
    g_stDecodeInit.u8Prescale = u8Prescale;
    g_stDecodeInit.u8Mode = u8TrigMode;
    g_stDecodeInit.u8Pol = u8OutPol;
```

```
g_stDecodeInit.u8Interval = u8Interval;
g_stDecodeInit.u8SignalSrc = u8SignalSrc;
// Decode GPIO Init
patch_hw_gpio_set_pin_pull_mode(GPIOA, GPIO_PIN_0, GPIO_PULL_NONE);
patch_hw_gpio_set_pin_input_output(GPIOA, GPIO_PIN_0, GPIO_MODE_IMPEDANCE);
patch_hw_gpio_set_pin_pull_mode(GPIOA, GPIO_PIN_1, GPIO_PULL_NONE);
patch_hw_gpio_set_pin_pull_mode(GPIOA, GPIO_PIN_2, GPIO_PULL_NONE);
// Enable ir rx amp.
patch_hw_gpio_enable_ir_rx_amp();
// Set IR Amplifier amplification factor
patch_hw_gpio_set_ir_rx_rtune_amp(1);
// Set indicated pin peripheral function
patch_hw_gpio_set_pin_pid(GPIO_PORT_TIM0_CHB, GPIO_PIN_TIM0_CHB,
PID_GTIM_DECODE0_CHB);
// Ir function enable
rom_hw_ir_enable(stTimerTestCfg.stTIMER_Handle);
// Init a indicated timer, config decode.
rom_hal_timer_decode_init(g_stTimerTestCfg.stTIMER_Handle,
(EN_DECODE_CH_T)g_stTimerTestCfg.EN_TIMER_CH, &g_stDecodeInit);
// Enable timer decode function.
rom_hal_timer_enable_decode(TIMER0, (EN_DECODE_CH_T)TIMER_CHB);
}
```

8 2nd-Boot

该单片机有三种类型的存储器：ROM、Flash 和 SRAM。ROM 用于存储不可修改的代码和数据；Flash 用于存储用户可修改的代码和数据。SRAM 用于存储运行代码和运行数据。该单片机存储器映射如下：

表 5. 存储器映射

存储器类型	起始地址	结束地址
SRAM	0x20000000	0x2003FFFF
Flash	0x10000000	0x100FFFFF
ROM	0x00000000	0x0003FFFF

存储器映射

单片机上电后，CPU 默认起始地址为 0x00000000 (ROM 中)。由于 ROM 代码无法修改，我们设计了用于加载 ROM 中的启动代码的代码，以帮助 CPU 正确跳转到 SDK。这个启动代码称为 2nd-Boot 代码。

2nd-Boot 代码存储在 Flash 中，并在运行时加载到 SRAM 中。2nd-Boot 代码是 ROM 代码和 SDK 之间的链接。单片机代码运行流程如下：

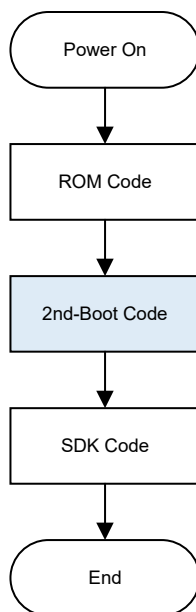


图 17. 启动流程

2nd-Boot 代码

2nd-Boot 代码有两个主要功能。首先，确定 OTA 代码区域中是否有新版本的应用层代码，并进行后续处理。其次是初始化并使能缓存，以便在 Flash 中运行的代码可以加载到缓存中。2nd-Boot 代码运行流程如下：

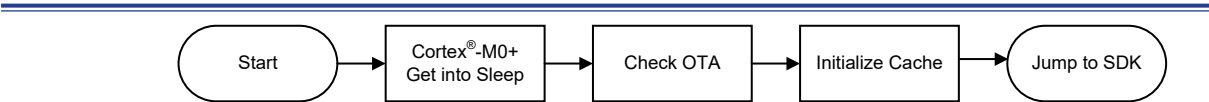


图 18. 2nd-Boot 代码流程

OTA 流程

我们将 Flash 分为四个区域，分别是：2nd-Boot 代码区、OTA 代码信息区、应用代码区和 OTA 代码区（每个区域的大小可由用户修改），具体如下：

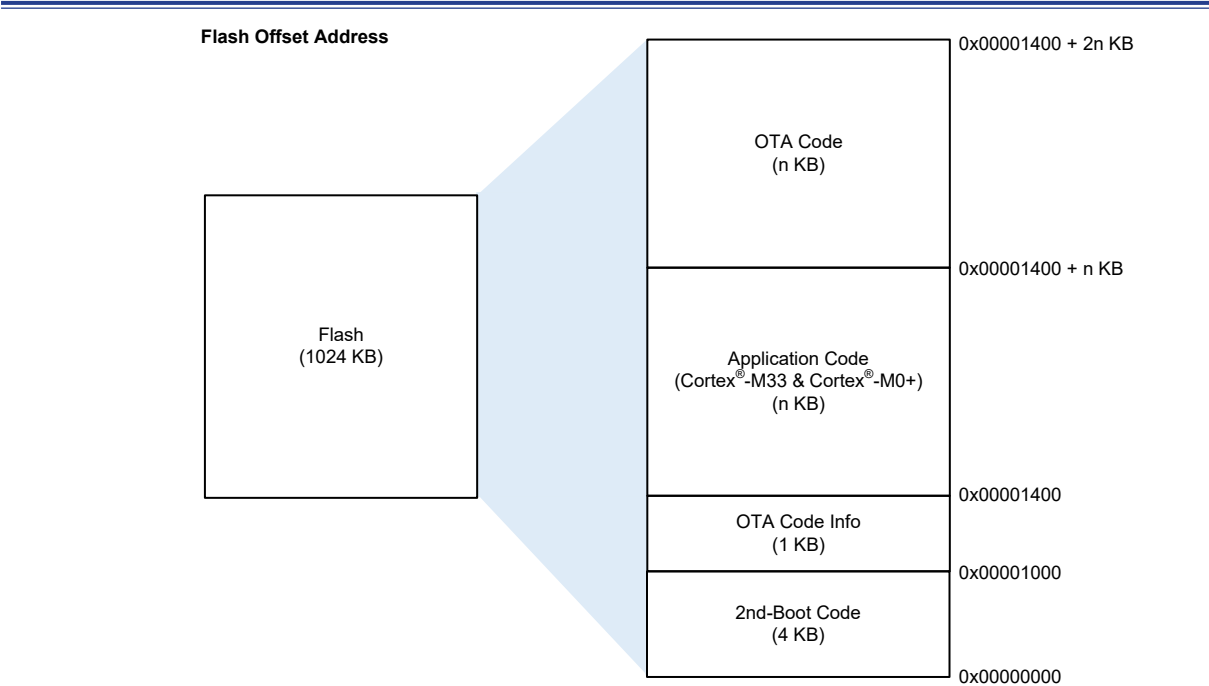


图 19. Flash 存储器映射

2nd-Boot 代码包括 2nd-Boot 代码；OTA 代码信息包括运行代码信息、OTA 代码大小和 CRC；应用代码包括 Cortex-M33 和 Cortex-M0+ 应用代码；OTA 代码包含需要升级的代码。2nd-Boot 的 OTA 流程包括以下 5 个主要步骤：

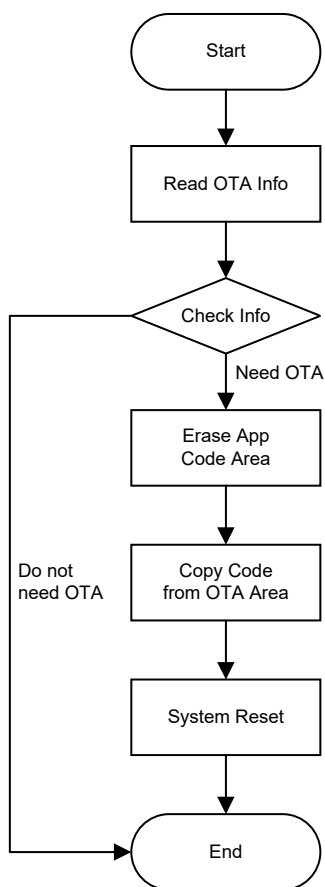


图 20. 2nd-Boot 的 OTA 流程

步骤 1. 从 Flash 中读取 OTA 信息，以检查是否需要升级。

如果不是，执行下一步，以初始化缓存；

如果是，我们将逐步检查 ROM 和 2nd-Boot 代码版本，OTA 代码 CRC。

步骤 2. 擦除应用代码，以准备将 OTA 代码复制到应用代码区。

步骤 3. 在 Flash 中从 OTA 代码区读取 OTA 代码，然后将其写入应用代码区。

步骤 4. 检查 OTA 代码和新应用代码的 CRC。

步骤 5. 如果两者相等，则修改 OTA 信息并复位系统。如果不相等，则重新执行步骤 2。

初始化缓存读取

初始化缓存读取包括设置缓存读取模式为 QSPI，和使能缓存读取。

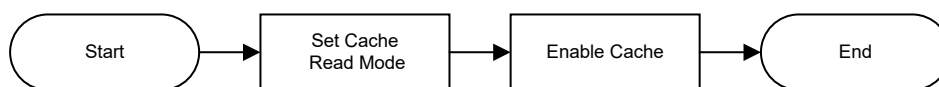


图 21. 初始化缓存

跳转到 SDK

在 2nd-Boot 代码执行完成后，代码将跳转到 SDK。我们将修改 PC 指针来复位 SDK 的处理程序。

9 IPC

简介

该单片机是一款双核心的 MCU，其包括主处理器 MP (Cortex®-M33) 与协处理器 CP (Cortex®-M0+)。MP 具有强大的性能，能够应用在需要较强性能需求的地方，而 CP 主要用于低功耗与通信。在双核的使用过程中，MP 与 CP 会经常进行数据交互，这种数据交互被称作核间通信 (IPC)。核间通信的底层实现由寄存器层面的类互斥锁功能实现，保证多核数据交互的安全性。为使用户更加明确 IPC 通信的使用方式，下面将对该内容 API 进行详细介绍。

IPC API 介绍

基于消息队列实现的 IPC API 介绍

ipc_general_init

函数原型

```
EN_ERR_STA_T ipc_general_init(stIpcInit_t *pstIpcInit);
```

功能描述

初始化 IPC。

参数

参数	描述
pstIpcInit	pu8Buf 指向双核数据共用缓冲器的指针。
	u32BufSize 双核数据共用的缓冲器大小。
	u32MsgCount 队列最大支持的消息数量。
	u32MsgSize 消息的大小。
	enMode QUEUE_MODE_OVERFLOW 数据满了会从最旧的数据开始复写。 QUEUE_MODE_NO_OVERFLOW 数据满了将会拒绝新消息。
	pfnCallback 当另一个核发来消息时会被调用的回调函数。

返回值

Status	参考 EN_ERR_STA_T。
--------	------------------

ipc_general_send_msg_blocking

函数原型

```
EN_ERR_STA_T ipc_general_send_msg_blocking(uint8_t *pu8Data, uint32_t u32DataLen);
```

功能描述

以阻塞形式发送消息。

参数

参数	描述
pu8Data	指向需要发送数据的指针。
u32DataLen	需要发送数据的大小。

返回值

Status	参考 EN_ERR_STA_T。
--------	------------------

ipc_general_send_msg_nonblocking

函数原型

EN_ERR_STA_T ipc_general_send_msg_nonblocking(uint8_t *pu8Data, uint32_t u32DataLen);

功能描述

以非阻塞形式发送消息。

参数

参数	描述
pu8Data	指向需要发送数据的指针。
u32DataLen	需要发送数据的大小。

返回值

Status	参考 EN_ERR_STA_T。
--------	------------------

基于数组实现的 IPC API 介绍

ipc_general_init

函数原型

EN_ERR_STA_T ipc_general_init(uint8_t *pu8DataBuf, uint32_t u32BufMaxLen, ipc_rx_msg_cb_func pfnCallback);

功能描述

初始化 IPC。

参数

参数	描述
pu8DataBuf	指向双核数据共用缓冲器的指针。
u32BufMaxLen	双核数据共用的缓冲器大小。
pfnCallback	当另一个核发来消息时会被调用的回调函数。

返回值

Status	参考 EN_ERR_STA_T。
--------	------------------

ipc_general_send_msg_blocking

函数原型

uint32_t ipc_general_send_msg_blocking(uint8_t *pu8Data, uint32_t u32DataLen);

功能描述

以阻塞形式发送消息。

参数

参数	描述
pu8Data	指向需要发送数据的指针。
u32DataLen	需要发送数据的大小。

返回值

Status	参考 EN_ERR_STA_T。
--------	------------------

IPC 范例代码

MP 与 CP 通过 IPC API 实现数据的交互。

IPC 初始化 (MP 与 CP 相同)

```
// IPC 通信缓冲器, 仅可作为 IPC 通信使用
static uint8_t gu8IpcRxBuf[IPC_MSG_DATA_LEN + 4] __attribute__((aligned(4))) =
{0};
// IPC 初始化
ipc_general_init(gu8IpcRxBuf, sizeof(gu8IpcRxBuf), ipc_receive_msg_callback);
```

IPC 消息接收回调函数

```
// MP IPC 回调函数
static uint32_t ipc_receive_msg_callback(uint8_t *pu8Data, uint32_t u32DataLen)
{
    PRINTF("IPC MP Recv:");
    for(int i=0; i<u32DataLen;i++)
    {
        PRINTF("0x%02X",pu8Data[i]);
    } PRINTF("\r\n");
    return 0;
}
// CP IPC 回调函数
static uint32_t ipc_receive_msg_callback(uint8_t *pu8Data, uint32_t u32DataLen)
{
    PRINTF("IPC CP Recv:");
    for(int i=0; i<u32DataLen;i++)
    {
        PRINTF("0x%02X",pu8Data[i]);
    } PRINTF("\r\n");
    return 0;
}
```

IPC 消息发送函数

```
// MP 通过 IPC 发送消息给 CP
for (i = 0; i < sizeof(gu8IpcTxBuf) - 1; i++)
{
    gu8IpcTxBuf[i] = rom_get_rand(); u8Checksum += gu8IpcTxBuf[i];
}
gu8IpcTxBuf[i] = u8Checksum;
PRINTF("IPC MP Send:");
for(int i=0; i<sizeof(gu8IpcTxBuf);i++)
{
    PRINTF("0x%02X",gu8IpcTxBuf[i]);
} PRINTF("\r\n");
ipc_general_send_msg_blocking(gu8IpcTxBuf, sizeof(gu8IpcTxBuf));
// CP 通过 IPC 发送消息给 MP
for (i = 0; i < sizeof(gu8IpcTxBuf) - 1; i++)
{
    gu8IpcTxBuf[i] = rom_get_rand(); u8Checksum += gu8IpcTxBuf[i];
}
gu8IpcTxBuf[i] = u8Checksum;
```

```
PRINTF("IPC CP Send:");  
for(int i=0; i<sizeof(gu8IpcTxBuf);i++)  
{  
    PRINTF("0x%02X",gu8IpcTxBuf[i]);  
} PRINTF("\r\n");  
ipc_general_send_msg_blocking(gu8IpcTxBuf, sizeof(gu8IpcTxBuf));
```

10 低功耗蓝牙 GAP

简介

低功耗蓝牙 (Bluetooth® Low Energy) GAP 保证不同的蓝牙产品可以互相发现对方并建立连接。GAP 定义了蓝牙设备如何发现和建立与其他设备的安全 / 不安全连接, 它处理一些一般模式的业务 (如询问、命名和搜索) 和一些安全性问题 (如担保), 同时还处理一些有关连接的业务 (如链路建立、通道和连接建立)。该低功耗蓝牙芯片, 蓝牙协议栈部分是在 ROM 中, 通过本文档熟悉如何通过标准的 API 访问蓝牙协议栈, 并熟练使用蓝牙功能。

低功耗蓝牙 GAP API 接口

通用接口

rom_gap_api_set_random_device_address

函数原型

```
bool rom_gap_api_set_random_device_address(uint8_t pu8Address[6]);
```

功能描述

设置设备随机地址。

参数

参数	描述
pu8Address[6]	要设置的随机地址。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口, 需要发消息到底层 Task 执行具体功能, 最后的执行结果通过事件上报的方式获取。
如果设备当前处在广播、扫描或发起态, 则此 API 将无法执行。

rom_gap_api_set_public_device_address

函数原型

```
bool rom_gap_api_get_public_device_address(uint8_t pu8Address[6]);
```

功能描述

设置设备公共地址。

参数

参数	描述
pu8Address[6]	要设置的公共地址。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。
如果设备当前处在广播、扫描或发起态，则此 API 将无法执行。

rom_gap_api_get_random_device_address

函数原型

bool rom_gap_api_get_random_device_address(uint8_t pu8Address[6]);

功能描述

获取设备随机地址。

参数

参数	描述
pu8Address[6]	保存获取的随机地址的指针。

返回值

true 或 false	表示获取成功或失败。
--------------	------------

备注

同步接口，函数执行完就得到了随机地址。

rom_gap_api_get_public_device_address

函数原型

bool rom_gap_api_get_public_device_address(uint8_t pu8Address[6]);

功能描述

获取设备公共地址。

参数

参数	描述
pu8Address[6]	保存获取的公共地址的指针。

返回值

true 或 false	表示获取成功或失败。
--------------	------------

备注

同步接口，函数执行完就得到了公共地址。

rom_gap_api_clear_white_list

函数原型

bool rom_gap_api_clear_white_list(void);

功能描述

清空白名单列表。

参数

无

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

rom_gap_api_add_device_to_white_list

函数原型

```
bool rom_gap_api_add_device_to_white_list(EN_GAP_WHITE_LIST_ADDRESS_TYPE_T  
enumDeviceType, uint8_t pu8DeviceAddress[6]);
```

功能描述

添加一个设备到白名单列表。

参数

参数	描述
enumDeviceType	要添加的设备地址类型。
pu8Address[6]	要添加的设备地址。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

rom_gap_api_remove_device_from_white_list

函数原型

```
bool rom_gap_api_remove_device_from_white_list(EN_GAP_WHITE_LIST_ADDRESS_TYPE_T  
enumDeviceType, uint8_t pu8DeviceAddress[6]);
```

功能描述

从白名单列表删除一个设备。

参数

参数	描述
enumDeviceType	要删除的设备地址类型。
pu8Address[6]	要删除的设备地址。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

广播接口

rom_gap_api_set_advertising_parameters

函数原型

```
bool rom_gap_api_set_advertising_parameters(uint8_t u8AdvertisingIndex,
stGapSetAdvertisingParameters_t* pstParam);
```

功能描述

设置广播参数。

参数

参数	描述
u8AdvertisingIndex	广播的实例索引。
pstParam	广播参数。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

广播参数结构：

```
typedef struct __attribute__((packed))
{
    uint16_t u16AdvertisingIntervalMin625us;
    uint16_t u16AdvertisingIntervalMax625us;
    EN_GAP_ADVERTISING_TYPE_T enumAdvertisingType;
    EN_GAP_OWN_ADDRESS_TYPE_T enumOwnAddressType;
    EN_GAP_ADV_PEER_ADDRESS_TYPE_T enumPeerAddressType;
    uint8_t pu8PeerAddress[6];
    uint8_t u8AdvertisingChannelMap;
    EN_GAP_ADV_FILTER_POLICY_T enumAdvertisingFilterPolicy;
} stGapSetAdvertisingParameters_t;
```

u16AdvertisingIntervalMin625us	最小广播间隔，单位：625 μs。
u16AdvertisingIntervalMax625us	最大广播间隔，单位：625 μs。
enumAdvertisingType	广播类型，分别是可连接可扫描，高频定向，可扫描，不可连接或扫描，低频定向。 GAP_ADVERTISING_TYPE_ADV_IND = 0x00, GAP_ADVERTISING_TYPE_DIRECT_IND_HIGH_DUTY_CYCLE = 0x01, GAP_ADVERTISING_TYPE_ADV_SCAN_IND = 0x02, GAP_ADVERTISING_TYPE_NONCONN_IND = 0x03, GAP_ADVERTISING_TYPE_DIRECT_IND_LOW_DUTY_CYCLE = 0x04。
enumOwnAddressType	本设备的广播地址类型，共 4 种，后两种是可解析私有地址。 GAP_OWN_ADDRESS_TYPE_PUBLIC = 0x00, GAP_OWN_ADDRESS_TYPE_RANDOM = 0x01, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_PUBLIC = 0x02, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_RANDOM = 0x03。
enumPeerAddressType	对端设备的广播地址类型，共 2 种，在定向广播或本设备地址类型为 2 或 3 时有效。 GAP_ADV_PEER_ADDRESS_TYPE_PUBLIC_OR_PUBLIC_IDENTITY = 0x00, GAP_ADV_PEER_ADDRESS_TYPE_RANDOM_OR_RANDOM_IDENTITY = 0x01。
pu8PeerAddress[6]	对端设备的广播地址，在定向广播或本设备地址类型为 2 或 3 时有效。

u8AdvertisingChannelMap	广播所用的通道，按 Bit 设置： GAP_ADV_CHANNEL_MAP_37 = (1U << 0), GAP_ADV_CHANNEL_MAP_38 = (1U << 1), GAP_ADV_CHANNEL_MAP_39 = (1U << 2)。
u8AdvertisingChannelMap	广播所用的通道，按 Bit 设置： GAP_ADV_CHANNEL_MAP_37 = (1U << 0), GAP_ADV_CHANNEL_MAP_38 = (1U << 1), GAP_ADV_CHANNEL_MAP_39 = (1U << 2)。
enumAdvertisingFilterPolicy	广播过滤规则： GAP_ADV_FILTER_POLICY_CONN_ALL_SCAN_ALL = 0x00, GAP_ADV_FILTER_POLICY_CONN_ALL_SCAN_WHITELIST = 0x01, GAP_ADV_FILTER_POLICY_CONN_WHITELIST_SCAN_ALL = 0x02, GAP_ADV_FILTER_POLICY_CONN_WHITELIST_SCAN_WHITELIST = 0x03, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_RANDOM = 0x03。
enumPeerAddressType	对端设备的广播地址类型，共 2 种，在定向广播或本设备地址类型为 2 或 3 时有效。 GAP_ADV_PEER_ADDRESS_TYPE_PUBLIC_OR_PUBLIC_IDENTITY = 0x00, GAP_ADV_PEER_ADDRESS_TYPE_RANDOM_OR_RANDOM_IDENTITY = 0x01。
pu8PeerAddress[6]	对端设备的广播地址，在定向广播或本设备地址类型为 2 或 3 时有效。
u8AdvertisingChannelMap	广播所用的通道，按 Bit 设置： GAP_ADV_CHANNEL_MAP_37 = (1U << 0), GAP_ADV_CHANNEL_MAP_38 = (1U << 1), GAP_ADV_CHANNEL_MAP_39 = (1U << 2)。
enumAdvertisingFilterPolicy	广播过滤规则： GAP_ADV_FILTER_POLICY_CONN_ALL_SCAN_ALL = 0x00, GAP_ADV_FILTER_POLICY_CONN_ALL_SCAN_WHITELIST = 0x01, GAP_ADV_FILTER_POLICY_CONN_WHITELIST_SCAN_ALL = 0x02, GAP_ADV_FILTER_POLICY_CONN_WHITELIST_SCAN_WHITELIST = 0x03。

rom_gap_api_set_advertising_data

函数原型

```
bool rom_gap_api_set_advertising_data(uint8_t u8AdvertisingIndex, uint8_t* pu8AdvertisingData,
uint8_t u8AdvertisingDataLength);
```

功能描述

设置广播数据。

参数

参数	描述
u8AdvertisingIndex	广播的实例索引。
pu8AdvertisingData	广播数据。
u8AdvertisingDataLength	广播数据长度。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

rom_gap_api_set_scan_response_data

函数原型

bool rom_gap_api_set_scan_response_data(uint8_t u8AdvertisingIndex, uint8_t* pu8ScanResponseData, uint8_t u8ScanResponseDataLength);

功能描述

设置广播的扫描回应数据。

参数

参数	描述
u8AdvertisingIndex	广播的实例索引。
pu8ScanResponseData	广播的扫描回应数据。
u8ScanResponseDataLength	广播的扫描回应数据长度。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

rom_gap_api_set_advertising_enable

函数原型

bool rom_gap_api_set_advertising_enable(uint8_t u8AdvertisingIndex, bool bAdvertisingEnable);

功能描述

广播的启动或停止。

参数

参数	描述
u8AdvertisingIndex	广播的实例索引。
bAdvertisingEnable	启动或停止，true 为启动，false 为停止。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取
--

扫描接口

rom_gap_api_set_scan_parameters

函数原型

bool rom_gap_api_set_scan_parameters(stGapSetScanParameters_t* pstParam);

功能描述

设置扫描参数。

参数

参数	描述
pstParam	扫描参数，具体结构参见备注。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

扫描参数结构：

```
typedef struct __attribute__((packed))
{
    EN_GAP_SCAN_TYPE_T enumScanType; uint8_t u8ScanChannelMap;
    uint16_t u16ScanInterval625us;
    uint16_t u16ScanWindow625us;
    EN_GAP_OWN_ADDRESS_TYPE_T enumOwnAddressType;
    EN_GAP_SCAN_FILTER_POLICY_T enumScanningFilterPolicy;
} stGapSetScanParameters_t;
```

u16ScanInterval625us	扫描间隔，单位：625 μs。
u16ScanWindow625us	扫描窗口，单位：625 μs。
enumScanType	扫描类型，分别是被动和主动扫描： GAP_SCAN_TYPE_PASSIVE = 0x00, GAP_SCAN_TYPE_ACTIVE = 0x01。
u8ScanChannelMap	扫描所用的通道，按 Bit 设置： GAP_ADV_CHANNEL_MAP_37 = (1U << 0), GAP_ADV_CHANNEL_MAP_38 = (1U << 1), GAP_ADV_CHANNEL_MAP_39 = (1U << 2)。
enumOwnAddressType	本设备的地址类型，共 4 种，后两种是可解析私有地址。 GAP_OWN_ADDRESS_TYPE_PUBLIC = 0x00, GAP_OWN_ADDRESS_TYPE_RANDOM = 0x01, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_PUBLIC = 0x02, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_RANDOM = 0x03。
enumScanningFilterPolicy	扫描过滤规则： GAP_SCAN_FILTER_POLICY_ACCEPT_ALL = 00, GAP_SCAN_FILTER_POLICY_ACCEPT_WHITELIST = 01, GAP_SCAN_FILTER_POLICY_ACCEPT_ALL_NOT_RESOLVED_DIRECTED = 02, GAP_SCAN_FILTER_POLICY_ACCEPT_WHITELIST_NOT_RESOLVED_DIRECTED = 03。

rom_gap_api_set_scan_enable

函数原型

```
bool rom_gap_api_set_scan_enable(bool bScanEnable, bool bFilterDuplicates);
```

功能描述

扫描的启动或停止。

参数

参数	描述
bScanEnable	启动或停止，true 为启动，false 为停止。
bFilterDuplicates	是否启动过滤重复设备，启动的话重复的设备不再上报。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

发起态接口

rom_gap_api_create_connection

函数原型

```
bool rom_gap_api_create_connection(stGapCreateConnection_t* pstParam);
```

功能描述

进入发起态创建连接。

参数

参数	描述
pstParam	连接创建的参数，具体结构参见备注。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

连接创建参数结构：

```
typedef struct __attribute__((packed))
{
    uint8_t u8ScanChannelMap;
    uint16_t u16ScanInterval625us;
    uint16_t u16ScanWindow625us;
    EN_GAP_INITIATOR_FILTER_POLICY_T enumInitiatorFilterPolicy;
    EN_GAP_INITIATOR_PEER_ADDRESS_TYPE_T enumPeerAddressType;
    uint8_t pu8PeerAddress[6];
    EN_GAP_OWN_ADDRESS_TYPE_T enumOwnAddressType;
    uint16_t u16ConnIntervalMin1250us;
    uint16_t u16ConnIntervalMax1250us; uint16_t u16ConnLatency;
    uint16_t u16SupervisionTimeout10ms;
} stGapCreateConnection_t;
```

u8ScanChannelMap	扫描通道，按 Bit 设置，Bit 0, 1, 2 分别代表通道 37, 38, 39。
u16ScanInterval625us	扫描间隔，单位：625 μs。
u16ScanWindow625us	扫描窗口，单位：625 μs。
enumInitiatorFilterPolicy	发起连接的过滤规则，有两种： GAP_INITIATOR_FILTER_POLICY_WHITELIST_IS_NOT_USED = 0x00, GAP_INITIATOR_FILTER_POLICY_WHITELIST_IS_USED = 0x01,
enumPeerAddressType	对端设备的广播地址类型，共 4 种，后两种为身份地址。 GAP_INITIATOR_PEER_ADDRESS_TYPE_PUBLIC = 0x00, GAP_INITIATOR_PEER_ADDRESS_TYPE_RANDOM = 0x01, GAP_INITIATOR_PEER_ADDRESS_TYPE_PUBLIC_IDENTITY = 0x02, GAP_INITIATOR_PEER_ADDRESS_TYPE_RANDOM_IDENTITY = 0x03,
pu8PeerAddress[6]	对端设备的地址
enumOwnAddressType	本设备的地址类型，共 4 种，后两种是可解析私有地址。 GAP_OWN_ADDRESS_TYPE_PUBLIC = 0x00, GAP_OWN_ADDRESS_TYPE_RANDOM = 0x01, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_PUBLIC = 0x02, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_RANDOM = 0x03,
u16ConnIntervalMin1250us	最小连接间隔，单位：1.25 ms。
u16ConnIntervalMax1250us	最大连接间隔，单位：1.25 ms。
u16ConnLatency	连接的延迟参数。
u16SupervisionTimeout10ms	连接的超时参数，单位：10 ms。

rom_gap_api_create_connection_cancel

函数原型

```
bool rom_gap_api_create_connection_cancel(void);
```

功能描述

取消创建连接，退出发起态。

参数

无

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

连接态接口

rom_gap_api_connection_parameters_update

函数原型

bool rom_gap_api_connection_parameters_update(uint16_t u16ConnHandle, stGapConnectionUpdate_t* pstParam);

功能描述

发起更新连接参数。

参数

参数	描述
u16ConnHandle	连接的索引。
pstParam	更新连接的参数，具体参见下面的备注。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果通过事件上报的方式获取。

更新连接的参数结构：

```
typedef struct __attribute__((packed))
{
    uint16_t u16ConnIntervalMin1250us;
    uint16_t u16ConnIntervalMax1250us;
    uint16_t u16ConnLatency;
    uint16_t u16SupervisionTimeout10ms;
} stGapConnectionUpdate_t;
```

u16ConnIntervalMin1250us	最小连接间隔，单位：1.25 ms。
u16ConnIntervalMax1250us	最大连接间隔，单位：1.25 ms。
u16ConnLatency	连接的延迟参数。
u16SupervisionTimeout10ms	连接的超时参数，单位：10 ms。

rom_gap_api_connection_set_phy

函数原型

bool rom_gap_api_connection_set_phy(uint16_t u16ConnHandle, stGapSetPhy_t* pstParam);

功能描述

设置连接的收发的 PHY。

参数

参数	描述
u16ConnHandle	连接的索引。
pstParam	收发的 PHY 的参数，具体参见下面的备注。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能。
底层 Task 会向对端设备协商设置连接收发 PHY，生效后的 PHY 信息通过事件上报的方式获取。

收发 PHY 的参数结构：

```
typedef struct __attribute__((packed))
{
    uint8_t u8PreferTxPhys; uint8_t u8PreferRxPhys;
    EN_GAP_CODED_PHY_OPTION_T enumCodedPhyOption;
} stGapSetPhy_t;
```

u8PreferTxPhys	希望使用的发数据的 PHY，共 3 种： GAP_PREFER_PHY_1M = 0, GAP_PREFER_PHY_2M = 1, GAP_PREFER_PHY_CODED = 2。
u8PreferRxPhys	希望使用的收数据的 PHY，共 3 种： GAP_PREFER_PHY_1M = 0, GAP_PREFER_PHY_2M = 1, GAP_PREFER_PHY_CODED = 2。
enumCodedPhyOption	前面的参数 u8PreferTxPhys 如果希望使用 Coded PHY，具体更希望使用哪个： GAP_CODED_PHY_OPTION_TX_NO_PREFER = 0, GAP_CODED_PHY_OPTION_TX_PREFER_S2 = 1, GAP_CODED_PHY_OPTION_TX_PREFER_S8 = 2。

rom_gap_api_read_remote_version

函数原型

```
bool rom_gap_api_read_remote_version(uint16_t u16ConnHandle);
```

功能描述

读取对端设备版本信息。

参数

参数	描述
u16ConnHandle	连接的索引。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能。
底层 Task 会向对端设备发消息获取版本信息，收到对方回复得到的版本信息通过事件上报的方式获取。

rom_gap_api_read_remote_features

函数原型

```
bool rom_gap_api_read_remote_features(uint16_t u16ConnHandle);
```

功能描述

读取对端设备支持的特征。

参数

参数	描述
u16ConnHandle	连接的索引。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能。
底层 Task 会向对端设备发消息获取其支持的特征，收到对方回复后，得到双方都支持的特征信息，通过事件上报的方式获取。

rom_gap_api_set_data_length

函数原型

bool rom_gap_api_set_data_length(uint16_t u16ConnHandle, uint16_t u16TxOctets);

功能描述

设置 PDU 的最大发包长度。

参数

参数	描述
u16ConnHandle	连接的索引。
u16TxOctets	PDU 的最大发包长度。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能。
底层 Task 会向对端设备协商各自支持的最大发包长度，最终确定好的各自支持的最大发包长度信息，通过事件上报的方式获取。

rom_gap_api_get_mtu

函数原型

uint16_t rom_gap_api_get_mtu(uint16_t u16ConnHandle);

功能描述

读取当前连接的 MTU。

参数

参数	描述
u16ConnHandle	连接的索引。

返回值

uint16_t	读取的 MTU。
----------	----------

备注

同步接口。

rom_gap_api_read_rssi

函数原型

bool rom_gap_api_read_rssi(uint16_t u16ConnHandle);

功能描述

读取当前连接的信号强度 RSSI。

参数

参数	描述
u16ConnHandle	连接的索引。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果及 RSSI 值，通过事件上报的方式获取。

rom_gap_api_update_channel_map

函数原型

bool rom_gap_api_update_channel_map(uint16_t u16ConnHandle, uint8_t pu8ChannelMap[5]);

功能描述

设置连接使用的通道。

参数

参数	描述
u16ConnHandle	连接的索引。
pu8ChannelMap	要设置的连接通道。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果，通过事件上报的方式获取。

rom_gap_api_read_channel_map

函数原型

bool rom_gap_api_read_channel_map(uint16_t u16ConnHandle);

功能描述

读取当前连接的使用的通道。

参数

参数	描述
u16ConnHandle	连接的索引。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果，通过事件上报的方式获取。
--

rom_gap_api_disconnect

函数原型

```
bool rom_gap_api_disconnect(uint16_t u16ConnHandle, EN_GAP_DISCONNECT_REASON_T enumDisconnectReason);
```

功能描述

断开当前连接。

参数

参数	描述
u16ConnHandle	连接的索引。
enumDisconnectReason	断连的原因。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果，通过事件上报的方式获取。
--

配对和加密连接相关接口

rom_gap_api_update_pair_para

函数原型

```
void rom_gap_api_update_pair_para(uint8_t iocap, uint8_t authreq, uint8_t keysize);
```

功能描述

设置配对相关参数。

参数

参数	描述
iocap	设备的 I/O 能力，包括输入 / 输出能力，取值 0~4，如下： IO_CAPABILITY_DISPLAY_ONLY = 0, IO_CAPABILITY_DISPLAY_YES_NO = 1, IO_CAPABILITY_KEYBOARD_ONLY = 2, IO_CAPABILITY_NO_INPUT_NO_OUTPUT = 3, IO_CAPABILITY_KEYBOARD_DISPLAY = 4。
authreq	配对的认证要求，Bit 0~3 有效，可以两个或多个 Bit 组合。 Bit 0: SM_AUTHREQ_BONDING Bit 1: SM_AUTHREQ_MITM_PROTECTION Bit 2: SM_AUTHREQ_SECURE_CONNECTION Bit 3: SM_AUTHREQ_KEYPRESS
Keysize	最小的加密密钥的长度，范围 7~16。

返回值

无

备注

同步接口。一般要在设备上电初始化时调用。这里设置的参数和是否有对端的 OOB 数据决定了使用哪种配对模式和配对方式。

rom_gap_api_register_oob_data_callback

函数原型

```
void rom_gap_api_register_oob_data_callback(int (*get_oob_data_callback) (uint8_t peer_addr_type, uint8_t peer_addr[6], uint8_t oob_data[16]));
```

功能描述

设置传统配对模式下获取对端设备 OOB 数据的回调函数。

参数

参数	描述
int (*get_oob_data_callback)	获取对端设备 OOB 数据的回调函数，配对过程中调用。

返回值

无

备注

同步接口。一般要在设备上电初始化时调用。如果未调用此 API 设置回调函数，则表示无对端的 OOB 数据。
回调函数的参数 oob_data[16] 就是对端的 OOB 数据。
传统配对模式下，如果设置了这个回调函数，则表示有对端设备的 OOB 数据，使用 OOB 配对方式 (双方设备都有对端的 OOB 数据才能使用 OOB 配对方式) 配对时使用 OOB 数据作为 TK 生成 Confirm 数据。

rom_gap_api_sm_generate_local_sc_oob_data_callback

函数原型

```
void rom_gap_api_register_sc_oob_data_callback(int (*get_sc_oob_data_callback) (uint8_t address_type, uint8_t addr[6], uint8_t * oob_sc_peer_confirm, uint8_t * oob_sc_peer_random));
```

功能描述

设置安全配对模式下获取对端设备 OOB 数据的回调函数，对端 OOB 数据可以在对端设备上通过 rom_gap_api_sm_generate_local_sc_oob_data 获取。

参数

参数	描述
int (*get_oob_data_callback)	获取对端设备 OOB 数据的回调函数，配对过程中调用。

返回值

无

备注

同步接口。一般要在设备上电初始化时调用。如果未调用此 API 设置回调函数，则表示无对端的 OOB 数据。
回调函数的参数 `confirm_value` 和 `random_value` 就是对端的 OOB 数据，分别是生成 Confirm 确认值和使用随机数 `random`，确认值是随机数和公共密钥共同生成的。
安全配对模式下，如果设置了这个回调函数，则表示有对端设备的 OOB 数据，使用 OOB 配对方式 (双方设备只要一方有对端的 OOB 数据就能使用 OOB 配对方式) 配对时使用回调函数获取的确认值和随机数进行认证校验。

rom_gap_api_sm_generate_local_sc_oob_data

函数原型

```
void rom_gap_api_sm_generate_local_sc_oob_data (void (*callback) (const uint8_t *confirm_value, const uint8_t *random_value));
```

功能描述

生成安全配对模式下本端 OOB 数据。

参数

参数	描述
<code>void (*callback)</code>	需要注册回调函数用于接收本端 OOB 数据：确认值和随机数，回调函数的参数 <code>confirm_value</code> 和 <code>random_value</code> 就是这些 OOB 数据，确认值是随机数和公共密钥共同生成的。

返回值

无

备注

同步接口。通过带外 (非蓝牙传输) 方式给到对端，对方才能有本端的 OOB 数据。

rom_gap_api_send_security_req

函数原型

```
void rom_gap_api_send_security_req (uint16_t u16ConnHandle);
```

功能描述

对于主机设备：启动配对，若已经配对过则启动加密。
对于从机设备：发送 security request (安全请求) 给主机设备，使得主机设备启动配对或加密。

参数

参数	描述
<code>u16ConnHandle</code>	连接的索引。

返回值

无

备注

同步接口。

rom_gap_api_input_passkey_during_pair

函数原型

void rom_gap_api_input_passkey_during_pair(uint16_t u16ConnHandle, uint32_t u32Passkey);

功能描述

设置配对的 PIN 码。配对过程中，需要本端输入 PIN 码的情况下，将输入的 PIN 码通过此 API 传递给协议栈完成后续配对过程。

参数

参数	描述
u16ConnHandle	连接的索引。
u32Passkey	输入的 PIN 码的值，PIN 码取值范围：000000~999999。

返回值

无

备注

同步接口。

rom_gap_api_sm_numeric_comparison_confirm

函数原型

void rom_gap_api_sm_numeric_comparison_confirm(uint16_t u16ConnHandle ,uint8_t confirm_sucess);

功能描述

安全配对过程中，数值比较的配对方式，两端设备各自显示一个确认码，需要本端比较是否与对端的确认码一致，比较结果通过此 API 传递给协议栈完成配对。

参数

参数	描述
u16ConnHandle	连接的索引。
confirm_sucess	为 1 表示本端与对端的确认码一致，为 0 表示不一致。

返回值

无

备注

同步接口。

rom_gap_api_set_secure_connections_only

函数原型

void rom_gap_api_set_secure_connections_only (uint8_t u8Enable);

功能描述

设置是否仅使用安全配对模式，若仅使用安全配对模式，则如果配对时未选择安全配对模式就不能进行配对。

参数

参数	描述
u8Enable	为 1 表示设置仅使用安全配对模式，为 0 表示不设置。

返回值

无

备注

同步接口。

rom_gap_api_long_term_key_request_reply

函数原型

void rom_gap_api_long_term_key_request_reply(uint16_t u16ConnHandle, uint8_t* pu8Ltk);

功能描述

设置用于加密连接的 LTK，仅用于从机设备。与对端设备配对成功后，再次连接，收到请求 LTK 的回调消息 MSG_LTK_REQ_WHEN_RECONNECT_AFTER_PAIR_IND 时将保存的 LTK 通过此 API 传递给协议栈完成连接的加密过程。

参数

参数	描述
u16ConnHandle	连接的索引。
pu8Ltk	用于加密连接的 LTK。

返回值

无

备注

同步接口。

rom_gap_api_long_term_key_request_negative_reply

函数原型

void rom_gap_api_long_term_key_request_negative_reply(uint16_t u16ConnHandle);

功能描述

连接后收到请求 LTK 的回调消息 MSG_LTK_REQ_WHEN_RECONNECT_AFTER_PAIR_IND 时，如果找不到对应保存的 LTK，则调用此 API，表示没有 LTK 不能完成连接的加密。也仅用于从机设备。

参数

参数	描述
u16ConnHandle	连接的索引。

返回值

无

备注

同步接口。

rom_gap_api_set_pair_pin_code

函数原型

void rom_gap_api_set_pair_pin_code(bool is_use_random_num, uint32_t u32PinNum);

功能描述

设置用于配对时的本端生成 PIN 码的方式。

参数

参数	描述
is_use_random_num	true 表示随机生成方式，每次配对本端需要生成 PIN 码时，都生成随机数作为 PIN 码，忽略第二个参数；false 表示使用固定 PIN 码的方式，固定的 PIN 码的值为第二个参数表示的值。
u32PinNum	固定 PIN 码的值。

返回值

无

备注

同步接口。

rom_gap_api_sm_bond_info_save_by_app_config_only_for_legacy_pair

函数原型

void rom_gap_api_sm_bond_info_save_by_app_config_only_for_legacy_pair(uint32_t onoff);

功能描述

传统配对方式下，配对后再次连接请求 LTK 时，协议栈可以从携带的 RAND 和 EDIV 信息里面计算出 LTK，所以可以不用 APP 保存配对信息。此 API 配置是否由 APP 保存配对信息，如果不保存的话，下次重新连接加密时协议栈自动处理计算出 LTK 完成加密，不会再发出请求 LTK 的回调消息 MSG_LTK_REQ_WHEN_RECONNECT_AFTER_PAIR_IND，否则配对信息由 APP 保存，下次重新连接加密时发出请求 LTK 的回调消息让 APP 来发送保存的 LTK 给协议栈，完成连接加密。

参数

参数	描述
onoff	为 0 表示不用 APP 保存配对信息，否则表示需要。

返回值

无

备注

同步接口。

rom_gap_api_set_bond_info_for_setup_encryption

函数原型

void rom_gap_api_set_bond_info_for_setup_encryption (uint16_t u16ConnHandle, stGapBondInfo_t* pstBondinfo);

功能描述

此 API 需要在进入连接时调用，用于给协议栈加载已经保存的配对信息 (包含加密连接所需的 LTK 和 RAND、EDIV 等参数)，用于加密流程。

参数

参数	描述
u16ConnHandle	连接的索引。
pstBondinfo	配对信息，具体参见下面的备注。

返回值

无

备注

同步接口。	
配对信息结构：	
typedef struct __attribute__((packed))	
{	
uint16_t u16Ediv;	
uint8_t pu8Rand[8];	
uint8_t pu8Ltk[16];	
uint8_t u8KeySize;	
uint8_t u8Authenticated;	
uint8_t u8Authorized;	
} stGapBondInfo_t;	
u16Ediv	EDIV 参数。
pu8Rand[8]	随机数 RAND 参数。
pu8Ltk[16]	加密密钥 LTK。
u8KeySize	密钥的有效长度。
u8Authenticated	是否经过认证。
u8Authorized	是否经过授权。

隐私功能相关接口

rom_gap_api_add_device_to_resolving_list

函数原型

bool rom_gap_api_add_device_to_resolving_list(uint8_t Peer_Identity_Address_Type , uint8_t *Peer_Identity_Address , uint8_t *Peer_IRK , uint8_t* Local_IRK);

功能描述

添加对端设备到私有地址解析列表。

参数

参数	描述
Peer_Identity_Address_Type	对端设备身份地址类型。
Peer_Identity_Address	对端设备身份地址。
Peer_IRK	对端设备 IRK。
Local_IRK	本端设备 IRK。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果及通道信息通过事件上报的方式获取。

rom_gap_api_remove_device_from_resolving_list

函数原型

bool rom_gap_api_remove_device_from_resolving_list (uint8_t Peer_Identity_Address_Type , uint8_t Peer_Identity_Address[6]);

功能描述

从私有地址解析列表删除一个设备。

参数

参数	描述
Peer_Identity_Address_Type	要删除的对端设备身份地址类型。
Peer_Identity_Address	要删除的对端设备身份地址。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果及通道信息通过事件上报的方式获取。

rom_gap_api_clear_resolving_list

函数原型

bool rom_gap_api_clear_resolving_list (void);

功能描述

清空私有地址解析列表。

参数

无

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果及通道信息通过事件上报的方式获取。

gap_api_set_period_to_update_resolved_address

函数原型

void gap_api_set_period_to_update_resolved_address(uint32_t u32Ms);

功能描述

设置私有地址更新周期。

参数

参数	描述
u16Second	私有地址更新周期 (单位： 秒)，取值范围 1~0xA1B8。

返回值

无

备注

同步接口。

rom_gap_api_set_privacy_mode_for_peer_device

函数原型

bool rom_gap_api_set_privacy_mode_for_peer_device (uint8_t Peer_Identity_Address_Type ,
uint8_t Peer_Identity_Address[6], uint8_t Privacy_Mode);

功能描述

设置解析列表中设备的隐私模式。

参数

参数	描述
Peer_Identity_Address_Type	对端设备身份地址类型。
Peer_Identity_Address	对端设备身份地址。
Privacy_Mode	隐私模式，0 为网络隐私模式，1 为设备隐私模式，其它值保留。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果及通道信息通过事件上报的方式获取。

rom_gap_api_set_addr_resolution_enable

函数原型

bool rom_gap_api_set_addr_resolution_enable (uint8_t enable);

功能描述

启动或停止私有地址解析。

参数

参数	描述
enable	0 为停止，1 为启动，其它值保留。

返回值

true 或 false	表示发消息到底层 Task 成功或失败。
--------------	----------------------

备注

异步接口，需要发消息到底层 Task 执行具体功能，最后的执行结果及通道信息通过事件上报的方式获取。

Stack 消息

MSG_BLE_CONNECTED_IND

表示连接建立的消息。当连接建立成功时会收到这个消息，里面携带了连接的参数及对端的地址信息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Subevent_Code;
    uint8_t Status;
    uint16_t Connection_Handle;
    uint8_t Role;
    uint8_t Peer_Address_Type;
    uint8_t Peer_Address[6];
    uint8_t Peer_RPA_Resoluted;
    uint8_t Peer_Identify_Address_Type;
    uint8_t Peer_Identify_Address[6];
    uint16_t Conn_Interval;
    uint16_t Conn_Latency;
    uint16_t Supervision_Timeout;
    uint8_t Master_Clock_Accuracy;
    uint8_t Own_Address_Type;
    uint8_t Own_Address_Be_RPA; uint8_t Own_Address[6];
} stHciEventParamVendorConnectionComplete_t;
```

其主要参数如下所示：

Connection_Handle	连接的索引。
Role	连接的角色，0 为主机，1 为从机。
Peer_Address_Type	对端设备地址类型，0 为公共地址，1 为随机地址。
Peer_Address[6]	对端设备地址（小端顺序），若是随机地址，且最高 2 个 Bit 是 01，则此地址为可解析的私有地址 RPA。
Peer_RPA_Resoluted	Peer_Address 是否解析成功（前提条件：Peer_Address 是 RPA）。
Peer_Identify_Address_Type	若 Peer_Address 是 RPA 且解析成功，其对应的身份地址类型。
Peer_Identify_Address[6]	若 Peer_Address 是 RPA 且解析成功，其对应的身份地址。
Conn_Interval	连接间隔，单位：1.25 ms。
Conn_Latency	连接的延迟参数。
Supervision_Timeout	连接的超时参数，单位：10 ms。
Master_Clock_Accuracy	对端主机设备的时钟精度，仅在本端是从机角色时有效。
Own_Address_Type	本端地址类型，0 为公共地址，1 为随机地址。
Own_Address_Be_RPA	本端地址是否为 RPA。
Own_Address[6]	本端地址。

MSG_BLE_DISCONNECTED_IND

表示连接断开的消息。当连接断开时会收到这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Status;
    uint16_t Connection_Handle;
    uint8_t Reason;
} stHciEventParamDisconnectionComplete_t;
其主要参数如下所示:
```

Connection_Handle	连接的索引。
Reason	断连的原因。

MSG_BLE_CONNECTION_UPDATE_COMPLETE_IND

表示连接参数更新完成的消息。当底层协议栈完成连接参数更新后会上报这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Subevent_Code;
    uint8_t Status;
    uint16_t Connection_Handle;
    uint16_t Conn_Interval;
    uint16_t Conn_Latency;
    uint16_t Supervision_Timeout;
} stHciEventParamLEConnectionUpdateComplete_t;
其主要参数如下所示:
```

Connection_Handle	连接的索引。
Conn_Interval	连接间隔，单位：1.25 ms。
Conn_Latency	连接的延迟参数。
Supervision_Timeout	连接的监督超时时间，单位：10 ms。

MSG_BLE_LL_VERSION_IND

表示对端蓝牙版本号的消息。当底层协议栈收到对方的版本号会上报这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Status;
    uint16_t Connection_Handle;
    uint8_t Version;
    uint16_t Manufacturer_Name;
    uint16_t Subversion;
} stHciEventParamReadRemoteVersionInformationComplete_t;
其主要参数如下所示:
```

Connection_Handle	连接的索引。
Version	链路层 (Link Layer) 版本号，例如 10 代表 5.1 版本，11 代表 5.2 版本。
Manufacturer_Name	蓝牙芯片制造商的 ID (SIG 给每个厂商分配的 ID 号)。
Subversion	子版本号。

MSG_BLE_LL_FEATRUE_IND

表示对端蓝牙特性的消息。当底层协议栈获取对方的蓝牙特性会上报这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Subevent_Code;
    uint8_t Status;
    uint16_t Connection_Handle;
    uint64_t LE_Features;
} stHciEventParamLEReadRemoteFeaturesComplete_t;
其主要参数如下所示:
```

Connection_Handle	连接的索引。
LE_Features	支持的蓝牙特性，共 64 Bit，每个 Bit 代表支持的一种特征。

MSG_BLE_DATA_LENGTH_UPDATE_COMPLETE_IND

表示链路层数据包长度更新完成的消息。当底层协议栈完成链路层数据包长度更新后会上报这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Subevent_Code;
    uint16_t Connection_Handle;
    uint16_t MaxTxOctets;
    uint16_t MaxTxTime;
    uint16_t MaxRxOctets;
    uint16_t MaxRxTime;
} stHciEventParamLEDataLengthChange_t;
其主要参数如下所示:
```

Connection_Handle	连接的索引。
MaxTxOctets	发送数据包最大长度。
MaxTxTime	发送数据包的最大时长，单位：μs。
MaxRxOctets	接收数据包的最大长度。
MaxRxTime	接收数据包的最大时长，单位：μs。

MSG_BLE_PHY_UPDATE_COMPLETE_IND

表示 PHY 更新完成的消息。当底层协议栈完成 PHY 更新后会上报这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Subevent_Code;
    uint8_t Status;
    uint16_t Connection_Handle;
    uint8_t TX_PHY;
    uint8_t RX_PHY;
} stHciEventParamLEPhyUpdateComplete_t;
其主要参数如下所示:
```

Connection_Handle	连接的索引。
TX_PHY	发数据使用的 PHY，其值为 1, 2, 3 分别代表 1M, 2M 和 LE Coded。
RX_PHY	收数据使用的 PHY，其值为 1, 2, 3 分别代表 1M, 2M 和 LE Coded。

MSG_BLE_LLCAP_CONN_UPDATE_RSP_IND

表示作为从机设备在 LLCAP 层发起的连接更新请求收到的回应消息。此消息主要显示对端设备是否接受本端发的连接更新请求：如果接受，则对端会发起连接参数更新流程（此流程完成时会上来 MSG_BLE_CONNECTION_UPDATE_COMPLETE_IND）；否则如果拒绝，则不会发起连接参数更新流程。其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint16_t conn_handle;
    uint16_t result;
} st_llcap_conn_update_rsp_event;
```

其主要参数如下所示：

Connection_Handle	连接的索引。
result	为 0 表示接受，否则表示拒绝。

MSG_BLE_MTU_EXCHANGED_IND

表示 MTU 更新完成的消息。当底层协议栈完成 MTU 更新后会上报这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint16_t conn_handle;
    uint16_t mtu;
} st_mtu_exchange_complete_event;
```

其主要参数如下所示：

Connection_Handle	连接的索引。
mtu	更新后的 MTU。

MSG_BLE_ENCRYPTED_CHANGED_IND

表示连接加密状态改变的消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Status;
    uint16_t Connection_Handle;
    uint8_t Encryption_Enabled;
} stHciEventParamEncryptionChange_t;
```

其主要参数如下所示：

Status	为 0 表示加密状态改变了，为 6 表示 PIN 码错误导致加密失败。
Connection_Handle	连接的索引。
Encryption_Enabled	改变后的加密状态，为 1 表示已经加密，为 0 表示没有加密。

MSG_BLE_ENCRYPTED_REFRESH_IND

表示连接加密的密钥更新完成的消息。当底层协议栈暂停加密后再更新密钥重新进入加密状态后会上报这个消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint8_t Status;
    uint16_t Connection_Handle;
} stHciEventParamEncryptionKeyRefreshComplete_t;
```

其主要参数如下所示：

Connection_Handle	连接的索引。
-------------------	--------

MSG_BLE_PAIR_USER_PASSKEYREQ_IND

配对时请求本端输入 PIN 码的消息。对于需要本端输入 PIN 码的配对方式，当收到此消息时，输入 PIN 码完成配对。此消息只有一个连接索引的参数 Connection_Handle。

MSG_BLE_PAIR_USER_PASSKEY_DISPLAY_IND

配对时请求本端输入 PIN 码的消息。对于需要本端输入 PIN 码的配对方式，当收到此消息时，密钥由对方指定，本端只需显示它即可。其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint16_t conn_handle;
    uint8_t passkey[4];
} st_passkey_display_event;
其主要参数如下所示：
```

Connection_Handle	连接的索引。
passkey[4]	显示的 PIN 码数值，范围 0 ~ 999999，小字节顺序。

MSG_BLE_PAIR_USER_PASSKEYREQ_CONF_IND

安全配对时显示确认码并要求本端确认的消息。在安全配对模式的数值比较配对方式下，两端设备都会生成一个确认码，并显示出来，需要用户进行对比确认。这个消息就是用来显示确认码并要求本端进行对比确认的，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint16_t conn_handle;
    uint8_t passkey[4];
} st_passkey_display_event;
其主要参数如下所示：
```

Connection_Handle	连接的索引。
passkey[4]	确认码。

MSG_BLE_PAIR_COMPLETED_IND

配对成功的消息。当底层协议栈完成配对流程，获取绑定信息后上报此消息，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint16_t conn_handle;
    uint8_t key_size;
    uint8_t authenticated;
    uint8_t authorized;
    uint8_t peer_addr_type;
    uint8_t peer_addr[6];
    uint8_t peer_irk[16];
    uint8_t local_irk[16];
    uint8_t ltk[16];
    uint8_t rand[8];
    uint16_t ediv;
} st_pair_complete_event;
```

其主要参数如下所示：

Connection_Handle	连接的索引。
key_size	密钥 LTK 的有效长度，范围 7~16，LTK 的实际长度都是 16，有效长度小于 16 的，补 0。
authenticated	是否认证过。
authorized	是否授权过。
peer_addr_type	对端设备的身份地址类型。
peer_addr[6]	对端设备的身份地址。
peer_irk[16]	对端设备的 IRK。
local_irk[16]	本端设备的 IRK。
ltk[16]	LTK 密钥，下次加密连接时需要用到。
rand[8]	随机数 RAND 参数，下次加密连接时需要用到。
ediv	EDIV 参数，下次加密连接时需要用到。

MSG_BLE_PAIR_FAIL_IND

配对失败的消息。此消息主要指示了配对失败及其原因，其数据是一个结构体，见以下说明。

```
typedef struct __attribute__((packed))
{
    uint16_t conn_handle;
    uint8_t reason;
} st_pair_failed_event;
```

其主要参数如下所示：

Connection_Handle	连接的索引。
reason	配对失败的原因

MSG_LTK_REQ_WHEN_RECONNECT_AFTER_PAIR_IND

请求 LTK 的消息，只有从机角色设备才会有此消息。当连接启动加密时底层协议栈上报此消息请求 LTK，此时需要把 LTK 数据通过 rom_gap_api_long_term_key_request_reply 发给协议栈完成加密，其数据是一个结构体，见以下说明。

启动加密是由主机设备发起的，主机通过发送 LL_ENC_REQ 控制包发起，这个控制包内包含下面结构体里的随机数 RAND 参数和 EDIV 参数，这两个参数也是配对完成时的绑定信息里同名的参数。对于传统配对方式，这两个参数可以用来索引 LTK，而对于使用安全配对模式，这两个参数是全 0 的数据，只能通过设备地址来索引到对应的 LTK。

```
typedef struct __attribute__((packed))
{
    uint16_t conn_handle;
    uint8_t rand[8];
    uint16_t ediv;
} st_ltk_req_event;
```

其主要参数如下所示：

Connection_Handle	连接的索引。
rand[8]	随机数 RAND 参数。
ediv	EDIV 参数。

MSG_BLE_ADV_REPORT_IND

上报广播数据的消息，设备启动扫描后，通过此消息上报扫描到广播数据。可能一次上报多个广播，所以其数据格式比较特殊，前两个参数是固定的，后面 6 个参数可

能会有多份（份数就是广播个数 Num_Reports）。具体参数结构见以下说明。

Subevent_Code	HCI event 的消息码。
Num_Reports	此参数后面的数据可能是多个广播的，这个参数是广播数据个数。
Event_Type[i]	第 i 个广播的广播类型，共 5 种类型： HCI_ADVERTISING_REPORT_EVENT_TYPE_ADV_IND = 0x00, HCI_ADVERTISING_REPORT_EVENT_TYPE_ADV_DIRECT_IND = 0x01, HCI_ADVERTISING_REPORT_EVENT_TYPE_ADV_SCAN_IND = 0x02, HCI_ADVERTISING_REPORT_EVENT_TYPE_ADV_NONCONN_IND = 0x03, HCI_ADVERTISING_REPORT_EVENT_TYPE_SCAN_RSP = 0x04。
Address_Type[i]	第 i 个广播的广播设备地址类型。
Address[i]	第 i 个广播的广播设备地址，长度固定的 6 个字节。
Length[i]	第 i 个广播的广播数据长度。
Data[i]	第 i 个广播的广播数据，其长度是前一个参数 Length[i]。
RSSI[i]	第 i 个广播的信号强度，这是一个有符号 8 位数，表示负值。

MSG_BLE_HIGH_DUTY_DIRECT_ADV_END_IND

上报高频定向广播超时结束的消息（超时时间是 1.28 秒），高频定向广播如果在 1.28 秒内没有收到连接请求进入连接状态则会自动停止，此消息无附加数据。

范例

通用接口

设置设备地址

设置设备地址，包括随机地址和公共地址的例程如下：

```
uint8 pulic_addr[6] = {0x66, 0x55, 0x44, 0x33, 0x22, 0x11};
uint8 random_addr[6] = {0x16, 0x15, 0x14, 0x13, 0x12, 0x11};
// 设置设备公共地址
rom_gap_api_set_public_device_address(pulic_addr);
// 设置设备随机地址
rom_gap_api_set_random_device_address(random_addr);
```

管理白名单

添加、删除、清空白名单简单事例程序如下：

```
uint8 public_addr[6] = {0x13, 0x71, 0xda, 0x7d, 0x1a, 0x00};
// 添加设备到白名单:
rom_gap_api_add_device_to_white_list(GAP_WHITE_LIST_ADDRESS_TYPE_PUBLIC,
public_addr);
// 从白名单删除一个设备:
rom_gap_api_remove_device_from_white_list(GAP_WHITE_LIST_ADDRESS_TYPE_PUBLIC,
public_addr);
// 清空白名单:
rom_gap_api_clear_white_list();
```

广播

启动广播

用户可以根据自己的应用自定义需要启动的广播类型以及自定义广播数据，具体类型可以参考前文 API 介绍。设置广播的参数和广播的报文数据并开启广播，简单事例程序如下：

```
// 设置广播参数:
stGapSetAdvertisingParameters_t stAdvParam =
{
    .u16AdvertisingIntervalMin625us = 200 * 8 / 5,
    .u16AdvertisingIntervalMax625us = 200 * 8 / 5,
    .enumAdvertisingType = GAP_ADVERTISING_TYPE_ADV_IND,
    .enumOwnAddressType = GAP_OWN_ADDRESS_TYPE_PUBLIC,
    .u8AdvertisingChannelMap = 7,
    .enumAdvertisingFilterPolicy = GAP_ADV_FILTER_POLICY_CONN_ALL_SCAN_ALL,
};
rom_gap_api_set_advertising_parameters(0, &stAdvParam);
// 设置广播数据和扫描回应数据:
uint8_t pu8AdvData[8] =
{
    2, GAP_ADTYPE_FLAGS,
    GAP_ADTYPE_FLAGS_GENERAL | GAP_ADTYPE_FLAGS_BREDR_NOT_SUPPORTED,
    4, GAP_ADTYPE_LOCAL_NAME_SHORT, 'R', 'C', 'U'
};
rom_gap_api_set_advertising_data(0, pu8AdvData, sizeof(pu8AdvData));
uint8_t pu8ScanResponseData[19] =
{
    2, // 数据长度
    GAP_ADTYPE_FLAGS,
    GAP_ADTYPE_FLAGS_GENERAL | GAP_ADTYPE_FLAGS_BREDR_NOT_SUPPORTED,
```

```

    9, GAP_ADTYPE_LOCAL_NAME_COMPLETE, 'R', 'C', 'U', ' ', 'D', 'E', 'M', 'O'
};
rom_gap_api_set_scan_response_data(0, pu8ScanResponseData,
sizeof(pu8ScanResponseData));
// 启动广播:
rom_gap_api_set_advertising_enable(0, true);
// 关闭广播:
rom_gap_api_set_advertising_enable(0, false);

```

广播时启动白名单过滤

广播要启用白名单过滤，首先要设置白名单，将需要过滤的设备地址添加到白名单，见 `rom_gap_api_add_device_to_white_list`；其次，在设置广播参数时，将参数 `enumAdvertisingFilterPolicy` 根据过滤扫描或连接请求的需要，设置为下面的数值的其中一个。例如，如果只想被白名单内的设备连接和扫描到，则设置为第 3 个。

```

GAP_ADV_FILTER_POLICY_CONN_ALL_SCAN_WHITELIST = 0x01,
GAP_ADV_FILTER_POLICY_CONN_WHITELIST_SCAN_ALL= 0x02,
GAP_ADV_FILTER_POLICY_CONN_WHITELIST_SCAN_WHITELIST =0x03,

```

广播时启用隐私功能

启用隐私功能，包括两个方面：

1. 使用可解析的私有地址 RPA 进行广播；
2. 启动私有地址解析，可以响应使用 RPA 的扫描或连接请求。

这些都要求，首先要设置解析列表，使用相关 API 添加对端设备身份地址和 IRK 及本端 IRK 到解析列表，这些是生成本端 RPA 或解析对端 RPA 需要用到的。

对于实现功能 1，使用 RPA 进行广播，需要将广播参数 `enumOwnAddressType` 设置为如下的第 3 或第 4 个值，并根据对端设备的身份地址，设置参数 `enumPeerAddressType` 和 `pu8PeerAddress[6]`，然后启动广播，底层协议栈会使用本端的 IRK 生成 RPA 作为广播地址进行广播。

```

GAP_OWN_ADDRESS_TYPE_PUBLIC = 0x00, GAP_OWN_ADDRESS_TYPE_RANDOM = 0x01, GAP_
OWN_ADDRESS_TYPE_RESOLVABLE_OR_PUBLIC = 0x02, GAP_OWN_ADDRESS_TYPE_RESOLVABLE_
OR_RANDOM = 0x03,

```

对于实现功能 2，可以响应使用 RPA 的扫描或连接请求，需要在广播启动前，调用相关的 API 启动私有地址解析。然后启动广播后，如果收到使用 RPA 的扫描或连接请求，则会先解析 RPA 得到其身份地址，然后再决定是否进行回应。下面是范例代码。

```

// 添加设备到解析列表:
uint8_t peer_addr_type = GAP_INITIATOR_PEER_ADDRESS_TYPE_PUBLIC;
uint8_t peer_addr[6] = {0xcd, 0x0e, 0x5f, 0x6c, 0x4a, 0x04};
uint8_t peer_irk[16] = {0x40, 0x69, 0x48, 0x36, 0xc6, 0x4b, 0xe0, 0xd1, 0x1c,
0x9a, 0xfe, 0x7d, 0x3e, 0xa2, 0xc7, 0xf4};
uint8_t local_irk[16] = {0x5a, 0x45, 0xe7, 0xa4, 0x57, 0x1d, 0x7f, 0x36, 0x61,
0x30, 0x7e, 0xfa, 0xce, 0xfc, 0xe2, 0x58};
rom_gap_api_add_device_to_resolving_list(peer_addr_type, peer_addr, peer_irk,
local_irk);
// 设备广播地址使 RPA:
stGapSetAdvertisingParameters_t stAdvParam =
{
    .u16AdvertisingIntervalMin625us = 200 * 8 / 5,
    .u16AdvertisingIntervalMax625us = 200 * 8 / 5,
    .enumAdvertisingType = GAP_ADVERTISING_TYPE_ADV_IND,
    .enumOwnAddressType = GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_PUBLIC,
    .enumPeerAddressType = peer_addr_type,
    .u8AdvertisingChannelMap = 7,
}

```

```
.enumAdvertisingFilterPolicy = GAP_ADV_FILTER_POLICY_CONN_ALL_SCAN_ALL,  
};  
memcpy(stAdvParam.enumPeerAddress,peer_addr,6);  
rom_gap_api_set_advertising_parameters(0, &stAdvParam);  
// 启用私有地址解析:  
rom_gap_api_set_addr_resolution_enable (1);
```

扫描

启动设备扫描

用户可以根据自己的应用自定义扫描的参数，原则上 window 越大越快扫描到需要的信息，相应的功耗也越大，最大可以等于 interval，此时所有时间都在扫描。具体参数可以参考前文 API 介绍。所以配置好相应的参数，该单片机就可以启动设备进行扫描了。设置扫描的参数并开启扫描，范例代码如下：

```
// 设置扫描参数:  
stGapSetScanParameters_t stParam =  
{  
.enumScanType = GAP_SCAN_TYPE_ACTIVE,  
.u8ScanChannelMap = 7,  
.u16ScanInterval625us = 200 * 8 / 5, // 200ms  
.u16ScanWindow625us = 100 * 8 / 5, // 100ms  
.enumOwnAddressType = GAP_OWN_ADDRESS_TYPE_PUBLIC,  
.enumScanningFilterPolicy = GAP_SCAN_FILTER_POLICY_ACCEPT_ALL,  
};  
rom_gap_api_set_scan_parameters(&stParam);  
// 启动扫描:  
rom_gap_api_set_scan_enable(true, true);  
// 停止扫描:  
rom_gap_api_set_scan_enable(false, true);
```

扫描时启动白名单过滤

扫描要启用白名单过滤，首先要设置白名单，将需要过滤的设备地址添加到白名单；其次，在设置扫描参数时，将设置过滤规则的参数设置为使用白名单过滤，如下。这样再启动扫描，扫描时仅对在白名单内的设备发起扫描请求。

```
stParam.enumScanningFilterPolicy=GAP_SCAN_FILTER_POLICY_ACCEPT_WHITELIST;
```

扫描时启用隐私功能

启用隐私功能，包括两个方面：

1. 发送扫描请求时使用可解析的私有地址 RPA。
2. 启动私有地址解析，当接收到 AdvA 使用 RPA 的广播时，链路层需要先使用解析名单解析这个 RPA，然后再根据白名单过滤规则（白名单要过滤的对象是解析出来的身份地址），决定是否发送扫描请求。

对于这 2 个功能，同样都要求，首先要设置解析列表，使用相关 API 添加对端设备身份地址和 IRK 及本端 IRK 到解析列表，这些是生成本端 RPA 或解析对端 RPA 需要用到的。然后，对于功能 1，启动扫描前需要将自身地址设置为私有地址类型；对于功能 2，启动扫描前开启私有地址解析。下面是范例代码：

```
// 添加设备到解析列表:  
uint8_t peer_addr_type = GAP_INITIATOR_PEER_ADDRESS_TYPE_PUBLIC;  
uint8_t peer_addr[6] = {0xcd, 0x0e, 0x5f, 0x6c, 0x4a, 0x04};  
uint8_t peer_irk[16] = {0x40, 0x69, 0x48, 0x36, 0xc6, 0x4b, 0xe0, 0xd1, 0x1c,  
0x9a, 0xfe, 0x7d, 0x3e, 0xa2, 0xc7, 0xf4};  
uint8_t local_irk[16] = {0x5a, 0x45, 0xe7, 0xa4, 0x57, 0x1d, 0x7f, 0x36, 0x61,
```

```
0x30, 0x7e, 0xfa, 0xce, 0xfc, 0xe2, 0x58};
rom_gap_api_add_device_to_resolving_list (peer_addr_type, peer_addr, peer_irk,
local_irk);
// 设备扫描地址使用 RPA:
stGapSetScanParameters_t stParam =
{
    .enumScanType = GAP_SCAN_TYPE_ACTIVE,
    .u8ScanChannelMap = 7,
    .u16ScanInterval625us = 200 * 8 / 5, // 200ms
    .u16ScanWindow625us = 100 * 8 / 5, // 100ms
    .enumOwnAddressType = GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_PUBLIC,
    .enumScanningFilterPolicy = GAP_SCAN_FILTER_POLICY_ACCEPT_ALL,
};
rom_gap_api_set_scan_parameters(&stParam);
// 启用私有地址解析:
rom_gap_api_set_addr_resolution_enable (1);
```

获取扫描到的隐私数据

在启动扫描后，如果扫描到广播，会通过消息 MSG_BLE_ADV_REPORT_IND 上报广播数据，这个广播消息的格式参考上述说明，下面是对消息的解析范例代码，pu8Buf 是广播数据的缓冲器：

```
case MSG_BLE_ADV_REPORT_IND:
{
    uint8_t i;
    uint8_t u8ReportsNum = pu8Buf[1];
    uint8_t u8NextReportOffset = 2;
    uint8_t u8DataLength;
    uint16_t u16ReportSize;
    for (i = 0; i < u8ReportsNum; i++)
    {
        // ADV_Type(1) + Addr_Type(1) + Addr(6) + DataLen(1) + Data(variable) + RSSI(1)
        u16ReportSize = 1 + 1 + 6 + 1 + u8DataLength + 1;
        // 广播类型
        uint8_t enumAdvType = pu8Buf[u8NextReportOffset];
        // 广播地址类型
        uint8_t u8AddrType = pu8Buf[u8NextReportOffset + 1];
        // 广播地址
        uint8_t* pu8Addr = &pu8Buf[u8NextReportOffset + 2];
        // 广播数据长度
        u8DataLength = pu8Buf[u8NextReportOffset + 8];
        // 广播数据
        uint8_t* pu8Data = &pu8Buf[u8NextReportOffset + 9];
        // 广播信号强度
        int8_t s8Rssi = pu8Buf[u8NextReportOffset + 9 + u8DataLength];
        PRINTF("ADV_REPORT[index:%d]-ADV_type:%u addr_type:%u rssi: %d", i,
            enumAdvType, u8AddrType, s8Rssi);
        print_hex("ADDR: ", pu8Addr, 6);
        print_hex("DATA: ", pu8Data, u8DataLength);
        u8NextReportOffset += u16ReportSize;
    }
    break;
}
```

外设 (Peripheral)

进入连接

当设备启动广播后,被其它设备连接上,从而进入连接状态,这种情况下设备就是 peripheral 从机角色。此时,消息回调函数会有消息 MSG_BLE_CONNECTED_IND 上报。下面是消息回调函数里上报 MSG_BLE_CONNECTED_IND 的范例代码:

```
case MSG_BLE_CONNECTED_IND:
{
    stHciEventParamVendorConnectionComplete_t* pstEvent =
    (stHciEventParamVendorConnectionComplete_t*)pu8Buf;
    // 连接索引和角色
    PRINTF("connHandle:0x%x,role=%d", pstEvent->Connection_Handle, pstEvent->Role);
    // 对端地址
    Printf_hex("Peer_Address", pstEvent->Peer_Address, 6);
    // pstEvent->Peer_Address 是 RPA, 下面是解析后的身份地址
    if (pstEvent->Peer_RPA_Resolved)
    {
        // 对端的身份地址
        Printf_hex("Peer_Address", pstEvent->Peer_Identify_Address, 6);
    }
    // 连接参数
    PRINTF("Interval:%d,Latency:%d Timeout:%d", pstEvent->Conn_Interval,
    pstEvent->Conn_Latency, pstEvent->Supervision_Timeout);
    // 本端地址类型, 及其是否为 RPA
    PRINTF("Own_Address_Type:%d,Be_RPA=%d", pstEvent->Own_Address_Type,
    pstEvent->Own_Address_Be_RPA);
    break;
}
```

更新连接参数

连接建立后,从机设备也可以调用相应的 API 请求连接参数更新,如下是一个范例代码:

```
stGapConnectionUpdate_t stParam =
{
    .ul6ConnIntervalMin1250us = 10,
    .ul6ConnIntervalMax1250us = 20,
    .ul6ConnLatency = 0,
    .ul6SupervisionTimeout10ms = 100,
};
rom_gap_api_connection_parameters_update (ul6ConnHandle, &stParam);
```

从机设备在 LLCAP 层发送的连接参数更新请求,需要主机接受后,才能由主机发起实际的连接参数更新流程,下面的消息就是主机响应的结果, result 为 0 表示接受:

```
case MSG_BLE_LLCAP_CONN_UPDATE_RSP_IND:
{
    st_llcap_conn_update_rsp_event* pstEvt = (st_llcap_conn_update_rsp_event*)
    pu8Buf;
    PRINTF("Peer device accept or reject new connection parameter, result=%d",
    pstEvt->result);
    break;
}
```

如果主机接受了从机的连接参数更新请求,则主机发起实际的连接参数更新流程,更新完成后会上报如下的消息,指示了更新后的连接参数。

```
case MSG_BLE_CONNECTION_UPDATE_COMPLETE_IND:
{
```

```
stHciEventParamLEConnectionUpdateComplete_t* pstEvt =
(stHciEventParamLEConnectionUpdateComplete_t*) pu8Buf;
PRINTF (" [MSG_BLE_CONNECTION_UPDATE_COMPLETE_IND] Conn_Interval = 0x%X,
Conn_Latency = 0x%X, Supervision_Timeout = 0x%X\n", pstEvt->Conn_
Interval, pstEvt->Conn_Latency, pstEvt->Supervision_Timeout);
break;
}
```

断开连接

连接建立后，从机设备也可以调用相应的 API 主动断开连接，如下是一个范例代码：

```
rom_gap_api_disconnect(u16ConnHandle, 0x13);
```

不论是本端或对端主动断开连接，或是其它原因导致的连接断开，都会上报如下的消息，指示了连接已经断开，及断连的原因。

```
case MSG_BLE_DISCONNECTED_IND:
{
stHciEventParamDisconnectionComplete_t* pstEvt =
(stHciEventParamDisconnectionComplete_t*) pu8Buf;
PRINTF (" [MSG_BLE_DISCONNECTED_IND] Handle = 0x%X, reason = 0x%X\n",
pstEvt->Connection_Handle, pstEvt->Reason);
break;
}
```

配对和加密

该单片机支持传统配对和安全配对两种模式，支持 OOB，Just Work，密码输入，数值比较（仅安全配对下有）这些配对方式。配对模式和配对方式是由双方设备的配对参数共同决定的，具体参见附录。

设置配对参数

1. 设置 OOB 传统配对方式

需要先把获取 OOB 数据的回调函数注册好。在配对时，协议栈通过此回调函数查询 OOB 数据，当获取到 OOB 数据，就会设置 OOB 标志，如果交换配对信息发现对方也设置了 OOB 标志，就会选用 OOB 方式进行配对，而忽略设备的 I/O 能力等信息。

// 实现回调函数：

```
int get_oob_data_callback (uint8 address_type, uint8 addr[6], uint8 oob_data[16])
{
if( 地址匹配 )
{
// 根据地址判断，地址符合，设置 OOB 数据，返回 1.
for(i=0; i<16; i++)
{
oob_data[i]=xx; // 设置 OOB 数据
}
return 1;
}
else
{
// 地址不满足，直接返回 0.
return 0;
}
}
// 注册回调函数：
rom_gap_api_register_oob_data_callback(get_oob_data_callback);
```

2. 设置输入密码的配对方式

在不使用 OOB 的情况下 (不去注册 OOB 回调函数), 根据双方的 I/O 能力来决定配对方式, 如果一方的 I/O 能力是 Keyboard Only, 另一方是 Display Only, 则会选用输入密码的配对方式, 由 Display Only 的一方显示密码, Keyboard Only 的一方输入密码。这种方式对传统配对和安全配对模式都一样。

设置 Display Only 一方的参数:

```
// 传统配对模式:
rom_gap_api_update_pair_para(IO_CAPABILITY_DISPLAY_ONLY, SM_AUTHREQ_BONDING |
SM_AUTHREQ_MITM_PROTECTION, 16);
// 安全配对模式:
```

```
rom_gap_api_update_pair_para(IO_CAPABILITY_DISPLAY_ONLY, SM_AUTHREQ_BONDING |
SM_AUTHREQ_MITM_PROTECTION | SM_AUTHREQ_SECURE_CONNECTION, 16);
```

配对过程中, Display Only 的设备会上来显示密码的消息 MSG_BLE_PAIR_USER_PASSKEY_DISPLAY_IND, 携带需要对端输入的 6 位十进制 PIN 码数据:

```
case MSG_BLE_PAIR_USER_PASSKEY_DISPLAY_IND:
{
    st_passkey_display_event* pstEvt = (st_passkey_display_event*) pu8Buf;
    PRINTF ("MSG_BLE_PAIR_USER_PASSKEY_DISPLAY_IND passkey = %d, Handle = 0x%X",
            rom_little_endian_read_32 (pstEvt->passkey, 0), pstEvt->conn_handle);
    break;
}
```

设置 Keyboard Only 一方的参数

```
// 传统配对模式:
rom_gap_api_update_pair_para(IO_CAPABILITY_KEYBOARD_ONLY, SM_AUTHREQ_BONDING |
SM_AUTHREQ_MITM_PROTECTION, 16);
// 安全配对模式
```

```
rom_gap_api_update_pair_para(IO_CAPABILITY_KEYBOARD_ONLY, SM_AUTHREQ_BONDING |
SM_AUTHREQ_MITM_PROTECTION | SM_AUTHREQ_SECURE_CONNECTION, 16);
```

配对过程中, Keyboard Only 的设备会上来显示请求输入密码的消息 MSG_BLE_PAIR_USER_PASSKEYREQ_IND, 如下:

```
case MSG_BLE_PAIR_USER_PASSKEYREQ_IND:
{
    uint16_t u16ConnectionHandle = * ((uint16_t*) pu8Buf);
    PRINTF ("MSG_BLE_PAIR_USER_PASSKEYREQ_IND Handle = 0x%X", u16ConnectionHandle);
    break;
}
```

设备收到此消息后, 需要输入对端设备显示的 6 位十进制 PIN 码数值 u32Passkey, 并通过相关 API 发送给协议栈, 完成配对:

```
rom_gap_api_input_passkey_during_pair(u16ConnHandle, u32Passkey);
```

3. 设置数值比较的配对方式

在安全配对模式下, 如果双方的 I/O 能力都是 Display YesNo, 则采用数值比较的配对方式。配对过程中双方设备都是显示一个数值, 需要用户进行对比确认, 完成配对。

设置配对参数:

```
rom_gap_api_update_pair_para(IO_CAPABILITY_DISPLAY_YES_NO, SM_AUTHREQ_BONDING
| SM_AUTHREQ_MITM_PROTECTION | SM_AUTHREQ_SECURE_CONNECTION, 16)
```

配对过程中, 双方设备会上来显示待比较确认的数值的消息 MSG_BLE_PAIR_USER_PASSKEYREQ_CONF_IND, 携带需要对端比较确认的 6 位十进制数值:

```
case MSG_BLE_PAIR_USER_PASSKEYREQ_CONF_IND:
{
    st_passkey_display_event* pstEvt = (st_passkey_display_event*) pu8Buf;
```

```
PRINTF ("MSG_BLE_PAIR_USER_PASSKEYREQ_CONF_IND Number= %d, Handle = 0x%X",
        rom_little_endian_read_32 (pstEvt->passkey, 0), pstEvt->conn_handle);
break;
}
```

设备将数值显示出来，用户和对端的数值进行比较，看数值是否一致，比较结果并通过相关 API 发送给协议栈，完成配对：

```
rom_gap_api_sm_numeric_comparison_confirm(u16ConnHandle, confirm_sucess);
```

4. 设置 Just Work 配对方式

如果双方的 I/O 能力其中一个是 No Input No Output 的话，会选用 Just Work 的配对方式（对传统配对和安全配对模式都一样）。

设置 Just Work 配对方式的参数：

```
// 传统配对模式：
rom_gap_api_update_pair_para(IO_CAPABILITY_NO_INPUT_NO_OUTPUT, SM_AUTHREQ_
BONDING , 16);
// 安全配对模式：
rom_gap_api_update_pair_para(IO_CAPABILITY_NO_INPUT_NO_OUTPUT, SM_AUTHREQ_
BONDING | SM_AUTHREQ_SECURE_CONNECTION, 16);
```

启动配对

作为从机设备，不能直接发起配对，只能调用相关 API 发送 security request 给主机设备，使得主机设备启动配对：

```
rom_gap_api_send_security_req (u16ConnHandle);
```

配对完成后，会上来配对完成的消息 MSG_BLE_PAIR_COMPLETED_IND，里面携带了绑定信息，需要将这些信息保存起来，下次加密或设置私有地址解析列表会用到：

```
case MSG_BLE_PAIR_COMPLETED_IND:
{
    st_pair_complete_event* pstPairEvt = (st_pair_complete_event*) pu8Buf;
    PRINTF ("connHdl= 0x%X,peer_addr_type =%d", pstPairEvt->conn_handle,
            pstPairEvt->peer_addr_type);
    PRINTF_hex ("peer_address", pstPairEvt->peer_addr,6);
    PRINTF_hex ("peer_IRK", pstPairEvt->peer_irk,16);
    PRINTF_hex ("local_IRK", pstPairEvt->local_irk,16);
    PRINTF_hex ("LTK", pstPairEvt->ltk,16);
    PRINTF_hex ("RAND", pstPairEvt->rand,8);
    PRINTF ("EDIV = 0x%x",pstPairEvt->ediv);
    PRINTF ("Keysize= %d, authenticated=%d",pstPairEvt->key_size, pstPairEvt->
            authenticated);
    break;
}
```

启动加密

如果已经和对端设备配对过，下次重连后，首先需要加载之前保存的绑定信息，如下：

```
stGapBondInfo_t stBondinfo =
{
    .u16Ediv= 0x58E6,
    .pu8Rand = {0x13, 0x02, 0xF1, 0xE0, 0xDF, 0xCE, 0xBD, 0xAC},
    .pu8Ltk = {0xbf, 0x01, 0xfb, 0x9d, 0x4e, 0xf3, 0xbc, 0x36, 0xd8, 0x74, 0xf5,
                0x39, 0x41, 0x38, 0x68, 0x4c},
    .u8KeySize=16,
    .u8Authenticated=1,
    .u8Authorized = 0,
};
rom_gap_api_set_bond_info_for_setup_encryption (connHdl, &stBondinfo);
```

然后，调用相关 API 发送 security request 给主机设备，则主机设备会启动加密：

```
rom_gap_api_send_security_req (u16ConnHandle)
```

启动加密的过程中，会上来请求 LTK 密钥的消息 MSG_LTK_REQ_WHEN_RECONNECT_AFTER_PAIR_IND 此时需要调用相关 API 将配对完成时保存的密钥 pu8LTK 传递给协议栈，完成加密流程，如下：

```
case MSG_LTK_REQ_WHEN_RECONNECT_AFTER_PAIR_IND:
{
    st_ltk_req_event* pstEvt = (st_ltk_req_event*) pu8Buf;
    PRINTF_hex ("RAND", pstEvt->rand,8);
    PRINTF ("EDIV = 0x%x",pstEvt->ediv);
    // 设置 LTK 密钥给协议栈
    rom_gap_api_long_term_key_request_reply(pstEvt->conn_handle, pu8LTK);
    break;
}
```

进入加密状态后，会上来消息 MSG_BLE_ENCRYPTED_CHANGED_IND，其中第 3 个参数指示了加密状态，如下：

```
case MSG_BLE_ENCRYPTED_CHANGED_IND:
{
    stHciEventParamEncryptionChange_t* pstEvt = (stHciEventParamEncryptionChange_t*)
    pu8Buf;
    PRINTF ("Handle = 0x%X,Status = 0x%X Encryption_Enabled = 0x%X\n", pstEvt->
    Connection_Handle, pstEvt->Status, pstEvt->Encryption_Enabled);
    break;
}
```

中央设备 (Central)

建立连接

从发起态进入连接的设备是主机设备，下面是进入发起态创建连接的范例代码：

```
stGapCreateConnection_t stParam =
{
    .u8ScanChannelMap = 7,
    .u16ScanInterval625us = 200 * 8 / 5,
    .u16ScanWindow625us = 100 * 8 / 5,
    .enumInitiatorFilterPolicy = GAP_INITIATOR_FILTER_POLICY_WHITELIST_IS_NOT_USED,
    .enumPeerAddressType = GAP_INITIATOR_PEER_ADDRESS_TYPE_PUBLIC,
    .pu8PeerAddress = {0xcd,0xe,0x5f,0x6c,0x4a,0x04},
    .enumOwnAddressType = GAP_OWN_ADDRESS_TYPE_PUBLIC,
    .u16ConnIntervalMin1250us = 50 * 4 / 5,
    // 50ms
    .u16ConnIntervalMax1250us = 50 * 4 / 5,
    // 50ms
    .u16ConnLatency = 0,
    .u16SupervisionTimeout10ms = 200, // 2s
};
rom_gap_api_create_connection(&stParam);
```

创建连接时要启用白名单过滤，首先要设置白名单，将需要过滤的设备地址添加到白名单，其次，在设置发起态参数时，将过滤规则设置为启动白名单过滤，如下：

```
stParam.enumInitiatorFilterPolicy = GAP_INITIATOR_FILTER_POLICY_WHITELIST_IS_USED;
```

与广播或扫描类似，创建连接时启用隐私功能，也包括两个方面：

1. 本身使用可解析的私有地址 RPA，发送连接请求时使用 RPA。
2. 启动私有地址解析启动私有地址解析，当接收到 AdvA 使用 RPA 的广播时，需要先解析这个 RPA，然后再根据白名单过滤规则（白名单要过滤的对象是解析出来的身份地址），决定是否发送连接请求。

对于这 2 个功能，同样都要求，首先要设置解析列表，使用相关 API 添加对端设备身份地址和 IRK 及本端 RK 到解析列表，这些是生成本端 RPA 或解析对端 RPA 需要用到的。

对于功能 1，启动创建连接前需要将自身地址设置为私有地址类型；

对于功能 2，启动创建连接前开启私有地址解析。

下面是范例代码。添加设备到解析列表：

```
uint8_t peer_addr_type = GAP_INITIATOR_PEER_ADDRESS_TYPE_PUBLIC;
uint8_t peer_addr[6] = {0xcd, 0x0e, 0x5f, 0x6c, 0x4a, 0x04};
uint8_t peer_irk[16] = {0x40, 0x69, 0x48, 0x36, 0xc6, 0x4b, 0xe0, 0xd1, 0x1c,
0x9a, 0xfe, 0x7d, 0x3e, 0xa2, 0xc7, 0xf4};
uint8_t local_irk[16] = {0x5a, 0x45, 0xe7, 0xa4, 0x57, 0x1d, 0x7f, 0x36, 0x61,
0x30, 0x7e, 0xfa, 0xce, 0xfc, 0xe2, 0x58};
rom_gap_api_add_device_to_resolving_list(peer_addr_type, peer_addr, peer_irk,
local_irk);
// 设备扫描地址使用 RPA:
stParam.enumOwnAddressType = GAP_OWN_ADDRESS_TYPE_RESOLVABLE_OR_PUBLIC ;
// 启用私有地址解析:
rom_gap_api_set_addr_resolution_enable (1);
```

等到连接建立成功，消息回调函数会有消息 MSG_BLE_CONNECTED_IND 上报，消息内容同从机模式。

更新连接参数

同从机更新连接参数调用的 API 相同，不同的地方是调用后，主机直接在链路层发起连接参数的更新流程，中间不会再有消息 MSG_BLE_LLCP_CONN_UPDATE_RSP_IND 上来。最后更新完成后同样会上来携带新的连接参数的更新完成的消息 MSG_BLE_CONNECTION_UPDATE_COMPLETE_IND。

断开连接

调用的 API 和上来的消息与从机一样。

启动配对加密

对于主机，设置配对参数及启动配对的流程和从机一致，重连后启动加密流程也基本一致，只是过程中不会上来会请求 LTK 密钥的消息。

11 低功耗蓝牙 GATT

简介

低功耗蓝牙 (Bluetooth® Low Energy) 连接都是建立在 GATT (Generic Attribute Profile) 协议之上，GATT 是一个在蓝牙连接之上的发送和接收很短的数据段的通用规范。本章介绍低功耗蓝牙 GATT 相关 API 及其使用范例，包括服务器端 (Server) 和客户端 (Client) 的实现。

低功耗蓝牙 GATT Server API 接口

API 接口

rom_gatts_api_init

函数原型

```
void rom_gatts_api_init (uint8_t *pu8ProfileBuf, uint16_t u16ProfileBufLen, gatts_serv_info_st  
*pstGattsServInfoBuffer, uint16_t u16GattsServInfoBufferNum);
```

功能描述

Server 端协议栈初始化。

参数

参数	描述
pu8ProfileBuf	指定一块内存区域用来存储将要添加的 Profile 信息，一般每个设备都会有多个 profile，此区域的长度根据要添加的 profile 的情况而定。
u16ProfileBufLen	内存区域 pu8ProfileBuf 的长度。

返回值

无

rom_gatts_add_service_start

函数原型

```
uint16_t rom_gatts_api_add_service_start (bool is_uuid128, uint16_t uuid16, uint8_t* uuid128, gatt_  
serviceCBs_t *pServiceCBs);
```

功能描述

添加一个服务。

参数

参数	描述
is_uuid128	指示服务 UUID 是否 128 位的，如果是的话，不需要配置第二个参数，只需要配置第三个参数；否则只需配置第二个参数。
uuid16	16 位的服务 UUID，服务 UUID 非 128 位的情况下配置此参数。
uuid128	128 位的服务 UUID，服务 UUID 是 128 位的情况下配置此参数。
pServiceCBs	特征读写的回调函数，特征读写时会被调用。

返回值

handle 或 0	成功则返回指示服务的开始的句柄，否则返回 0。
------------	-------------------------

备注

此 API 必须和 rom_gatts_add_service_end 成对使用。先调用此 API 开始添加服务，中间添加服务包含的特征及其描述符，最后调用 rom_gatts_add_service_end 完成服务的添加。

rom_gatts_add_service_end

函数原型

uint16_t rom_gatts_api_add_service_end(void);

功能描述

结束添加服务，标识一个服务添加完成。

参数

无

返回值

handle 或 0	成功则返回指示服务的结束的句柄，否则返回 0。
------------	-------------------------

备注

此 API 必须和 rom_gatts_add_service_start 成对使用。先调用 rom_gatts_add_service_start 开始添加服务，中间添加服务包含的特征及其描述符，最后调用此 API 完成服务的添加。

rom_gatts_api_add_include_service

函数原型

uint16_t rom_gatts_api_add_include_service (uint16_t u16SrvHandle, uint16_t u16SrvHandleEnd , bool is_uuid128, uint16_t u16SrvUuid16, uint8_t *pu8ServUuid128);

功能描述

添加一个包含的服务的声明。

参数

参数	描述
u16SrvHandle	包含服务的开始句柄。
u16SrvHandleEnd	包含服务的结束句柄，即被包含服务的最后一个特征的句柄。
is_uuid128	指示包含服务 UUID 是否 128 位的，如果是的话，不需要配置 u16SrvUuid16，只需要配置 pu8ServUuid128；否则只需配置 pu8ServUuid128。
u16SrvUuid1	16 位的包含服务 UUID，UUID 非 128 位的情况下配置此参数。
pu8ServUuid128	128 位的服务 UUID，UUID 是 128 位的情况下配置此参数。

返回值

handle 或 0	成功则返回指示包含的服务的声明的句柄，否则返回 0。
------------	----------------------------

备注

必须紧跟 rom_gatts_add_service_start 后面调用，用来声明当前服务所包含的服务。

rom_gatts_api_add_char

函数原型

uint16_t rom_gatts_api_add_char(bool is_uuid128, uint16_t uuid16, uint8_t* uuid128, uint16_t properties, uint8_t* data, uint16_t data_len);

功能描述

添加一个特征。

参数

参数	描述
is_uuid128	指示特征 UUID 是否 128 位的，如果是的话，不需要配置第二个参数，只需要配置第三个参数；否则只需配置第二个参数。
u16Uuid16	16 位的特征 UUID，特征 UUID 非 128 位的情况下配置此参数。
pu8Uuid128	128 位的特征 UUID，特征 UUID 是 128 位的情况下配置此参数。
u16Properties	特征的属性，见“特征的属性说明”小节。
pu8Data	特征值，对于内容固定的特征，可以在这里配置；否则对于特征值可能有变化的赋为 NULL，然后在读特征的回调函数中返回当时的特征值。
u16DataLen	特征值长度，如果 pu8Data 为 NULL 则忽略。

返回值

handle 或 0	成功则返回指示特征值的句柄，否则返回 0。
------------	-----------------------

备注

无

rom_gatts_api_add_char_descrip

函数原型

uint16_t rom_gatts_api_add_char_descrip(uint16_t uuid16, uint16_t properties, uint8_t * data, uint16_t data_len);

功能描述

添加一个特征描述符。

参数

参数	描述
U16Uuid16	16 位的特征描述符。
u16Properties	特征描述符的属性，见“特征的属性说明”小节。
pu8Data	特征值，对于内容固定，可以在这里配置；否则对于特征值可能有变化的赋为 NULL，然后在读特征的回调函数中返回当时的特征值。
u16DataLen	特征值描述符长度，如果 pu8Data 为 NULL 则忽略。

返回值

handle 或 0	成功则返回指示特征值描述符的句柄，否则返回 0。
------------	--------------------------

备注

必须在 rom_gatts_add_char 后面调用，用来描述前面的特征。

rom_gatts_api_add_char_descrip_client_config

函数原型

uint16_t rom_gatts_api_add_char_descrip_client_config(void);

功能描述

添加一个 Client Config 特征描述符。

参数

无

返回值

handle 或 0	成功则返回指示 Client Config 特征描述符的句柄，否则返回 0。
------------	--

备注

Client Config 特征描述符经常用到，设置此 API 简化添加此类描述符的流程。必须在 rom_gatts_add_char 后面调用，用来表明这是前面的特征的 Client Config 特征描述符。此描述符用来控制前面的特征是否可以发送通知 (notify) 或指示 (indication)，值为 0x0001 表示可以发送 notify，为 0x0002 表示可以发送 indication，为 0 表示都不可以发。其值必须由 Client 端来配置。

rom_gatts_api_set_next_attribute_handle

函数原型

bool rom_gatts_api_set_next_attribute_handle (uint16_t u16Handle);

功能描述

设置下一个要添加的服务或特征的句柄。

参数

参数	描述
u16Handle	想要设置的句柄值。

返回值

true 或 false	成功则返回 true，否则返回 false。
--------------	------------------------

备注

这个 API 需要在添加服务或特征 (包含特征描述符) 的 API 之前调用，且设置的句柄值要比当前的 (这些添加服务或特征的 API 的返回值) 句柄值大，否则失败。

rom_gatts_api_send_notify

函数原型

uint32_t rom_gatts_api_send_notify(uint16_t conn_handle, uint16_t attribute_handle, uint8_t *value, uint16_t value_len);

功能描述

发送 notify 数据。

参数

参数	描述
u16Connhandle	当前连接的句柄。
u16AttHdl	需要发送 notify 的特征的句柄。

参数	描述
pu8Value	需要发送 notify 的数据的指针。
u16ValLen	需要发送 notify 的数据长度。

返回值

true 或 false	成功则返回 0，否则返回非 0。
--------------	------------------

备注

需要发送 notify 的特征的 Client Config 特征描述符的值须被置为 0x0001 才可以发送，且发送的数据长度不能大于 MTU-3。

rom_gatts_api_send_indicate

函数原型

uint32_t rom_gatts_api_send_indicate(uint16_t conn_handle, uint16_t attribute_handle, uint8_t *value, uint16_t value_len);

功能描述

发送 Indication 数据。

参数

参数	描述
u16Connhandle	当前连接的句柄。
u16AttHdl	需要发送 Indication 的特征的句柄。
pu8Value	需要发送 Indication 的数据的指针。
u16ValLen	需要发送 Indication 的数据长度。

返回值

true 或 false	成功则返回 0，否则返回非 0。
--------------	------------------

备注

需要发送 Indication 的特征的 Client Config 特征描述符的值必须被置为 0x0002，才可以发送。另外，发送的数据长度不能大于 MTU-3。

(*att_get_attribute_length_callback_t)

函数原型

uint16_t (*att_get_attribute_length_callback_t)(uint16_t conn_handle, uint16_t attribute_handle);

功能描述

添加服务时需要通过 p_service_CBs 指定回调函数，包含有 3 个回调函数，此处的回调函数为返回需要读取的特征的长度。

参数

参数	描述
u16Connhandle	当前连接的句柄。
u16AttHdl	需要读取的特征的句柄。

返回值

uint16_t	需要读取的特征的特征值的总长度。
----------	------------------

备注

协议栈在 Client 端读取一个特征值内容不固定的特征时调用此 API 获取特征值最大长度，然后再通过下面的回调函数 att_read_callback_t 获取特征值内容数据。

(*att_read_callback_t)

函数原型

uint16_t (*att_read_callback_t)(uint16_t conn_handle, uint16_t attribute_handle, uint16_t offset, uint8_t *buffer, uint16_t buffer_size);

功能描述

添加服务时需要通过 p_service_CBs 指定回调函数，包含有 3 个回调函数，此处的回调函数为获取要读取的特征的特征值内容。

参数

参数	描述
u16Connhandle	当前连接的句柄。
u16AttHdl	需要读取的特征的句柄。
u16Offset	读取特征值内容的偏移，若为 0 表示从头开始读取。
pu8Buffer	用于存储特征值内容的内存地址，特征值内容需要拷贝到这个地址，从偏移 u16Offset 个字节开始拷贝。
U16BufSize	用于存储特征值内容的内存的最大长度，拷贝时不能超过这个长度。

返回值

uint16_t	返回拷贝的数据长度。
----------	------------

备注

协议栈在 Client 端读取一个特征值内容不固定的特征时调用此 API 获取特征值数据。

(*att_write_callback_t)

函数原型

uint32_t (*att_write_callback_t)(uint16_t conn_handle, uint16_t attribute_handle, uint16_t transaction_mode, uint16_t offset, uint8_t *buffer, uint16_t buffer_size);

功能描述

添加服务时需要通过 p_service_CBs 指定回调函数，包含有 3 个回调函数，此处的回调函数为改写特征值内容。

参数

参数	描述
u16Connhandle	当前连接的句柄。
u16AttHdl	需要改写的特征的句柄。
u16TransMode	改写操作的模式。
u16Offset	改写特征值内容的偏移，若为 0 表示从头开始改写，此参数只有 Prepare write 模式有效。
pu8Buffer	要改写的特征值的数据。
u16BufSize	改写数据的长度。

返回值

uint32_t	成功则返回 0，否则返回非 0。
----------	------------------

备注

协议栈在 Client 端改写一个特征值内容时调用此 API 改写特征值。另外，发送 Indication 后收到对方的 Confirm 时，也会调用此回调函数，通过 u16TransMode 判断。

特征的属性说明

常用的特征属性有如下几个，每个属性由一个 bit 位标识，一个特征可以有多个属性，例如一个可读可写的特征其属性为 ATT_PROPERTY_READ | ATT_PROPERTY_WRITE | ATT_PROPERTY_DYNAMIC。完整特征属性如下表。

属性定义	说明
ATT_PROPERTY_READ(0x02)	读属性，表明此特征可读。
ATT_PROPERTY_WRITE_WITHOUT_RESPONSE(0x04)	写属性，表明此特征可写（非可靠写方式，即写完不需要对方确定的方式）。
ATT_PROPERTY_WRITE(0x08)	可靠写属性，表明此特征可写（可靠写，即写完需等待对方确定的方式）。
ATT_PROPERTY_NOTIFY(0x10)	通知属性，表明此特征可以发送 notify（具有此属性的特征须增加一个 Client Config 特征描述符）。
ATT_PROPERTY_INDICATE(0x20)	指示属性，表明此特征可以发送 indication（具有此属性的特征须增加一个 Client Config 特征描述符）。
ATT_PROPERTY_DYNAMIC (0x100)	特征值动态可变属性，表明此特征值可能会改变，凡是设置了此属性的特征都需要在读写回调函数里做相应的处理，具有可写属性的特征必须同时设置此属性。
ATT_PROPERTY_AUTHENTICATION_REQUIRED(0x400)	认证属性，表明此特征需要配对加密后才能访问，加密前访问会失败。

写特征回调函数中操作模式说明

写特征回调函数中操作模式 (Transaction Mode) 共有 5 个类型，如下表。中间 3 个类型是处理排队写的。排队写一般前面先有几次 Prepare Write（每次写不同的偏移位置），最后再执行写或取消写。

操作模式	说明
ATT_TRANSACTION_MODE_NONE(0x0)	普通写模式，用传进来的数据立即改写特征值。
ATT_TRANSACTION_MODE_ACTIVE(0x1)	准备写 (Prepare Write) 发来的数据，这类数据需要暂存起来。
ATT_TRANSACTION_MODE_EXECUTE(0x2)	执行写，用前面所有 Prepare 写的的数据改写特征值。
ATT_TRANSACTION_MODE_CANCEL(0x3)	取消写，放弃前面暂存的 Prepare 写的的数据。
ATT_TRANSACTION_MODE_CONFIRM_FOR_INDICAT(0x4)	在发送 Indication 数据后，收到了对方的 Confirm。

范例代码：透传 Profile

透传服务概述

透传 Profile 只包含一个服务，自定义的透传服务，服务 UUID 为 0xff0，包括的特征如下：

特征 UUID	特征属性	说明
0xff2	ATT_PROPERTY_WRITE_WITHOUT_RESPONSE ATT_PROPERTY_DYNAMIC	下行传输特征，用于数据下行，对端通过 Write 方式发数据下来。具有不可靠写属性及特征值动态可变属性。
0xff1	ATT_PROPERTY_NOTIFY ATT_PROPERTY_DYNAMIC	上行传输特征，用于数据上行，通过 notify 方式发送数据给对端。具有通知属性及特征值动态可变属性。
0x2902	ATT_PROPERTY_READ ATT_PROPERTY_WRITE ATT_PROPERTY_WRITE_WITHOUT_RESPONSE ATT_PROPERTY_DYNAMIC	上行传输特征描述符，用于控制特征的 notify 是否开启。具有读及可靠写以及不可靠写属性及特征值动态可变属性。
0xff3	ATT_PROPERTY_NOTIFY ATT_PROPERTY_WRITE_WITHOUT_RESPONSE ATT_PROPERTY_DYNAMIC	命令特征，用于传输自定义的命令（例如修改连接参数的命令）。具有通知属性和不可靠写属性及特征值动态可变属性。
0x2902	ATT_PROPERTY_READ ATT_PROPERTY_WRITE ATT_PROPERTY_WRITE_WITHOUT_RESPONSE ATT_PROPERTY_DYNAMIC	0xff3 特征的 Client Configuration 描述符，用于控制特征的 notify 是否开启。具有读及可靠写以及不可靠写属性及特征值动态可变属性。
0xff4	ATT_PROPERTY_READ ATT_PROPERTY_AUTHENTICATION_REQUIRED	认证属性测试特征，用于测试带认证属性的特征。具有读和认证属性，此特征须链路配对加密后才能读。
0xff6	ATT_PROPERTY_READ ATT_PROPERTY_WRITE ATT_PROPERTY_DYNAMIC	测试 Blob read 和 prepare write 的特征。具有读及可靠写及特征值动态可变属性。
0xff7	ATT_PROPERTY_INDICATE ATT_PROPERTY_DYNAMIC	测试 Indication 的特征。具有指示属性及特征值动态可变属性。
0x2902	ATT_PROPERTY_READ ATT_PROPERTY_WRITE ATT_PROPERTY_WRITE_WITHOUT_RESPONSE ATT_PROPERTY_DYNAMIC	0xff7 特征的 Client Configuration 描述符，用于控制特征的 Indication 是否开启。具有读及可靠写以及不可靠写属性及特征值动态可变属性。
0x2901	ATT_PROPERTY_READ ATT_PROPERTY_WRITE ATT_PROPERTY_DYNAMIC	0xff7 特征的用户描述符。具有读和写属性及特征值动态可变属性。

透传服务实现代码

透传服务详细代码见 `trx_service.c` 和 `trx_service.h`，这里列出并说明关键代码。下面是透传服务里用到的 UUID：

#define TX_RX_SERVICE_UUID	0xFFF0
#define RX_CUS_UUID	0xFFF1
#define TX_CUS_UUID	0xFFF2
#define TRX_CMD_UUID	0xFFF3
#define TRX_READ_FOR_ENCPT_UUID	0xFFF4
#define TRX_BLOBREAD_PREPWRITE_TEST_UUID	0xFFF6
#define TRX_INDICATE_UUID	0xFFF7

服务初始化

```
uint32_t trx_init_service (void)
{
    int16_t hdl = 0;
    // 这里调用开始添加服务的 API, 声明服务, 设置服务 UUID 和回调函数
    hdl = rom_gatts_add_service_start (0, TX_RX_SERVICE_UUID, NULL, &txrxCBs);
    if (hdl == 0)
    {
        return RTN_FAIL;
    }
    // 这里记录一下特征的句柄, 便于在回调函数里区分找到此特征
    txrx_server_hdl[RX_NOTIFY] = hdl;
    /* 这里添加特征的 Client Configuration 描述符, 并记录其句柄, 便于在回调函数里区分找到
       此特征描述符 */
    hdl = om_gatts_add_char_descrip_client_config();
    if (hdl == 0)
    {
        return RTN_FAIL;
    }
    txrx_server_hdl[RX_NOTIFY_CONFIG] = hdl;
    /* 这里添加服务包含的第 3 个特征, 用于传输一些自定义的命令及其回应, 此特征带有 notify 属
       性和写属性, 可以进行数据的上行和下行传输, 下行传输命令, 上行传输命令的回应 */
    hdl = rom_gatts_add_char (0, TRX_CMD_UUID, NULL, (ATT_PROPERTY_NOTIFY | ATT_
        PROPERTY_WRITE_WITHOUT_RESPONSE | ATT_PROPERTY_
        DYNAMIC), NULL, 0); if (hdl == 0)
    {
        return RTN_FAIL;
    }
    // 同样需要记录其句柄
    txrx_server_hdl[TXRX_CMD] = hdl;
    // 这里添加第 3 个特征的 Client Configuration 描述符, 并记录其句柄
    hdl = rom_gatts_add_char_descrip_client_config();
    if (hdl == 0)
    {
        return RTN_FAIL;
    }
    txrx_server_hdl[TXRX_CMD_CONFIG] = hdl;
    /* 这里添加服务包含的第 4 个特征, 用于测试特征的认证属性, 其特征值只有一个固定字节, 不用添
       加 ATT_PROPERTY_DYNAMIC 属性, 无须在回调函数里处理。此特征具有读属性, 但是必须在配对
       加密后才可以读, 在连接进入加密前读此特征会返回认证不足 (INSUFFICIENT_AUTHENTICATION)
       的错误, 对端收到这个错误后需要先加密连接再去进行读操作。*/
    hdl = rom_gatts_add_char (0, TRX_READ_FOR_ENCRYPT_UUID, NULL,
        (ATT_PROPERTY_READ | ATT_PROPERTY_AUTHENTICATION_REQUIRED), &val, 1);
    if (hdl == 0)
    {
        return RTN_FAIL;
    }
    txrx_server_hdl[TXRX_READ_FOR_ENCRYPT] = hdl;
    /* 这里添加服务包含的第 5 个特征, 用于测试 Blob read 和 Prepare Write 操作, 具有读和
       写及 ATT_PROPERTY_DYNAMIC 属性, 数据读写需要在回调函数里处理。*/
    hdl = rom_gatts_add_char (0, TRX_BLOBREAD_PREPWRITE_TEST_UUID,
        NULL, (ATT_PROPERTY_READ | ATT_PROPERTY_WRITE | ATT_PROPERTY_DYNAMIC), NULL, 0);
    if (hdl == 0)
    {
        return RTN_FAIL;
    }
    // 同样需要记录其句柄
    txrx_server_hdl[BLOBREAD_PREPWRITE_IDX] = hdl;
```

```

/* 这里添加服务包含的第 6 个特征，用于测试 Indication，具有指示属性和
ATT_PROPERTY_DYNAMIC 属性，发送的指示数据的确认，在写回调函数里处理。*/
hdl = gatts_add_char (0, TRX_INDICATE_UUID, NULL, (ATT_PROPERTY_INDICATE |
ATT_PROPERTY_DYNAMIC), NULL, 0);

if (hdl == 0)
{
    return RTN_FAIL;
}
txrx_server_hdl[TRX_INDICATE_IDX] = hdl;
/* 这里添加第 6 个特征的用户自定义描述符，并记录其句柄，此描述符可读可写，需要在回调函
数里处理。*/
hdl = gatts_add_char_descrip (GATT_CHAR_USER_DESC_UUID,
ATT_PROPERTY_READ | ATT_PROPERTY_WRITE | ATT_PROPERTY_DYNAMIC, NULL, 0);
if (hdl == 0)
{
    return RTN_FAIL;
}
txrx_server_hdl[TRX_INDICATE_USERD_IDX] = hdl;
/* 下面的 API 表示透传服务配置完成，必须调用此 API 结束配置，和前面调用的 gatts_add_
service_start 一起，两个 API 中间添加的特征及描述符都属于当前服务。*/
rom_gatts_add_service_end();
/* 下面是对第 5 个特征的特征值及第 6 个特征的用户自定义描述符的特征值进行初始化，这两者的
特征值保存在全局数组 char_blobread_prewrite 和 desp_blobread_prewrite 里。*/
for (i = 0; i < sizeof (char_blobread_prewrite); i++)
{
    char_blobread_prewrite[i] = i;
    desp_blobread_prewrite[i] = i;
}
return 0;
}

```

透传服务的回调函数

透传服务初始化时，有一个参数 txrxCBs 指定了回调函数，实际指定了 3 个回调函数。

```

gatt_serviceCBs_t txrxCBs =
{
    trx_get_attrLenCB, // get dynamic attr len
    trx_read_attrCB,   // Read callback function
    trx_write_attrCB,  // Write callback function
};

```

其中 `trx_get_attrLenCB` 和 `trx_read_attrCB` 是成对且顺序调用的，即 `trx_read_attrCB` 被调用前，都会先调用 `trx_get_attrLenCB`。

/* 获取特征值长度的回调函数，在对端设备读取相应的特征时被调用，用来返回相应的特征值的长度，其参数 `u16AttHdl` 是要操作的特征的句柄，这些句柄在服务初始化过程中都有记录。只有带有读属性的特征才需要处理此回调函数。*/

```

uint16_t trx_get_attrLenCB (uint16_t u16ConnHdl, uint16_t u16AttHdl)
{
    /*
    第 2 个特征的 Client Configuration 描述符的长度，此类描述符的取值可能有 3 个：
    0x0000: 表示特征的 notify 或 Indication 关闭
    0x0001: 开启 notify
    0x0002: 开启 Indication，所以其长度是 2
    */
    if (txrx_server_hdl[RX_NOTIFY_CONFIG] == u16AttHdl)
    {
        return 2;
    }
}

```

```
// 第 3 个特征的 Client Configuration 描述符
if (txrx_server_hdl[TXRX_CMD_CONFIG] == u16AttHdl)
{
    return 2;
}
// 第 6 个特征的 Client Configuration 描述符
if (txrx_server_hdl[TRX_INDICATE_CONFIG_IDX] == u16AttHdl)
{
    return 2;
}
/* 第 5 个特征的特征值长度, 这个特征值保存在全局数组
char_blobread_prepwrite 里所以要返回此数组的长度 */
if (txrx_server_hdl[BLOBREAD_PREPWRITE_IDX] == u16AttHdl)
{
    return sizeof (char_blobread_prepwrite);
}
/* 第 6 个特征的用户自定义描述符的特征值长度, 这个特征值保存在全局数组
desp_blobread_prepwrite 里, 所以要返回此数组的长度 */
if (txrx_server_hdl[TRX_INDICATE_USERD_IDX] == u16AttHdl)
{
    return sizeof (desp_blobread_prepwrite);
}
return 0;
}
/* 获取特征值的回调函数, 在对端设备读取相应的特征时被调用, 用来获取相应的特征值内容数据。其参
数 attribute_handle 是要操作的特征的句柄, 这些句柄在服务初始化过程中都有记录。只有带有读属
性的特征才需要处理此回调函数。*/
uint16_t trx_read_attrCB (uint16_t u16ConnHdl, uint16_t attribute_handle,
uint16_t offset, uint8* buffer, uint16_t buffer_size)
{
    /* 读取第 2 个特征的 Client Configuration 描述符的内容, 这个描述符的值保存在全局变量
    rx_client_char_cfg 里 */
    if (txrx_server_hdl[RX_NOTIFY_CONFIG] == attribute_handle)
    {
        if (buffer)
        {
            buffer[0] = rx_client_char_cfg;
            buffer[1] = 0;
        }
        return 2;
    }
    /* 读取第 3 个特征的 Client Configuration 描述符的内容,
    这个描述符的值保存在全局变量 txrx_cmd_char_cfg 里 */
    if (txrx_server_hdl[TXRX_CMD_CONFIG] == attribute_handle)
    {
        if (buffer)
        {
            buffer[0] = txrx_cmd_char_cfg;
            buffer[1] = 0;
        }
        return 2;
    }
    /* 读取第 6 个特征的 Client Configuration 描述符的内容,
    这个描述符的值保存在全局变量 txrx_indicate_char_cfg 里 */
    if (txrx_server_hdl[TRX_INDICATE_CONFIG_IDX] == attribute_handle)
    {
        if (buffer)
        {
```

```

        buffer[0] = txrx_indicate_char_cfg;
        buffer[1] = 0;
    }
    return 2;
}
/* 读取第 5 个特征的特征值，对端可能采用 Blob read 的方式来读取，所以可能读取的
   偏移量 offset 不是 0，这个特征值保存在全局数组 char_blobread_prepwrite 里，
   要根据对端要读取的偏移量和长度来填充 buffer 返回数据。*/
if (txrx_server_hdl[BLOBREAD_PREPWRITE_IDX] == attribute_handle) //blob read
{
    int copylen = buffer_size;
    if ((offset + copylen) > sizeof (char_blobread_prepwrite))
    {
        copylen = sizeof (char_blobread_prepwrite) - offset;
    }
    if (buffer)
    {
        memcpy (buffer, char_blobread_prepwrite + offset, copylen);
    }
    return copylen;
}
/* 读取第 6 个特征的用户自定义描述符的特征值，对端可能采用 Blob read 的方式来读取，所以
   可能读取的偏移量 offset 不是 0，这个特征值保存在全局数组 desp_blobread_prepwrite
   里，要根据对端要读取的偏移量和长度来填充 buffer 返回数据。*/
if (txrx_server_hdl[TRX_INDICATE_USERD_IDX] == attribute_handle) //blob read
{
    int copylen = buffer_size;
    if ((offset + copylen) > sizeof (desp_blobread_prepwrite))
    {
        copylen = sizeof (desp_blobread_prepwrite) - offset;
    }
    if (buffer)
    {
        memcpy (buffer, desp_blobread_prepwrite + offset, copylen);
    }
    return copylen;
}
return 0;
}
/*
改写特征值的回调函数，在对端设备写相应的特征时被调用，用来传输要写的相应的特征值内容数据，
另外，具有指示属性的特征发送完指示 (Indication) 数据后，协议栈收到对方回应的确认包 (confirm)
也会调用此函数，告诉用户已经收到确认。
其参数 attribute_handle 是要操作的特征的句柄，这些句柄在服务初始化过程中都有记录。只有带
有读属性或 Indication 属性的特征才需要处理此回调函数。
*/
uint32_t trx_write_attrCB (uint16_t ul6ConnHdl, uint16_t attribute_handle,
uint16_t transaction_mode, uint16_t offset, uint8* buffer, uint16_t buffer_size)
{
    if (txrx_server_hdl[TX_WRITE] == attribute_handle)
    {
        /* 第 1 个特征收到对方写过来的数据，数据存放在 buffer 里，其长度为
           buffer_size，txrx_server_receive 是处理接收到的数据的函数 */
        txrx_server_receive (ul6ConnHdl, buffer, buffer_size);
    }
    else if (txrx_server_hdl[TXRX_CMD] == attribute_handle)
    {
        // 第 3 个特征收到对方写过来的数据，txrx_cmd_receive 是处理此特征接收到的数据的函数
    }
}

```

```
txrx_cmd_receive (ul6ConnHdl, buffer, buffer_size);
}
else if (txrx_server_hdl[RX_NOTIFY_CONFIG] == attribute_handle)
{
    /* 第 2 个特征的 Client config 描述符收到对方写过来的数据，对端要通过这个方式开启或
    关闭第 2 特征的 notify*/
    rxClientCharCfg = buffer[0];
}
else if (txrx_server_hdl[TXRX_CMD_CONFIG] == attribute_handle)
{
    /* 第 3 个特征的 Client config 描述符收到对方写过来的数据，对端要通过这个方式开启或
    关闭第 3 特征的 notify*/
    txrx_cmd_char_cfg = buffer[0];
}
else if (txrx_server_hdl[TRX_INDICATE_CONFIG_IDX] == attribute_handle)
{
    /* 第 6 个特征的 Client config 描述符收到对方写过来的数据，对端要通过这个方式开启或
    关闭第 6 特征的 Indication*/
    txrx_indicate_char_cfg = buffer[0];
}
else if (txrx_server_hdl[BLOBREAD_PREPWRITE_IDX] == attribute_handle)
{
    if (ATT_TRANSACTION_MODE_NONE == transaction_mode) //normal write
    {
        /* 第 5 个特征收到对方写过来的数据，这是普通的写方式，写的偏移量 offset 为 0. 收到后
        需要立即改写特征值 */
        int copylen = buffer_size;
        if (buffer_size > sizeof (char_blobread_prepwrite))
        {
            copylen = sizeof (char_blobread_prepwrite);
        }
        memcpy (char_blobread_prepwrite, buffer, copylen);
    }
    else if (ATT_TRANSACTION_MODE_ACTIVE == transaction_mode) //prepare write
    {
        /* 第 5 个特征收到对方写过来的数据，这是 Prepare Write 的写方式，一般对方会进行
        多次 Prepare write，每次写特征值不同位域的内容，通过偏移量和长度确定要写的位域的
        位置和长度，收到这类数据后需要先保存起来，此时不能立即去改写相应的特征值，需要对方
        发来 execute write 时再去实际改写特征值内容。*/
        if (NULL == pBuffer_for_prepwrite)
        {
            pBuffer_for_prepwrite = mem_for_prepwrite; offset_for_prepwrite = 0;
        }
        if (NULL != pBuffer_for_prepwrite)
        {
            int copylen = buffer_size;
            if (offset >= BLE_MEM_POOL_SIZE)
            {
                copylen = 0;
            }
            else if ((offset + copylen) > BLE_MEM_POOL_SIZE)
            {
                copylen = BLE_MEM_POOL_SIZE - offset;
            }
            if (copylen && buffer)
            {
                memcpy (pBuffer_for_prepwrite + offset, buffer, copylen);
                offset_for_prepwrite += copylen;
            }
        }
    }
}
```

```

    }
}
else if (ATT_TRANSACTION_MODE_EXECUTE == transaction_mode) //execute write
{
    /* 第 5 个特征收到对方 execute write 的命令，收到此命令，用之前缓存的对方通过
    Prepare Write 的数据，改写特征值。*/
    if (pbuffer_for_prepwrite && (offset_for_prepwrite > 0))
    {
        int copylen = offset_for_prepwrite;
        if (offset_for_prepwrite > sizeof (char_blobread_prepwrite))
        {
            copylen = sizeof (char_blobread_prepwrite);
        }
        memcpy (char_blobread_prepwrite, pbuffer_for_prepwrite, copylen);
    }
    pbuffer_for_prepwrite = NULL;
    offset_for_prepwrite = 0;
}
else if (ATT_TRANSACTION_MODE_CANCEL == transaction_mode) //cancel write
{
    /* 第 5 个特征收到对方 Cancel write 的命令，收到此命令，把之前缓存的对方通过
    Prepare Write 的数据丢弃掉，不做其它操作。*/
    pbuffer_for_prepwrite = NULL;
    offset_for_prepwrite = 0;
}
}
else if ((txrx_server_hdl[TRX_INDICATE_IDX] == attribute_handle) &&
(ATT_TRANSACTION_MODE_CONFIRM_FOR_INDICAT == transaction_mode))
{
    /* 第 6 个特征发送指示数据后，收到了对方回应的确认，收到确认说明对方已经处理了发送指
    示数据，下面这些代码只是一个 demo，用于完成连续多次向对方发送指示数据。*/
    if (g_data_indicate_ent > 0)
    {
        txrx_test_indicate_data (g_data_indicate_ent);
        g_data_indicate_ent--;
    }
}
else if (txrx_server_hdl[TRX_INDICATE_USERD_IDX] == attribute_handle)
{
    if (ATT_TRANSACTION_MODE_NONE == transaction_mode) //normal write
    {
        /* 第 6 个特征的用户自定义描述符收到对方写过来的数据，这是普通的写方式，
        写的偏移量 offset 为 0，收到后需要立即改写描述符的值 */
        int copylen = buffer_size;
        if (buffer_size > sizeof (desp_blobread_prepwrite))
        {
            copylen = sizeof (desp_blobread_prepwrite);
        }
        memcpy (desp_blobread_prepwrite, buffer, copylen);
    }
    else if (ATT_TRANSACTION_MODE_ACTIVE == transaction_mode) //prepare write
    {
        /* 第 6 个特征的用户自定义描述符收到对方写过来的数据，这是 Prepare Write 的写方式，
        一般对方会进行多次 Prepare write，每次写描述符不同位域的内容，通过偏移量和长度
        确定要写的位域的位置和长度，收到这类数据后需要先保存起来，此时不能立即去改写相应
        的描述符，需要对方发来 execute write 时再去实际改写描述符内容，这里是暂存
        prepare write 的数据的范例代码。*/
    }
}

```

```
if (NULL == pBuffer_for_prepwrite)
{
    pBuffer_for_prepwrite = mem_for_prepwrite;
    offset_for_prepwrite = 0;
}
if (NULL != pBuffer_for_prepwrite)
{
    int copylen = buffer_size;
    if (offset >= BLE_MEM_POOL_SIZE)
    {
        copylen = 0;
    }
    else if ((offset + copylen) > BLE_MEM_POOL_SIZE)
    {
        copylen = BLE_MEM_POOL_SIZE - offset;
    }
    if (copylen && buffer)
    {
        memcpy (pBuffer_for_prepwrite + offset, buffer, copylen);
        offset_for_prepwrite += copylen;
    }
}
}
else if (ATT_TRANSACTION_MODE_EXECUTE == transaction_mode) //execute write
{
    /* 第 6 个特征的用户自定义描述符收到对方 execute write 的命令，收到此命令，用之前缓存的对方通过 Prepare Write 的数据，改写描述符，这里是范例代码，因为描述符保存在 desp_blobread_prepwrite 里，所以要用之前暂存的 prepare write 的数据来更新 desp_blobread_prepwrite 的内容 */
    if (pBuffer_for_prepwrite && (offset_for_prepwrite > 0))
    {
        int copylen = offset_for_prepwrite;
        if (offset_for_prepwrite > sizeof (desp_blobread_prepwrite))
        {
            copylen = sizeof (desp_blobread_prepwrite);
        }
        memcpy (desp_blobread_prepwrite, pBuffer_for_prepwrite, copylen);
    }
    pBuffer_for_prepwrite = NULL; offset_for_prepwrite = 0;
}
else if (ATT_TRANSACTION_MODE_CANCEL == transaction_mode) //cancel write
{
    /* 第 6 个特征的用户自定义描述符收到对方 cancel write 的命令，收到此命令，把之前缓存的对方通过 Prepare Write 的数据丢弃掉，不改写描述符。*/
    if (pBuffer_for_prepwrite)
    {
        pBuffer_for_prepwrite = NULL;
    }
    offset_for_prepwrite = 0;
}
}
return 0;
}
```

发送通知

```
uint8_t txrx_server_transfer (uint16_t ul6ConnHdl, uint8_t* pu8Data, uint16_t
length)
{
    // 第 2 个特征发送通知, 如果其 Client Config 特征描述符的值为 0 则不能发送通知
    // 注意, 每次发送通知数据的长度不能超过 (MTU-3)
    if (rx_client_char_cfg)
    {
        return gatts_send_notify (ul6Connhandle, txrx_server_hdl[RX_NOTIFY],
                                pu8Data, length);
    }
    return 0xFE;// notify disable
}
uint8_t txrx_test_cmd_transfer (uint16_t ul6ConnHdl, uint8_t* pu8Data, uint16_t
length)
{
    // 第 3 个特征发送通知, 如果其 Client Config 特征描述符的值为 0 则不能发送通知
    if (txrx_cmd_char_cfg)
    {
        return gatts_send_notify (ul6ConnHdl,txrx_server_hdl[TXRX_CMD], pu8Data,
                                length);
    }
    return 0xFE;// notify disable
}
```

发送指示

```
uint8_t txrx_test_indicate_data (uint16_t ul6ConnHdl, uint8_t* pu8Data, uint16_t
length)
{
    // 第 6 个特征发送指示, 如果其 Client Config 特征描述符的值为 0 则不能发送指示
    // 这里是一个例子, 每次发送 20 字节数据
    if (txrx_indicate_char_cfg)
    {
        return rom_gatts_send_indicate (ul6ConnHdl, txrx_server_hdl[TRX_INDICATE_
                                IDX], pu8Data, length);
    }
    return 0xFE;// indicate disable
}
```

低功耗蓝牙 GATT Client API 接口

API 接口

rom_gatt_client_api_init

函数原型

```
void rom_gatt_client_api_init (gatt_client_call_back callback);
```

功能描述

GATT Client 端协议栈初始化。

参数

参数	描述
callback	指定一个回调函数用来接收服务或特征的查询结果, 及其它操作的结果。

返回值

无

rom_gatt_client_api_discover_primary_services

函数原型

uint8_t rom_gatt_client_api_discover_primary_services (uint16_t u16ConnHandle);

功能描述

查询对端设备所有的主要服务。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。

返回值

status	调用成功返回 0，否则返回非 0。调用成功不代表查询成功。
--------	-------------------------------

备注

查询的结果会在协议栈初始化时注册的回调函数里返回。	
---------------------------	--

rom_gatt_client_api_discover_primary_services_by_uuid16

函数原型

uint8_t rom_gatt_client_api_discover_primary_services_by_uuid16 (uint16_t u16ConnHandle, uint16_t u16Uuid16);

功能描述

查询特定 16 位 UUID 的主要服务。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16Uuid16	要查询的服务的 UUID。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

查询单个服务的 API，查询的结果会在协议栈初始化时注册的回调函数里返回。	
---------------------------------------	--

rom_gatt_client_api_discover_primary_services_by_uuid128

函数原型

uint8_t rom_gatt_client_api_discover_primary_services_by_uuid128 (uint16_t u16ConnHandle, uint8_t* pu8Uuid128);

功能描述

查询特定 128 位 UUID 的主要服务。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
pu8Uuid128	要查询的服务的 UUID。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

查询单个服务的 API，查询的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_discover_characteristics_for_handle_range_by_uuid16

函数原型

uint8_t rom_gatt_client_api_discover_characteristics_for_handle_range_by_uuid16 (uint16_t u16ConnHandle, uint16_t u16StartHandle, uint16_t u16EndHandle, uint16_t u16Uuid16);

功能描述

在指定的句柄范围内查询具有特定 16 位 UUID 的特征。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16StartHandle	起始句柄。
u16EndHandle	结束句柄。
u16Uuid16	要查询的服务的 UUID。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

在 (u16StartHdl, u16EndHdl) 范围内查询，查询的结果会在协议栈初始化时注册的回调函数里返回。
--

rom_gatt_client_api_discover_characteristics_for_handle_range_by_uuid128

函数原型

uint8_t rom_gatt_client_api_discover_characteristics_for_handle_range_by_uuid128 (uint16_t u16ConnHandle, uint16_t u16StartHandle, uint16_t u16EndHandle, uint8_t* pu8Uuid128);

功能描述

在指定的句柄范围内查询具有特定 128 位 UUID 的特征。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16StartHandle	起始句柄。
u16EndHandle	结束句柄。
pu8Uuid128	指定的 128 位 UUID。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

在 (u16StartHdl, u16EndHdl) 范围内查询，查询的结果会在协议栈初始化时注册的回调函数里返回。
--

rom_gatt_client_api_discover_characteristic_descriptors

函数原型

uint8_t rom_gatt_client_api_discover_characteristic_descriptors (uint16_t u16ConnHandle, gatt_client_characteristic_t* pstCharacteristic);

功能描述

查询指定特征的特征描述符。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
pstCharacteristic	指定的特征。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

查询的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_discover_characteristics_for_service

函数原型

uint8_t rom_gatt_client_api_discover_characteristics_for_service (uint16_t u16ConnHandle, gatt_client_service_t* pstService);

功能描述

查询指定服务包含的所有特征。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
pstService	指定的服务。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

查询的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_find_included_services_for_service

函数原型

uint8_t rom_gatt_client_api_find_included_services_for_service (uint16_t u16ConnHandle, gatt_client_service_t* pstService);

功能描述

查询指定服务的包含服务。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
pstService	指定的服务。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

查询的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_write_client_characteristic_configuration

函数原型

uint8_t rom_gatt_client_api_write_client_characteristic_configuration (uint16_t u16ConnHandle, gatt_client_characteristic_t* pstCharacteristic, uint16_t configuration);

功能描述

Client 端用来配置 Server 端 Characteristic 的 CCCD(client_characteristic_configuration descriptor)。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
pstCharacteristic	指定的特征。
configuration	设定值。 0: 关闭 notify 和 indicate 1: 开启 notify 和关闭 indicate 2: 开启 indicate 和关闭 notify 3: 开启 indicate 和 notify

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

是否发送 notify 还是由 server 决定。

rom_gatt_client_api_read_long_value_of_characteristic_using_value_handle_with_offset

函数原型

```
uint8_t rom_gatt_client_api_read_long_value_of_characteristic_using_value_handle_with_offset  
(uint16_t u16ConnHandle, uint16_t characteristic_value_handle, uint16_t u16Offset);
```

功能描述

读取长特征值。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
characteristic_value_handle	要读取的特征句柄。
u16Offset	特征值的 offset，此值置 0 表示从头开始读取。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

如果要读取的特征值长度大于 MTU-3，则会在协议栈初始化时注册的回调函数里多次返回其内容，直到读完为止。

rom_gatt_client_api_cancel_write

函数原型

```
uint8_t rom_gatt_client_api_cancel_write (uint16_t u16ConnHandle);
```

功能描述

此操作之前，会有采用 Prepare write 的方式准备修改特征的特征值的操作，调用此 API 会把之前暂存的内容丢弃掉。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_execute_write

函数原型

```
uint8_t rom_gatt_client_api_execute_write (uint16_t u16ConnHandle);
```

功能描述

此操作之前，会有采用 Prepare write 的方式准备修改特征的特征值的操作，调用此 API 会使用之前暂存的内容修改特征值。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_prepare_write

函数原型

uint8_t rom_gatt_client_api_prepare_write (uint16_t u16ConnHandle, uint16_t attribute_handle, uint16_t u16Offset, uint16_t u16Len, uint8_t* pu8Data);

功能描述

采用 Prepare write 的方式准备修改特征的特征值，一般在特征值长度比较长的时候采用此方式。这个操作不会立即修改特征值，只是把要写的内容先暂存起来。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
attribute_handle	特征的句柄。
u16Offset	Prepare write 的起始偏移。
u16Len	要写的内容的长度。
pu8Data	要写的内容。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回。返回的数据里包含了对方收到的传送数据。可以将此数据和 prepare write 所传送的数据进行比较，以判断对方是否接收正确。

rom_gatt_client_api_reliable_write_long_value_of_characteristic

函数原型

uint8_t rom_gatt_client_api_reliable_write_long_value_of_characteristic (uint16_t u16ConnHandle, uint16_t u16ValueHandle, uint16_t value_length, uint8_t* pu8Value);

功能描述

用来修改长的特征值。具体实现是多次进行 Prepare write，最后再执行写，相当于组合使用 rom_gatt_client_api_discover_characteristics_for_service 和 rom_gatt_client_api_discover_characteristics_for_handle_range_by_uuid16 所述的 API。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16ValueHandle	特征的句柄。

参数	描述
value_length	要写的内容的长度。
pu8Value	要写的内容。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回。此 API 与 rom_gatt_client_api_write_long_value_of_characteristic 功能基本相同，其区别在于，此 API 会在 Prepare write 过程中检测对端是否正确接收到了数据，如果对方接收有误，则停止传输返回错误。而 rom_gatt_client_api_write_long_value_of_characteristic 不会检测对方是否正确接收了数据。

rom_gatt_client_api_write_long_value_of_characteristic

函数原型

uint8_t rom_gatt_client_api_write_long_value_of_characteristic (uint16_t u16ConnHandle, uint16_t u16ValueHandle, uint16_t value_length, uint8_t* pu8Value);

功能描述

用来修改长的特征值。具体实现是多次进行 Prepare write，最后再执行写，相当于组合使用 rom_gatt_client_api_discover_characteristics_for_service 和 rom_gatt_client_api_discover_characteristics_for_handle_range_by_uuid16 所述的 API。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16ValueHandle	特征的句柄。
value_length	要写的内容的长度。
pu8Value	要写的内容。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_write_value_of_characteristic_without_response

函数原型

uint8_t rom_gatt_client_api_write_value_of_characteristic_without_response (uint16_t u16ConnHandle, uint16_t u16ValueHandle, uint16_t value_length, uint8_t* pu8Value);

功能描述

采用不可靠写的方式修改指定句柄的特征的特征值。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16ValueHandle	特征的句柄。

参数	描述
value_length	要写的内容的长度。
pu8Value	要写的内容。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

此操作在回调函数里不会返回任何结果。

rom_gatt_client_api_write_value_of_characteristic

函数原型

uint8_t rom_gatt_client_api_write_value_of_characteristic (uint16_t u16ConnHandle, uint16_t u16ValueHandle, uint16_t value_length, uint8_t* pu8Data);

功能描述

采用可靠写的方式修改指定句柄的特征的特征值。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16ValueHandle	特征的句柄。
value_length	要写的内容的长度。
pu8Value	要写的内容。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_read_value_of_characteristic_using_value_handle

函数原型

uint8_t rom_gatt_client_api_read_value_of_characteristic_using_value_handle (uint16_t u16ConnHandle, uint16_t u16ValueHandle);

功能描述

读取指定句柄的特征的特征值。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16ValueHandle	特征的句柄。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

读取的内容会在协议栈初始化时注册的回调函数里返回。

rom_gatt_client_api_read_value_of_characteristics_by_uuid16

函数原型

uint8_t rom_gatt_client_api_read_value_of_characteristics_by_uuid16 (uint16_t u16ConnHandle, uint16_t u16StartHandle, uint16_t u16EndHandle, uint16_t u16Uuid16);

功能描述

在 u16StartHandle 和 u16EndHandle 之间查找和读取指定 16 位 UUID 的特征的特征值。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16StartHandle	查询的起始句柄。
u16EndHandle	查询的结束句柄。
u16Uuid16	指定的 UUID。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

读取的内容会在协议栈初始化时注册的回调函数里返回。	
---------------------------	--

rom_gatt_client_api_read_value_of_characteristics_by_uuid128

函数原型

uint8_t rom_gatt_client_api_read_value_of_characteristics_by_uuid128 (uint16_t u16ConnHandle, uint16_t u16StartHandle, uint16_t u16EndHandle, uint8_t* pu8Uuid128);

功能描述

在 u16StartHandle 和 u16EndHandle 之间查找和读取指定 128 位 UUID 的特征的特征值。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16StartHandle	查询的起始句柄。
u16EndHandle	查询的结束句柄。
pu8Uuid128	指定的 UUID。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

读取的内容会在协议栈初始化时注册的回调函数里返回。	
---------------------------	--

rom_gatt_client_api_write_characteristic_descriptor_using_descriptor_handle

函数原型

```
uint8_t rom_gatt_client_api_write_characteristic_descriptor_using_descriptor_handle (uint16_t u16ConnHandle, uint16_t u16DescriptorHandle, uint16_t u16Len, uint8_t* pu8Data);
```

功能描述

改写特征描述符。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16DescriptorHandle	描述符的句柄。
u16Len	改写的内容的长度。
pu8Data	改写的内容。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回。	
---------------------------	--

rom_gatt_client_api_mtu_exchange

函数原型

```
uint8_t rom_gatt_client_api_mtu_exchange (uint16_t conn_handle, uint16_t u16Mtu);
```

功能描述

和对端协商修改 MTU。

参数

参数	描述
u16ConnHandle	连接句柄，用来区分不同的连接。
u16Mtu	本端支持的 MTU 的值。

返回值

status	调用成功返回 0，否则返回非 0。
--------	-------------------

备注

操作的结果会在协议栈初始化时注册的回调函数里返回，实际返回双方都支持的 MTU。	
--	--

回调函数

函数原型

uint32_t (*gatt_client_call_back) (uint16_t u16ConnHandle, uint16_t u16OpCode, uint16_t u16ErrCode, uint8_t* pu8Buf, uint16_t u16Len);

功能描述

用来返回 Client 端 API 的操作结果。

参数

参数	描述
u16OpCode	操作码，用来区分是哪个操作的结果。参见下面操作码定义。
u16ErrCode	低 8 位表示错误码，bit15 是否为 1 表示操作是否结束。
pu8Buf	返回的数据，不同操作数据的含义不同。
u16Len	返回的数据的长度。

返回值

status	忽略，此函数是协议栈内部调用的。
--------	------------------

备注

除了不可靠写 gatt_client_write_value_of_characteristic_without_response，其它调用成功的 Client 端 API，都由此回调函数返回其操作结果。操作码定义及其含义：

```
typedef enum
{
    GATTC_USER_NOTIFICATION,           // 对端设备发送来 notify 数据
    GATTC_USER_INDICATION,             // 对端设备发来 Indication 数据
    GATTC_USER_MTU_EXCHG,              // MTU 修改结果
    GATTC_USER_INCLUDE_SERVICES_QUERY, // 查询包含服务
    GATTC_USER_PRIMARY_SERVICES_QUERY_ALL, // 查询所有主要服务
    GATTC_USER_PRIMARY_SERVICES_QUERY_ONE, // 查询单个服务
    GATTC_USER_CHARACTERISTICS_QUERY_ALL, // 查询服务包含的所有特征
    GATTC_USER_CHARACTERISTICS_QUERY_ONE_BY_UUID, // 查询指定 UUID 的特征
    GATTC_USER_CHARACTERISTICS_DESCRIPTOR_QUERY, // 查询特征描述符
    GATTC_USER_CHARACTERISTICS_WRITE_RESULT, // 采用可靠写的方式修改特征值
    GATTC_USER_CHARACTERISTICS_READ_BY_HANDLE_RESULT, // 读指定句柄的特征的特征值
    GATTC_USER_CHARACTERISTICS_READ_BY_UUID_RESULT, // 读指定 UUID 的特征的特征值
    GATTC_USER_CHARACTERISTICS_BLOB_READ_BY_HANDLE_RESULT, // 读长特征值
    GATTC_USER_PREPARE_WRITE_CHAR_RESULT, // Prepare write 的方式准备写特征值
    GATTC_USER_PREPARE_WRITE_EXECUTE_CHAR_RESULT, // 执行写特征值操作
    GATTC_USER_PREPARE_WRITE_CANCEL_CHAR_RESULT, // 放弃写特征值操作
    GATTC_USER_WRITE_LONG_CHAR_RESULT, // 写长特征值
    GATTC_USER_RELIABLE_WRITE_LONG_CHAR_RESULT, // 可靠方式写长特征值
    GATTC_USER_CHARACTERISTICS_DESCRIPTOR_WRITE_RESULT, // 写特征描述符
} user_client_state;
```

结构体定义

服务信息结构体如下，包含服务的起始和结束句柄，及服务的 UUID。其中，UUID 可能是 16 位或 128 位的，分别保存在 uuid16 和 uuid128 里，如果是 128 位的话，uuid16 为 0，反之，uuid128 全为 0。

```
typedef struct
{
    uint16_t start_group_handle;
    uint16_t end_group_handle;
    uint16_t uuid16;
    uint8_t uuid128[16];
} gatt_client_service_t;
```

特征信息结构体，包含特征的句柄，property 和 UUID 等信息。每个特征定义时必须先有一个特征声明，然后是特征值的定义，后面可能还会跟随特征描述符的定义。

结构体中 start_handle 是指向特征声明的句柄，value_handle 是指向特征值的句柄，而 end_handle 在没有描述符的情况下，就等于 value_handle，有描述符的话，end_handle 就是最后一个描述符的句柄。

```
typedef struct
{
    uint16_t start_handle;
    uint16_t value_handle;
    uint16_t end_handle;
    uint16_t properties;
    uint16_t uuid16;
    uint8_t uuid128[16];
} gatt_client_characteristic_t;
```

特征描述符信息的结构体，如下，包含描述符的句柄和 UUID。

```
typedef struct
{
    uint16_t handle;
    uint16_t uuid16;
    uint8_t uuid128[16];
} gatt_client_characteristic_descriptor_t;
```

包含服务的信息的结构体，如下，其中，include_handle 是当前包含服务声明的句柄，include_serv_start_group_handle 和 include_serv_end_group_handle 是包含的服务的起始和结束句柄，最后两个参数是其 UUID。

```
typedef struct
{
    uint16_t include_handle;
    uint16_t include_serv_start_group_handle;
    uint16_t include_serv_end_group_handle;
    uint16_t uuid16;
    uint8_t uuid128[16];
} gatt_client_include_service_t;
```

错误码

1. 错误码：

#define ATT_ERROR_INVALID_HANDLE	0x01
#define ATT_ERROR_READ_NOT_PERMITTED	0x02
#define ATT_ERROR_WRITE_NOT_PERMITTED	0x03
#define ATT_ERROR_INVALID_PDU	0x04
#define ATT_ERROR_INSUFFICIENT_AUTHENTICATION	0x05
#define ATT_ERROR_REQUEST_NOT_SUPPORTED	0x06
#define ATT_ERROR_INVALID_OFFSET	0x07
#define ATT_ERROR_INSUFFICIENT_AUTHORIZATION	0x08
#define ATT_ERROR_PREPARE_QUEUE_FULL	0x09
#define ATT_ERROR_ATTRIBUTE_NOT_FOUND	0x0A
#define ATT_ERROR_ATTRIBUTE_NOT_LONG	0x0B
#define ATT_ERROR_INSUFFICIENT_ENCRYPTION_KEY_SIZE	0x0C
#define ATT_ERROR_INVALID_ATTRIBUTE_VALUE_LENGTH	0x0D
#define ATT_ERROR_UNLIKELY_ERROR	0x0E
#define ATT_ERROR_INSUFFICIENT_ENCRYPTION	0x0F
#define ATT_ERROR_UNSUPPORTED_GROUP_TYPE	0x10
#define ATT_ERROR_INSUFFICIENT_RESOURCES	0x11
#define ATT_ERROR_HCI_DISCONNECT_RECEIVED	0x1F
#define ATT_ERROR_BONDING_INFORMATION_MISSING	0x70
#define ATT_ERROR_DATA_MISMATCH	0x7E
#define ATT_ERROR_TIMEOUT	0x7F

2. 比较常用的错误码如下：

ATT_ERROR_INVALID_HANDLE：使用的句柄是非法的；

ATT_ERROR_READ_NOT_PERMITTED：对一个不具有读属性的特征进行读操作；

ATT_ERROR_WRITE_NOT_PERMITTED：对一个不具有写属性的特征进行写操作；

ATT_ERROR_INSUFFICIENT_AUTHENTICATION：认证不足，需要加密连接后才能访问；

ATT_ERROR_INVALID_OFFSET：偏移量无效；

ATT_ERROR_ATTRIBUTE_NOT_FOUND：特征没有找到；

ATT_ERROR_PREPARE_QUEUE_FULL：prepare write 的队列已满，不能继续进行 prepare write；

ATT_ERROR_DATA_MISMATCH：数据传输失败，用于长特征值的写操作。

范例代码

初始化并注册回调函数

首先，需要进行协议栈初始化，并注册回调函数。其中 ble_gatt_client_call_back 是实现的回调函数：

```
gatt_client_init (ble_gatt_client_call_back);
```

查询服务

在进入连接后，需要执行如下代码，开启查询所有的主要服务：

```
gatt_client_discover_primary_services (u16ConnHandle);
```

查询结果在回调函数里返回，操作码为 GATTC_USER_PRIMARY_SERVICES_QUERY_ALL。每查询到一个主要服务，都会调用一次回调函数返回服务的信息。由于对端可能包含多个主要服务，所以回调函数可能被多次调用。最后查询结束，也会被调用一次，此时 errcode 的 bit15 为 1，表明查询主要服务完成。

```
uint32 ble_gatt_client_call_back(uint16 connHdl, uint16 opcode, uint16 errcode,
uint8 *buf, uint16 len)
{
    uint16_t isClientOver = (errcode & 0x8000); errcode &= 0xff;
    int ret = 0;
    switch (opcode)
    {
        case GATTC_USER_PRIMARY_SERVICES_QUERY_ALL:
        {
            // 查询未结束，返回查询到的一个服务的信息
            if (is_client_over == 0)
            {
                if (buf && (errcode == 0))
                {
                    // 返回的服务信息保存在 buf 里，它实际是一个服务结构体的数据，
                    // 里面包含服务的起始句柄和结束句柄，以及服务的 UUID
                    gatt_client_service_t *pser = (gatt_client_service_t *)buf;
                    // 这是一个 demo 代码，把服务信息保存起来
                    ret = ble_central_add_a_service((gatt_client_service_t *)buf);
                    if (TX_RX_SERVICE_UUID == pser->uuid16)
                    {
                        // 如果是透传服务，将其信息保存起来，后面透传测试使用
                        memcpy(&g_trx_ser, (void *)pser, sizeof(gatt_client_service_t));
                    }
                }
            }
        }
        else
        {
            // 查询结束
            PRINTF("discover_primary_services over");
        }
        break;
        .....
    }
}
```

如果只是查询单个服务，例如 GAP 服务，其服务 UUID 为 0x1800，可以执行如下代码：

```
gatt_client_discover_primary_services_by_uuid16 (0, 0x1800);
```

查询结果在回调函数里返回，操作码为 GATTC_USER_PRIMARY_SERVICES_QUERY_ONE，如下面的代码是回调函数 ble_gatt_client_call_back 里面的。

```
case GATTC_USER_PRIMARY_SERVICES_QUERY_ONE:
{
    if ((errcode == 0) && buf)
    {
        // 查询成功，返回的服务信息保存在 buf 里，它实际是一个结构体的数据，
        // 里面包含服务的起始句柄和结束句柄，以及服务的 UUID
        gatt_client_service_t *pser = (gatt_client_service_t *)buf;
        gSer_start_handle = pser->start_group_handle;
        gSer_end_handle = pser->end_group_handle;
    }
    else if (ATT_ERROR_ATTRIBUTE_NOT_FOUND == errcode)
    {
        // 查询失败，未找到服务
        PRINTF("not found service");
    }
    break;
}
```

查询包含服务。如果要查询透传服务所包含的服务，可以执行如下代码 (g_trx_ser 是之前保存透传服务信息的结构体):

```
gatt_client_find_included_services_for_service (0, &g_trx_ser);
```

其查询结果在回调函数里返回，操作码为 GATTC_USER_INCLUDE_SERVICES_QUERY，如下：

```
case GATTC_USER_INCLUDE_SERVICES_QUERY:
{
    if (buf && (errcode == 0))
    {
        // 查询到包含服务，buf 里包含了其信息，实际是包含服务的结构体数据。
        gatt_client_include_service_t *pinclude_ser = (gatt_client_include_
                                                    service_t *)buf;
    }
    if (isClientOver)
    {
        // 查询完成
        PRINTF("find include service over");
    }
    break;
}
```

查询特征

查询某个服务包含的所有特征，可以执行如下代码：

```
gatt_client_discover_characteristics_for_service (0, &g_trx_ser);
```

其中 g_trx_ser 是之前保存透传服务的结构体，这样就会去查询透传服务包含的所有特征。查询结果在回调函数里返回，操作码为 GATTC_USER_CHARACTERISTICS_QUERY_ALL，如下。每查找到一个特征，此回调函数都会被调用一次，由于服务可能包含多个特征，所以可能会被多次调用：

```
case GATTC_USER_CHARACTERISTICS_QUERY_ALL:
{
    if (isClientOver == 0)
    {
        if (errcode == 0 && (buf != NULL))
        {
            // 找到一个特征，buf 其实是特征信息结构体数据，包含了特征的信息
            gatt_client_characteristic_t *pchr = (gatt_client_characteristic_t *)
            buf;
            ret = ble_central_add_a_characteristic((gatt_client_characteristic_t *)
            buf);
            if (RX_CUS_UUID == pchr->uuid16)
            {
                // 透传服务里用于上行的特征，保存一下其信息，用于后面透传测试
                memcpy(&g_rx_char, (void *)pchr, sizeof(gatt_client_characteristic_t));
            }
            else if (TX_CUS_UUID == pchr->uuid16)
            {
                // 透传服务里用于下行的特征，保存一下其信息，用于后面透传测试
                memcpy(&g_tx_char, (void *)pchr, sizeof(gatt_client_characteristic_t));
            }
            else if (TRX_CMD_UUID == pchr->uuid16)
            {
                // 透传服务里用于传输命令的特征，保存一下其信息，用于后面透传测试
                memcpy(&g_cmdrx_char, (void *)pchr, sizeof(gatt_client_
                    characteristic_t));
            }
        }
    }
}
```

```

    }
    else if (TRX_BLOBREAD_PREPWRITE_TEST_UUID == pchr->uuid16)
    {
        // 透传服务里用于测试长特征读写的特征，保存一下其信息，用于后面透传测试
        memcpy(&g_blobread_preparewrite_char, (void *)pchr, sizeof(gatt_client_characteristic_t));
    }
    else if (TRX_INDICATE_UUID == pchr->uuid16)
    {
        // 透传服务里用于测试 Indication 的特征，保存一下其信息，用于后面透传测试
        memcpy(&g_indicate_char, (void *)pchr, sizeof(gatt_client_characteristic_t));
    }
}
}
else
{
    // 特征查询结束
}
break;
}
}

```

查询特征描述符

查询透传服务里数据上行的特征包含的所有描述符，可以执行如下代码：

```
gatt_client_discover_characteristics_for_service (0, & g_rx_char);
```

其中，**g_rx_char** 是之前保存的特征的结构体，这样就会去查询此特征包含的描述符。查询结果在回调函数里返回，操作码为 **GATTC_USER_CHARACTERISTICS_DESCRIPTOR_QUERY**，如下。每查找到一个描述符，此回调函数都会被调用一次，由于特征可能包含多个描述符，所以可能会被多次调用：

```

case GATTC_USER_CHARACTERISTICS_DESCRIPTOR_QUERY:
{
    if (isClientOver == 0)
    {
        if (errcode == 0 && (buf != NULL))
        {
            // 返回一个特征描述符信息
            gatt_client_characteristic_descriptor_t *per = (gatt_client_characteristic_descriptor_t *)buf;

            if (GATT_CLIENT_CHAR_CFG_UUID == pchr->uuid16)
            {
                // 特征的 Client Config 描述符，保存一下其句柄，用于后面测试
                g_tx_cc_hdl = pchr->handle;
            }
        }
    }
    else
    {
        // 查询完成
        Printf("found descriptor over");
    }
}
}

```

特征值读取

要读取的特征必须具有读属性 (ATT_PROPERTY_READ)，才能进行读操作。特征值读取有两种方式：

第一种方式：通过指定句柄读取，执行如下代码。这是普通读取方式，如果特征值长度超过了 MTU-3，那么这种读方式只能返回的特征值内容的长度是 MTU-3：

```
gatt_client_read_value_of_characteristic_using_value_handle(0,g_blobread_preparewrite_char.handle);
```

第 2 个参数是特征的句柄，而 g_blobread_preparewrite_char 是保存此特征的结构体。执行的结果在回调函数里返回，操作码为 GATT_USER_CHARACTERISTICS_READ_BY_HANDLE_RESULT，如下。这种读取方式只会有一次带读取到的内容的回调。

```
case GATT_USER_CHARACTERISTICS_READ_BY_HANDLE_RESULT:
{
    if (isClientOver == 0)
    {
        if (buf && (errcode == 0))
        {
            // buf 里携带读取到的内容数据，长度为 len
            PRINTF("read by handle[%d]:", len);
        }
    }
    else
    {
        // 如果读取失败，则回调函数只被调用一次，携带错误码，如果读取成功，则回调函数
        // 只被调用 2 次，第一次携带读到的数据，第二次 (errcode=0) 只是表示读取完成了。
        PRINTF("read by handle over [%x]\n", errcode );
    }
    break;
}
```

第二种方式：通过指定 UUID 读取。这也是普通读取方式，如果特征值长度超过了 MTU-3，那么这种读方式返回的特征值内容的长度也是 MTU-3：

```
gatt_client_read_value_of_characteristics_by_uuid16(0,1,0xffff, g_blobread_preparewrite_char.uuid16);
```

第 4 个参数是特征的 UUID，而 g_blobread_preparewrite_char 是保存此特征的结构体。第 2，3 个参数分别是查询的起始和结束句柄。这种读取方式，会先在第 2，3 个参数确定的句柄范围内查询等于第 4 个参数 UUID 的特征，然后读取其特征值返回。执行的结果在回调函数里返回，操作码为 GATT_USER_CHARACTERISTICS_READ_BY_UUID_RESULT，如下。这种读取方式只会有一次带读取到的内容的回调。

```
case GATT_USER_CHARACTERISTICS_READ_BY_UUID_RESULT:
{
    if (isClientOver == 0)
    {
        if (buf && (errcode == 0))
        {
            // buf 里携带读取到的内容数据，长度为 len
            printf("read by uuid[%d]:", len);
        }
    }
    else
    {
        // 如果读取失败，则回调函数只被调用一次，携带错误码，如果读取成功，则回调函数
        // 只被调用 2 次，第一次携带读到的数据，第二次 (errcode=0) 只是表示读取完成了。
        printf("read by uuid over [%x]\n", errcode);
    }
}
```

```
    }  
    break;  
}
```

长特征值读取

前面一节介绍的读取方式，不能读取到长特征值（特征值长度超过了 MTU-3）的所有内容，如果要读取长特征值的所有内容，须采用长特征值读取方式，还以前面一节使用的特征为例，执行如下代码：

```
gatt_client_read_long_value_of_characteristic_using_value_handle_with_offset  
(0,g_blobread_preparewrite_char.uuid16, 0);
```

第 2 个参数是特征的句柄，而 `g_blobread_preparewrite_char` 是保存此特征的结构体。第 3 个参数是 0，表示从头开始读取。执行的结果在回调函数里返回，操作码为 `GATTC_USER_CHARACTERISTICS_BLOB_READ_BY_HANDLE_RESULT`，如下。这种读方式会通过多次回调返回特征值内容，直到特征值读完：

```
case GATTC_USER_CHARACTERISTICS_BLOB_READ_BY_HANDLE_RESULT:  
{  
    if (isClientOver == 0)  
    {  
        if (buf && (errcode == 0))  
        {  
            // buf 里携带读取到的内容数据，对于长特征值，将在这里依次返回特征值内容，  
            // 比如第一次是前 20 字节，第二次是第 20 到 40 字节，……  
            PRINTF("blob read[%d]:", len);  
        }  
    }  
    else  
    {  
        // 读取成功完成或者发生读取错误而结束。  
        PRINTF("blob read over [%x]\n", errcode);  
    }  
    break;  
}
```

特征值写操作

写操作有两种方式，一种是可靠的写方式，一种是不可靠的写方式。

1. 可靠的写方式：特征必须具有 `ATT_PROPERTY_WRITE` 属性，才能使用此种方式。例如，要改写透传服务里测试长特征值读写的特征的特征值，可以执行如下代码：

```
gatt_client_write_value_of_characteristic(0,g_blobread_preparewrite_char.  
handle, len, pdata);
```

其中第 2 个参数是特征的句柄，第 3、4 参数指定了要写的数据及长度。执行的结果在回调函数里返回，操作码为 `GATTC_USER_CHARACTERISTICS_WRITE_RESULT`，如下。这个操作只有一次回调，返回是否写成功了。

```
case GATTC_USER_CHARACTERISTICS_WRITE_RESULT:  
{  
    // errcode 表示是否写操作成功。值为 0 代表成功。  
    PRINTF("write over [%x]\n", errcode); // write over;  
    break;  
}
```

2. 不可靠的写方式：特征必须具有 `ATT_PROPERTY_WRITE_WITHOUT_RESPONSE` 属性，才能使用此种方式。例如，要改写透传服务里数据下行的特征的特征值，可以执行如下代码：

```
gatt_client_write_value_of_characteristic_without_response (0, g_tx_char.  
handle, len, pdata);
```

其中第2个参数是特征的句柄，`g_tx_char` 是之前保存数据下行特征的信息的结构体。第3、4参数指定了要写的数据及长度。这个操作没有执行的结果返回，不能知道是否写成功，所以它是不可靠的写操作。

另外，`g_tx_char` 这个特征不具有 `ATT_PROPERTY_WRITE` 属性，所以不能使用可靠的写方式进行操作，同样，不具有 `ATT_PROPERTY_WRITE_WITHOUT_RESPONSE` 属性的特征 `g_blobread_preparewrite_char` 也不能采用不可靠写的方式操作。

长特征值写操作

写长特征值的方法共分三种方法。

1. 采用连续多次 `prepare write` 再 `execute write` 的方式，还是以透传服务里用于测试长特征读写的特征为例，当前 MTU 是 23，如果要把 50 字节的数组内容改写去，则需要执行如下代码：

先准备写前 20 字节，由于 MTU 是 23，所以每次最多写 20 字节。第2个参数是特征的句柄，第3个参数是 `offset`，指示当前数据写的偏移量，0 是从头开始。第3、4个参数是写的数据及长度。

```
gatt_client_prepare_write (0, g_blobread_preparewrite_char.handle, 0, 20,  
pdata);
```

第二次再准备写 20 字节，`offset` 参数变为 20，要偏移 20 字节写入。

```
gatt_client_prepare_write (0, g_blobread_preparewrite_char.handle, 20, 20,  
pdata+20);
```

第三次再准备写 10 字节，`offset` 参数变为 40，要偏移 40 字节写入，因为前面已经写了 40 字节了。

```
gatt_client_prepare_write (0, g_blobread_preparewrite_char.handle, 40, 10,  
pdata+40);
```

最后再执行写操作，把前面准备的数据实际改写到特征值里。因为前面三次 `prepare write` 操作都没有实际把数据写到特征值里，对端设备收到 `prepare write` 的数据都会暂存起来，等到 `execute_write` 操作时才能写入。

```
gatt_client_execute_write (0);
```

另外，如果最后不想要改写特征值的话，可以执行如下代码，放弃之前准备写的数据：

```
gatt_client_cancel_write (0);
```

这三个操作在回调函数里都会返回相应的结果，如下：

```
case GATTC_USER_PREPARE_WRITE_CHAR_RESULT:  
{  
    // 准备写的操作的返回  
    if (buf && (errcode == 0))  
    {  
        // 操作成功，返回对端收到的数据，此处可以将已发的数据和这个数据做个对比，  
        // 以确定数据是否传输成功。  
        PRINTF("prepare wr [len%d]:", len);  
    }  
    if (isClientOver)  
    {  
        // 操作成功完成，或写操作失败返回错误码  
        PRINTF("prepare wr over [%x]\n", errcode);  
    }  
    break;
```

```

    }
    case GATTC_USER_PREPARE_WRITE_EXECUTE_CHAR_RESULT:
    {
        // 执行写的操作的返回
        if (isClientOver)
        {
            // 操作成功完成, 或操作失败返回错误码
            PRINTF("execute write over [%x]\n", errcode);
        }
        break;
    }
    case GATTC_USER_PREPARE_WRITE_CANCEL_CHAR_RESULT:
    {
        // 执行取消写的操作的返回
        if (isClientOver)
        {
            // 操作成功完成, 或操作失败返回错误码
            PRINTF("prepare write cancel over [%x]\n", errcode);
        }
        break;
    }
}

```

2. 第二种方式:

```
gatt_client_write_long_value_of_characteristic(0,g_blobread_preparewrite_
char.handle, 0,50, pdata);
```

3. 第三种方式:

```
gatt_client_reliable_write_long_value_of_characteristic(0,g_blobread_
preparewrite_char.handle, 0,50, pdata);
```

这 2 个操作, 协议栈内部其实也是通过连续多次 prepare write 再 execute write 的方式实现的, 所不同的是, 第三种方式, 协议栈在每次 prepare write 的回应消息里, 会去判断数据是否传输成功, 如果数据不对, 则结束传输, 返回错误; 而第二种方式不会检查数据是否传输成功。他们的执行结果会在回调函数里返回。

```

case GATTC_USER_WRITE_LONG_CHAR_RESULT:
{
    // 第二种方式的返回
    if (isClientOver)
    {
        // 操作成功, 或操作失败返回错误码
        PRINTF("write long char over [%x]\n", errcode);
    }
    break;
}
case GATTC_USER_RELIABLE_WRITE_LONG_CHAR_RESULT:
{
    // 第三种方式的返回
    if (ATT_ERROR_DATA_MISMATCH == errcode)
    {
        // prepare write 过程中, 如果检查到数据传输错误, 会返回这个错误码
        PRINTF("reliable write long char error cause _DATA_MISMATCH \n");
    }
    if (isClientOver)
    {
        // 操作成功, 或操作失败返回错误码
        PRINTF("reliable write long char over [%x]\n", errcode);
    }
    break;
}
}

```

另外注意，如果特征不具有 ATT_PROPERTY_WRITE 属性，是不能采用这三种方式进行写操作的。

修改特征描述符

透传测试数据上行，需要先修改上行特征的 Client Config 描述符，使其能够发送通知，需要执行如下代码：

```
// 此特征具有通知属性，描述符配置为 0x0001 可以使能通知，对于具有指示属性的特征，  
// 描述符的值应该配置为 0x0002，  
unsigned char data[2] = {1, 0};  
gatt_client_write_characteristic_descriptor_using_descriptor_handle(0, g_tx_cc_hdl, 2, data);
```

g_tx_cc_hdl 是前面保存的上行的特征的 Client Config 描述符的句柄，其操作结果在回调函数里返回，这个操作只会有一次回调返回，如下：

```
case GATTC_USER_CHARACTERISTICS_DESCRIP_WRITE_RESULT:  
{  
    // errcode 表示是否写操作成功。值为 0 代表成功。  
    PRINTF("write descriptor over [%x]\n", errcode); // write over;  
    break;  
}
```

修改 MTU

如果要修改 MTU，可以执行如下代码：

```
gatt_client_MTU_exchange(0, _mtu);
```

参数 _mtu 是本端支持的 MTU，这个操作会协商出一个双方都支持的 MTU，然后在回调函数里返回：

```
case GATTC_USER_MTU_EXCHG:  
{  
    if (buf && (errcode == 0))  
    {  
        // buf 的前两个字节是协商后的 MTU，如果前 2 个字节是 08 和 01 则 MTU=0x0108  
        uint16_t mtu = little_endian_read_16(buf, 0);  
        PRINTF("exchang mtu [len%d--used mtu=%d]:", len, mtu);  
    }  
    if (isClientOver)  
    {  
        PRINTF("exchang mtu over [%x]\n", errcode);  
    }  
    break;  
}
```

通知 (Notification) 和指示 (Indication) 数据

回调函数会返回对方发过来的 notification 和 Indication 数据，如下：

```
case GATTC_USER_NOTIFICATION:  
{  
    /* 收到 notification 数据，buf 里第 1、2 字节是发数据的特征的句柄，后面的数据是  
    notification 数据，这块数据长度是 len-2 */  
    uint16_t value_handle = little_endian_read_16(buf, 1);  
    if (g_rx_char.handle == value_handle)  
    {  
        // 这是透传服务里数据上行特征发来的数据  
        PRINTF("notify data len= [%d]\n", len - 2);  
    }  
}
```

```
}
case GATTC_USER_INDICATION:
{
    /* 收到 Indication 数据, buf 里第 1, 2 字节是发数据的特征的句柄, 后面的数据是
       Indication 数据, 这块数据长度是 len-2 */
    uint16_t value_handle = little_endian_read_16(buf, 1);
    if (g_indicate_char.handle == value_handle)
    {
        // 这是透传服务里用于测试 Indication 的特征发来的数据
        PRINTF("recv indication, data len= [%d]\n", len - 2);
    }
}
```

实现 Client 端透传 Profile 的总结

通过前面范例的 Client 端的操作, 可以很容易的实现透传 Profile:

1. 通过查询主要服务找到透传服务;
2. 查询透传服务包含的所有特征及其描述符;
3. 对于数据上行的特征的 Client Config 描述符进行修改, 使其能发送通知数据;
4. 进行数据透传测试, 通过写数据下行的特征向对方发数据;
5. 进行数据透传测试, 对方通过发通知的方式发送上行数据, 在回调函数里接收到对方的通知数据;
6. 根据需要进行其它操作测试。

附录

常用 UUID 参考

GATT 服务

```
#define GAP_SERVICE_UUID          0x1800 // 通用访问配置文件
#define GATT_SERVICE_UUID        0x1801 // 通用属性配置文件
```

GATT 声明

```
#define GATT_PRIMARY_SERVICE_UUID 0x2800 // 主要服务
#define GATT_SECONDARY_SERVICE_UUID 0x2801 // 次要服务
#define GATT_INCLUDE_SERVICE_UUID 0x2802 // 包含
#define GATT_CHARACTERISTICS_UUID 0x2803 // 特征
```

GATT 描述符

```
#define GATT_CHAR_EXT_PROPS_UUID 0x2900 // 特征扩展属性
#define GATT_CHAR_USER_DESC_UUID 0x2901 // 特征用户描述
#define GATT_CLIENT_CHAR_CFG_UUID 0x2902 // 客户端特征配置
#define GATT_SERV_CHAR_CFG_UUID 0x2903 // 服务器特征配置
#define GATT_CHAR_FORMAT_UUID 0x2904 // 特征表示格式
#define GATT_CHAR_AGG_FORMAT_UUID 0x2905 // 特征聚合格式
#define GATT_VALID_RANGE_UUID 0x2906 // 有效范围
#define GATT_EXT_REPORT_REF_UUID 0x2907 // 外部报告参考描述符
#define GATT_REPORT_REF_UUID 0x2908 // 报告参考描述符
```

GATT 特征

```
#define DEVICE_NAME_UUID          0x2A00 // 设备名称
#define APPEARANCE_UUID          0x2A01 // 外观
#define PERI_PRIVACY_FLAG_UUID    0x2A02 // 外设隐私标志
#define RECONNECT_ADDR_UUID      0x2A03 // 重新连接地址
```

```
#define PERI_CONN_PARAM_UUID      0x2A04 // 外设首选连接参数
#define SERVICE_CHANGED_UUID      0x2A05 // 服务变更
```

GATT 服务 UUID

```
#define IMMEDIATE_ALERT_SERV_UUID 0x1802 // 立即警报
#define LINK_LOSS_SERV_UUID       0x1803 // 链路丢失
#define TX_PWR_LEVEL_SERV_UUID    0x1804 // TX 功率
#define CURRENT_TIME_SERV_UUID    0x1805 // 当前时间服务
#define REF_TIME_UPDATE_SERV_UUID 0x1806 // 参考时间更新服务
#define NEXT_DST_CHANGE_SERV_UUID 0x1807 // 下一次夏令时更改服务
#define GLUCOSE_SERV_UUID         0x1808 // 血糖
#define THERMOMETER_SERV_UUID     0x1809 // 健康温度计
#define DEVINFO_SERV_UUID         0x180A // 设备信息
#define NWA_SERV_UUID             0x180B // 网络可用性
#define HEARTRATE_SERV_UUID       0x180D // 心率
#define PHONE_ALERT_STS_SERV_UUID 0x180E // 电话警报状态服务
#define BATT_SERV_UUID            0x180F // 电池服务
#define BLOODPRESSURE_SERV_UUID   0x1810 // 血压
#define ALERT_NOTIF_SERV_UUID     0x1811 // 警报通知服务
#define HID_SERV_UUID             0x1812 // 人机接口设备
#define SCAN_PARAM_SERV_UUID      0x1813 // 扫描参数
#define RSC_SERV_UUID             0x1814 // 跑步速度和步频
#define CSC_SERV_UUID             0x1816 // 骑行速度和步频
#define CYCPWR_SERV_UUID          0x1818 // 骑行功率
#define LOC_NAV_SERV_UUID         0x1819 // 位置和导航
```

GATT 特征 UUID

```
#define GAP_DEVICE_NAME_UUID      0x2A00
#define GAP_APPEARANCE_UUID       0x2A01
#define GAP_PERIPHERAL_PRIVACY_FLAG 0x2A02
#define GAP_RECONNECTION_ADDRESS_UUID 0x2A03
#define GAP_PERIPHERAL_PREFERRED_CONNECTION_PARAMETERS_UUID 0x2A04
#define GAP_SERVICE_CHANGED       0x2A05
#define ALERT_LEVEL_UUID          0x2A06 // 警报级别
#define TX_PWR_LEVEL_UUID        0x2A07 // 发射功率级别
#define DATE_TIME_UUID           0x2A08 // 日期时间
#define DAY_OF_WEEK_UUID         0x2A09 // 星期几
#define DAY_DATE_TIME_UUID       0x2A0A // 日期时间 (天)
#define EXACT_TIME_256_UUID      0x2A0C // 精确时间 256
#define DST_OFFSET_UUID          0x2A0D // 夏令时偏移
#define TIME_ZONE_UUID           0x2A0E // 时区
#define LOCAL_TIME_INFO_UUID     0x2A0F // 本地时间信息
#define TIME_WITH_DST_UUID       0x2A11 // 夏令时时间
#define TIME_ACCURACY_UUID       0x2A12 // 时间准确性
#define TIME_SOURCE_UUID         0x2A13 // 时间源
#define REF_TIME_INFO_UUID       0x2A14 // 参考时间信息
#define TIME_UPDATE_CTRL_PT_UUID 0x2A16 // 时间更新控制点
#define TIME_UPDATE_STATE_UUID   0x2A17 // 时间更新状态
#define GLUCOSE_MEAS_UUID        0x2A18 // 血糖测量
#define BATT_LEVEL_UUID          0x2A19 // 电池电量
#define TEMP_MEAS_UUID           0x2A1C // 温度测量
#define TEMP_TYPE_UUID           0x2A1D // 温度类型
#define IMMEDIATE_TEMP_UUID      0x2A1E // 中间温度
#define MEAS_INTERVAL_UUID       0x2A21 // 测量间隔
#define BOOT_KEY_INPUT_UUID      0x2A22 // 启动键盘输入报告
#define SYSTEM_ID_UUID           0x2A23 // 系统 ID
#define MODEL_NUMBER_UUID        0x2A24 // 型号字符串
#define SERIAL_NUMBER_UUID       0x2A25 // 序列号字符串
#define FIRMWARE_REV_UUID        0x2A26 // 固件版本字符串
```

```
#define HARDWARE_REV_UUID          0x2A27 // 硬件版本字符串
#define SOFTWARE_REV_UUID          0x2A28 // 软件版本字符串
#define MANUFACTURER_NAME_UUID    0x2A29 // 制造商名称字符串
#define IEEE_11073_CERT_DATA_UUID 0x2A2A // IEEE 11073-20601 监管认证数据列表
#define CURRENT_TIME_UUID         0x2A2B // 当前时间
#define SCAN_REFRESH_UUID         0x2A31 // 扫描刷新
#define BOOT_KEY_OUTPUT_UUID      0x2A32 // 启动键盘输出报告
#define BOOT_MOUSE_INPUT_UUID     0x2A33 // 启动鼠标输入报告
#define GLUCOSE_CONTEXT_UUID      0x2A34 // 血糖测量上下文
#define BLOODPRESSURE_MEAS_UUID   0x2A35 // 血压测量
#define IMEDIATE_CUFF_PRESSURE_UUID 0x2A36 // 中间袖带压力
#define HEARTRATE_MEAS_UUID       0x2A37 // 心率测量
#define BODY_SENSOR_LOC_UUID      0x2A38 // 人体传感器位置
#define HEARTRATE_CTRL_PT_UUID    0x2A39 // 心率控制点
#define NETWORK_AVAIL_UUID        0x2A3E // 网络可用性
#define ALERT_STATUS_UUID         0x2A3F // 警报状态
#define RINGER_CTRL_PT_UUID       0x2A40 // 铃声控制点
#define RINGER_SETTING_UUID       0x2A41 // 铃声设置
#define ALERT_CAT_ID_BMASK_UUID   0x2A42 // 警报类别 ID 位掩码
#define ALERT_CAT_ID_UUID         0x2A43 // 警报类别 ID
#define ALERT_NOTIF_CTRL_PT_UUID  0x2A44 // 警报通知控制点
#define UNREAD_ALERT_STATUS_UUID  0x2A45 // 未读警报状态
#define NEW_ALERT_UUID            0x2A46 // 新警报
#define SUP_NEW_ALERT_CAT_UUID    0x2A47 // 支持的新警报类别
#define SUP_UNREAD_ALERT_CAT_UUID 0x2A48 // 支持的未读警报类别
#define BLOODPRESSURE_FEATURE_UUID 0x2A49 // 血压功能
#define HID_INFORMATION_UUID      0x2A4A // HID 信息
#define REPORT_MAP_UUID           0x2A4B // 报告映射
#define HID_CTRL_PT_UUID          0x2A4C // HID 控制点
#define REPORT_UUID               0x2A4D // 报告
#define PROTOCOL_MODE_UUID        0x2A4E // 协议模式
#define SCAN_INTERVAL_WINDOW_UUID 0x2A4F // 扫描间隔窗口
#define PNP_ID_UUID               0x2A50 // PnP ID
#define GLUCOSE_FEATURE_UUID       0x2A51 // 血糖功能
#define RECORD_CTRL_PT_UUID       0x2A52 // 记录访问控制点
#define RSC_MEAS_UUID             0x2A53 // RSC 测量
#define RSC_FEATURE_UUID          0x2A54 // RSC 功能
#define SC_CTRL_PT_UUID           0x2A55 // SC 控制点
#define CSC_MEAS_UUID             0x2A5B // CSC 测量
#define CSC_FEATURE_UUID          0x2A5C // CSC 功能
#define SENSOR_LOC_UUID           0x2A5D // 传感器位置
#define CYCPWR_MEAS_UUID          0x2A63 // 骑行功率测量
#define CYCPWR_VECTOR_UUID        0x2A64 // 骑行功率向量
#define CYCPWR_FEATURE_UUID       0x2A65 // 骑行功率功能
#define CYCPWR_CTRL_PT_UUID       0x2A66 // 骑行功率控制点
#define LOC_SPEED_UUID            0x2A67 // 位置和速度
#define NAV_UUID                  0x2A68 // 导航
#define POS_QUALITY_UUID          0x2A69 // 位置质量
#define LN_FEATURE_UUID           0x2A6A // LN 功能
#define LN_CTRL_PT_UUID           0x2A6B // LN 控制点
```

外观属性定义

```
#define GAP_APPEARE_UNKNOWN          0x0000 // 未知
#define GAP_APPEARE_GENERIC_PHONE    0x0040 // 通用电话
#define GAP_APPEARE_GENERIC_COMPUTER  0x0080 // 通用计算机
#define GAP_APPEARE_GENERIC_WATCH     0x00C0 // 通用手表
#define GAP_APPEARE_WATCH_SPORTS      0x00C1 // 手表: 运动手表
#define GAP_APPEARE_GENERIC_CLOCK     0x0100 // 通用时钟
#define GAP_APPEARE_GENERIC_DISPLAY   0x0140 // 通用显示器
#define GAP_APPEARE_GENERIC_RC        0x0180 // 通用遥控器
#define GAP_APPEARE_GENERIC_EYE_GLASSES 0x01C0 // 通用眼镜
#define GAP_APPEARE_GENERIC_TAG       0x0200 // 通用标签
#define GAP_APPEARE_GENERIC_KEYRING   0x0240 // 通用钥匙扣
#define GAP_APPEARE_GENERIC_MEDIA_PLAYER 0x0280 // 通用媒体播放器
#define GAP_APPEARE_GENERIC_BARCODE_SCANNER 0x02C0 // 通用条码扫描器
#define GAP_APPEARE_GENERIC_THERMOMETER 0x0300 // 通用温度计
#define GAP_APPEARE_GENERIC_THERMO_EAR 0x0301 // 温度计: 耳温枪
#define GAP_APPEARE_GENERIC_HR_SENSOR 0x0340 // 通用心率传感器
#define GAP_APPEARE_GENERIC_HRS_BELT   0x0341 // 心率传感器: 心率带
#define GAP_APPEARE_GENERIC_BLOOD_PRESSURE 0x0380 // 通用血压
#define GAP_APPEARE_GENERIC_BP_ARM     0x0381 // 血压: 手臂
#define GAP_APPEARE_GENERIC_BP_WRIST   0x0382 // 血压: 手腕
#define GAP_APPEARE_GENERIC_HID        0x03C0 // 人机接口设备 (HID)
#define GAP_APPEARE_HID_KEYBOARD       0x03C1 // HID 键盘
#define GAP_APPEARE_HID_MOUSE          0x03C2 // HID 鼠标
#define GAP_APPEARE_HID_JOYSTIC        0x03C3 // HID 操纵杆
#define GAP_APPEARE_HID_GAMEPAD        0x03C4 // HID 游戏手柄
#define GAP_APPEARE_HID_DIGITIZER_TABLET 0x03C5 // HID 数字化仪平板
#define GAP_APPEARE_HID_DIGITAL_CARDREADER 0x03C6 // HID 读卡器
#define GAP_APPEARE_HID_DIGITAL_PEN    0x03C7 // HID 数字笔
#define GAP_APPEARE_HID_BARCODE_SCANNER 0x03C8 // HID 条形码扫描器
```

Copyright© 2025 by HOLTEK SEMICONDUCTOR INC. All Rights Reserved.

本文件出版时 HOLTEK 已针对所载信息为合理注意，但不保证信息准确无误。文中提到的信息仅是提供作为参考，且可能被更新取代。HOLTEK 不担保任何明示、默示或法定的，包括但不限于适合商品化、令人满意的质量、规格、特性、功能与特定用途、不侵害第三方权利等保证责任。HOLTEK 就文中提到的信息及该信息之应用，不承担任何法律责任。此外，HOLTEK 并不推荐将 HOLTEK 的产品使用在会由于故障或其他原因而可能会对人身安全造成危害的地方。HOLTEK 特此声明，不授权将产品使用于救生、维生或安全关键零部件。在救生 / 维生或安全应用中使用 HOLTEK 产品的风险完全由买方承担，如因该等使用导致 HOLTEK 遭受损害、索赔、诉讼或产生费用，买方同意出面进行辩护、赔偿并使 HOLTEK 免受损害。HOLTEK (及其授权方，如适用) 拥有本文件所提供信息 (包括但不限于内容、数据、示例、材料、图形、商标) 的知识产权，且该信息受著作权法和其他知识产权法的保护。HOLTEK 在此并未明示或暗示授予任何知识产权。HOLTEK 拥有不事先通知而修改本文件所载信息的权利。如欲取得最新的信息，请与我们联系。